



SOFTWARE REFERENCE MANUAL

ENSEMBLE™ FAMILY EMBEDDED FUSION PROCESSORS AND MICROCONTROLLERS

Contents

1 Preface	1
2 Overview of the Device System Architecture	1
2.1 System Architecture	1
2.2 Secure Enclave Subsystem (SESS)	1
2.3 Application Processor Subsystem (APSS)	3
2.4 Real-Time Subsystem (RTSS)	4
2.5 Shared Memory	5
2.6 Shared Peripherals	8
2.6.1 Interrupt Requests Routing	9
2.6.2 Event Routing and DMA Requests Multiplexing	10
2.6.3 Peripherals Security	11
2.7 System Control	11
2.7.1 Inter-Process Communication with MHU	11
2.7.2 Inter-Process Synchronization with HWSEM	13
2.8 Power Management	14
2.9 System Startup and Control	14
2.9.1 Boot Process	14
2.9.2 Application Deployment in NVM	17
2.10 Additional Information	17
3 Secure Enclave Subsystem (SESS)	18
3.1 SESS Introduction	18
3.2 SEROM Bootloader	18
3.2.1 SEROM Boot Sequence	19
3.2.1.1 Initialization	19
3.2.2 BootROM_ColdBootSequence() Function	19
3.2.2.1 STOC Localization	19
3.2.2.2 Selected TOC Entry Processing	19
3.2.2.3 Certificate Chain Location and Identification	19
3.2.3 Certificate Chain Processing	19
3.2.3.1 Raw Loading	19
3.2.3.2 Certificate Chain Verification	20
3.2.3.3 Warm Boot	20
3.2.3.4 Cold Boot	20
3.2.4 Error and Panic Values	20
3.2.4.1 Error Status Values	20
3.2.4.2 Panic Code Values	21

3.2.5 Security Lifecycle States	22
3.2.5.1 Security Lifecycle States Overview	22
3.2.5.2 Chip Manufacturing LCS	22
3.2.5.3 Device Manufacturing LCS	22
3.2.5.4 Secure Enable LCS	22
3.2.5.5 Return Merchandise Authorization LCS	23
3.3 SES Code Image	23
3.3.1 APSS and RTSS Boot Up Sequence and ATOC	23
3.3.1.1 ATOC Attributes	23
3.3.1.2 NVM (MRAM) Storage	24
3.3.1.3 Boot Sequence	25
3.3.1.4 Cold Boot	25
3.3.1.5 Warm Boot	25
3.4 SETOOLS	26
3.5 SEROM UART Console Output	26
3.5.1 SEROM UART Banner Message	26
3.5.2 SEROM Factory Data Print (REV_A1)	26
3.5.3 SEROM Factory Data Print (REV_B)	26
3.5.4 SEROM UART Error and Warning Messages	26
3.5.5 Hard Fault Exceptions Display	27
3.5.6 Other Exception Handler Display Messages	27
3.5.7 Error Status Values	28
3.5.8 Panic Code Values	28
3.5.9 SEROM UART Console Configuration	28
3.5.10 Maintenance and Recovery Mode	28
3.6 SESS Message Handling Unit (MHU)	28
3.6.1 SESS MHU Overview	28
3.6.2 Service Request Mechanism	29
3.6.3 Runtime Services	29
3.6.3.1 Application Services	29
3.6.3.2 Power Management Services	29
3.6.3.3 System Security Services	29
4 Ensemble Application Anatomy	30
4.1 Application Architecture	30
4.2 Application Source Code Organization	32
4.3 CMSIS-Packs and Header Files	33
4.4 Compiler Libraries	35

4.5 Project Settings	35
4.6 Linker Script	38
4.7 Applications Table of Contents (ATOC)	38
4.8 Debug Configurations and Application Reset Methods	41
5 Integrated Design Environments and Toolchains	44
6 Application Development Workflow	45
6.1 Create Device Configuration File	45
6.1.1 DevKit and AppKit Configuration	45
6.1.2 Production Device Configuration Using the Conductor Tool	45
6.1.2.1 Resources Configuration Management	45
6.1.2.2 Pins Configuration Management	45
6.1.2.3 Clocks Configuration Management	46
6.2 Create or Import Project Files into the IDE	46
6.3 Create Debug Configuration	46
6.4 Compile and Build Application	46
6.5 Debugging a Single Core Application Via Debugger	47
6.6 Using NVM Storage	47
6.6.1 Generate New RoT if Necessary	47
6.6.2 Creating Application Package Binary TOCs	47
6.6.3 Programming the MRAM Memory	48
6.7 Additional Information	48
7 Supported Software Device Drivers	49
7.1 List of Device Drivers Supported in the APSS and RTSS	49
7.1.1 Message Handling Unit (MHU)	49
7.1.1.1 MHU Overview	49
7.1.1.2 Inter-Process Communication (IPC) with MHU	50
7.1.1.3 IPC Topology Using MHU	50
7.1.2 Hardware Semaphore (HWSEM)	51
7.1.2.1 HWSEM Overview	51
7.1.2.2 HWSEM Design	51
7.1.2.3 Inter-Process Synchronization with HWSEM	52
7.1.2.4 HWSEM Example Scenario	52
7.1.3 Ethernet (ETH)	53
7.1.3.1 ETH Overview	53
7.1.4 Low-Power Real-Time Counter (LPRTC)	55
7.1.4.1 LPRTC Overview	55
7.1.5 Digital-to-Analog Convertor (DAC)	56

7.1.5.1 DAC Overview	56
7.1.5.2 DAC Design	56
7.1.6 Octal SPI (OSPI)	56
7.1.6.1 OSPI Overview	56
7.1.6.2 OSPI Design	57
7.1.6.3 OSPI with AES	57
7.1.7 Watchdog Timer (WDT_APSS)	58
7.1.7.1 WDT_APSS Overview	58
7.1.8 USB Device	59
7.1.8.1 USB Overview	59
7.1.8.2 Dual-Role USB	60
7.1.9 USB Host	60
7.1.9.1 USB Host Overview	60
7.1.10 USB Host or Device Controller	61
7.1.11 Secure Digital / MultiMediaCard (SDMMC)	62
7.1.11.1 SDMMC Overview	62
7.1.11.2 SDMMC Interfaces	62
7.1.11.3 System-Level Block Diagram	63
7.1.12 Camera	64
7.1.12.1 Camera Overview	64
7.1.12.2 Camera Parallel Interface (CPI)	65
7.1.12.3 MIPI Camera Serial Interface 2 (CSI)	65
8 Linux Software Development Kit (SDK) Setup	68
8.1 Linux SDK Architecture Overview	68
8.2 Memory Architecture Used by the APSS	70
8.3 APSS Boot Up	70
9 Alif Semiconductor RTSS SDK	73
9.1 RTSS SDK Components	73
9.2 Generic CMSIS Components	73
9.2.1 RTE Device and RTE Components	73
9.2.2 RTOS Project Configuration Using Conductor Tool	73
9.2.3 RTOS Application Software Code/Libraries	73
9.2.4 RTOS Software Project	74
9.2.5 RTOS Executable Image	74
9.3 Using SDK to Create/Compile an RTOS Project in an IDE	74
9.4 RTOS Memory Regions and Predefined Linker Script	74
9.5 RTOS System Startup	74

9.6 RTSS SDK Drivers	74
9.6.1 RTSS SDK Architecture	74
9.6.2 CMSIS-Driver Overview	74
9.6.3 RTSS SDK Custom CMSIS Drivers	75
9.6.4 RTSS SDK Configuration	75
10 Azure RTOS Software Development Kit	77
10.1 Azure RTOS SDK Components	77
10.2 Azure RTOS Middleware Support	77
10.2.1 Azure RTOS ThreadX	77
10.2.2 NetX Duo	78
10.2.2.1 NetX Duo Support in Alif Semiconductor RTSS SDK	78
10.2.3 FileX	79
10.2.4 GUIX	80
10.2.4.1 Azure RTOS GUIX Studio	81
10.2.5 USBX	81
10.2.5.1 USBX Stack Layout	82
10.2.5.2 USBX Support for Ensemble DevKit/AppKit	83
10.2.5.3 USBX Working in Device/Host Mode	83
10.3 Getting Started App Notes for Azure RTOS	84
11 FreeRTOS Software Development Kit	85
11.1 FreeRTOS SDK Components	85
11.2 FreeRTOS Middleware Support	85
11.2.1 FreeRTOS	85
11.2.1.1 FreeRTOSConfig.h	85
11.2.1.2 FreeRTOS Common Source Files	86
11.2.1.3 FreeRTOS Port Specific Source Files	86
11.2.2 FreeRTOS+TCP	86
11.2.2.1 Architecture	86
11.2.2.2 Network Interface Layer	87
11.2.2.3 Network Buffer Allocation/Management	87
11.2.2.4 Setting Up and Configuring a FreeRTOS+TCP Application	88
11.2.3 FreeRTOS+FAT	88
11.2.3.1 FreeRTOS+FAT Configuration	88
11.2.4 Graphics Middleware Support	88
11.2.5 USB Middleware Support	89
11.2.5.1 TinyUSB	89
11.2.5.2 Application	89

11.2.5.3 TinyUSB Stack	89
11.2.5.4 TinyUSB Support for Alif Semiconductor Ensemble DevKit/AppKit	90
11.2.5.5 TinyUSB Working in Device/Host Mode	90
11.2.5.6 TinyUSB Device Stack	91
12 Zephyr Alif SDK (ZAS)	92
12.1 Zephyr Introduction	92
12.2 Overview of Zephyr Alif SDK	92
12.3 ZAS Repositories	92
12.3.1 Zephyr	92
12.3.2 HAL	92
12.3.3 sdk-alif	92
12.4 mcuboot_alif	93
12.5 cmsis_alif	93
13 Legal and Support Information	94
13.1 Disclaimers	94
13.2 Related Documents and Tools	94
13.3 Contact Information	94
13.4 Trademarks	95
13.5 Abbreviations	95
14 Revision History	113

1 Preface

The Ensemble™ family of fusion processors and microcontrollers from Alif Semiconductor creates a scalable and compatible continuum of highly integrated processors for use in intelligent embedded Internet of Things (IoT) applications. Architected for power efficiency and long battery life, it delivers high computation and Machine Learning / Artificial Intelligence (ML/AI) imaging, audio and sensing capability, wired multi-layered security, and highly interactive human-machine interface.

Individual selections within the Ensemble family of devices scale up, starting with single-core MCUs, to dual-core MCUs, triple-core MCU/MPU fusion processors, and quad-core MCU/MPU fusion processors, to meet the needs of a broad range of IoT applications. The devices share a common set of peripherals and power management schemes and are built on a unified interconnection fabric that simplifies software and hardware reuse across diverse projects.

The following table is a top-level feature view of the device family:

Family Name	Quad-Core Fusion Processor Series	Triple-Core Fusion Processor Series	Dual-Core MCU Series	Single-Core MCU Series	Connectivity
Ensemble	E7	E5	E3	E1	Wired

This document introduces the system architecture that the [Ensemble family](#) of devices is built on, and their programming models. It includes descriptions of the various software execution environments, memories, peripherals, and the connections between them. This document also covers application development, debugging, and deployment workflows. Lastly, it provides an overview of the boot process, security features, Inter-Process Communication (IPC), and power management options.

For detailed specifications, functional descriptions, and device features and peripherals supported, refer to the device series-specific Datasheet and Hardware Reference Manual.

2 Overview of the Device System Architecture

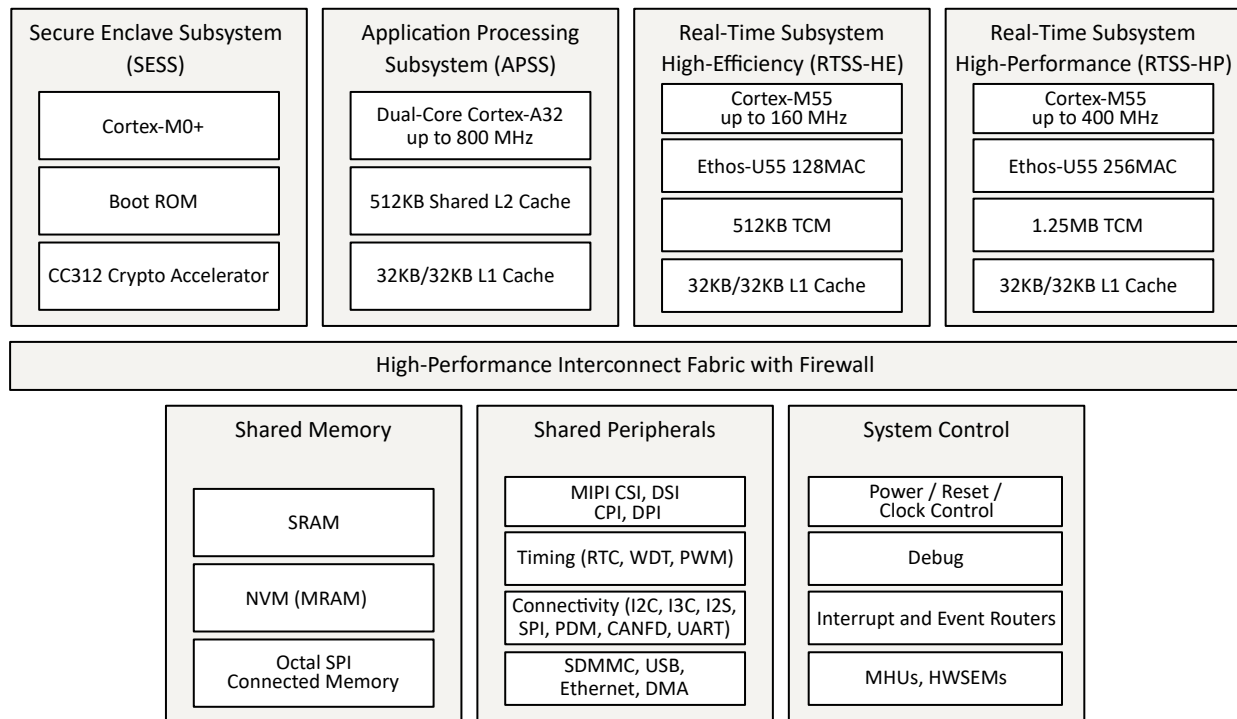
2.1 System Architecture

Embedded system designers need to balance many, often contradictory requirements: high-performance vs. high-efficiency, security vs. application flexibility, device versatility vs. low latency. Alif Semiconductor's system architecture is designed with the goal of creating powerful and highly scalable solutions that efficiently address these challenges.

The system architecture is comprised of multiple subsystems optimized for specific roles. Each subsystem has one or two CPUs, an optional Neural Processor Unit (NPU), memory, and peripherals. Each subsystem can execute code independently of others, but may share a power domain and can access shared memory and peripherals. The subsystems running in a specific power domain are referred in this document as execution domains. Each of them can be considered as a microcontroller optimized for a specific class of embedded applications.

The system architecture includes the infrastructure required to integrate and manage the different execution domains (see [Figure 2-1](#)). The backbone of the system interconnect is based on a high-performance interconnect fabric. It is secured by a number of Firewalls that monitor all master and slave ports on the bus. The Firewalls can filter transactions based on the desired security policy and perform address translation when an execution domain requires a different view of the global address map.

Figure 2-1 Top-Level System Architecture for E7 Devices



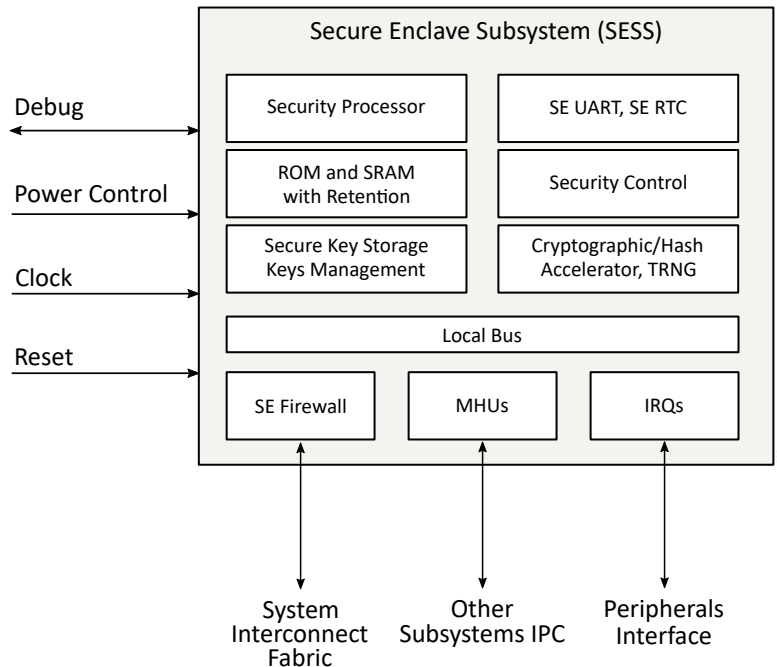
108

2.2 Secure Enclave Subsystem (SESS)

The SESS has the role of a system supervisor. It is isolated from the bus fabric and is not accessible for running application code. The SESS performs the initial system configuration after reset, securely boots

application subsystems, and responds to service requests from them afterwards. Figure 2-2 shows the SESS structure.

Figure 2-2 SESS Architecture for All Devices



The SESS has its own ROM, SRAM, and security processor. It also includes a True Random Number Generator (TRNG), along with hardware accelerators for various cryptographic functions, calculating hash codes and performing signature checks. The SESS also manages the immutable secure key storage.

The SESS is a bus master on the main system interconnect and does not expose any of its internals to the application cores. Service requests to the SESS are communicated over a dedicated IPC logic called the Message Handling Unit (MHU), which provides message passing between processors residing in different power and clock domains.

The SESS functions and services are summarized below:

- Device Startup and Configuration
 - Secure First and Second Stage Bootloaders (SEROM and SERAM, respectively)
 - Device Configuration
 - Secure Firmware Update
- Application Startup Management
 - Secure Boot Application Subsystems
 - On-demand Subsystem Boot
- Application Services
 - Pins Configuration
 - Interrupt Router Configuration
 - Event Router Configuration
 - Clock Generation Configuration
- Power/Reset Services
 - STOP Mode Entry and Wakeup Logic
 - Associating Wakeup Events and CPU Subsystems
 - DC-DC Dynamic Operation Control
- System Security Services

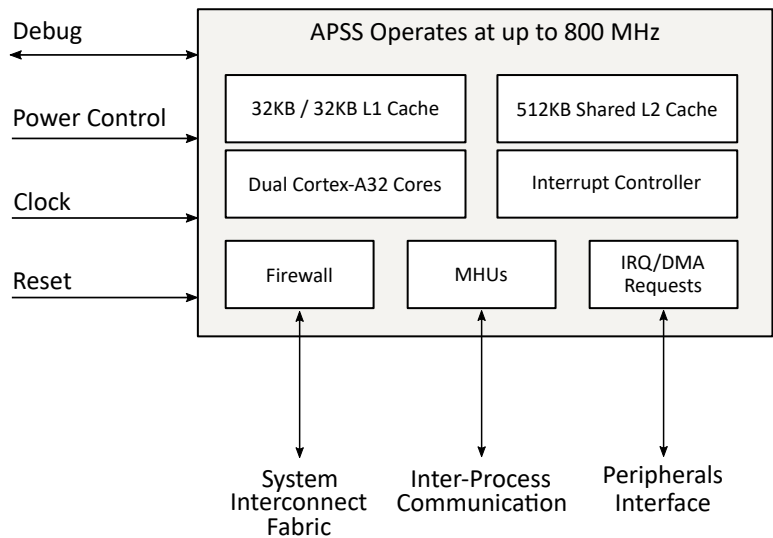
- Root-of-Trust (Private Key Storage, Public Key Report)
- Key Management
- Device Identity and Authentication
- Lifecycle Management / eFuse Control
- TRNG Functions
- Cryptographic Functions
- Signature Verification
- Debug Session Authentication
- Infrastructure Utilities
 - Secure Enclave (SE) Maintenance Functions (Provision OTP, ISP MRAM)
 - Alif Semiconductor's Firmware Update Service
 - MHU Driver/Service Dispatcher
 - Octal SPI (OSPI) AES Keys Management

2.3 Application Processor Subsystem (APSS)

The APSS hosts up to two Arm Cortex-A32 cores outfitted with an ample amount of cache. Each Cortex-A32 core has a 32KB instruction and a 32KB data Level 1 cache. Both cores share 512KB of Level 2 cache with a built-in cache coherency controller. The cache size is chosen to maximize processor performance, given that the system memory blocks have different and limited bandwidth. This approach enables balance between performance and energy efficiency.

The APSS is optimized for running Linux-based embedded applications that can benefit from the power efficiency and high-performance of the Cortex-A32 cores (see [Figure 2-3](#)).

Figure 2-3 Application Processor Subsystem Architecture for E7 Devices



The APSS includes an interrupt controller that handles interrupt requests from local and shared peripherals. The local peripherals include MHU that enables IPC to the SESS and to the other subsystems.

The Cortex-A32 cores include a full-featured Memory Management Unit (MMU), enabling execution of a Linux® operating system. The Linux kernel can be configured with Symmetric Multiprocessing (SMP) support, making most of the dual-core performance capabilities.

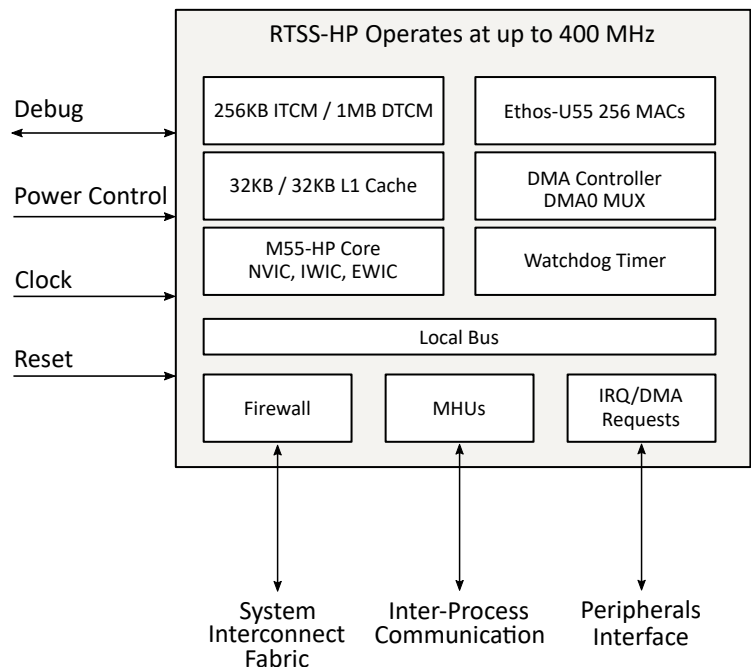
Alternatively, the Cortex-A32 cores can be utilized by several SMP-capable Real-Time Operating Systems (RTOS), such as Azure RTOS (formerly ThreadX).

2.4 Real-Time Subsystem (RTSS)

The Ensemble family of devices has two RTSSs (except for E1 devices, which have one), each featuring a Cortex-M55 CPU with Tightly Coupled Memory (TCM) and, optionally, one or two Ethos-U55 microNPUs (see [Figure 2-4](#)). The High-Performance RTSS (RTSS-HP) is optimized for fast compute performance, and the High-Efficiency RTSS (RTSS-HE) is optimized for power-sensitive use cases. Both RTSS-HP and RTSS-HE enable highly deterministic control or data collection applications. Both RTSS also feature MHUs that facilitate IPC, and programmable Direct Memory Access (DMA) controllers that off-load data movement tasks.

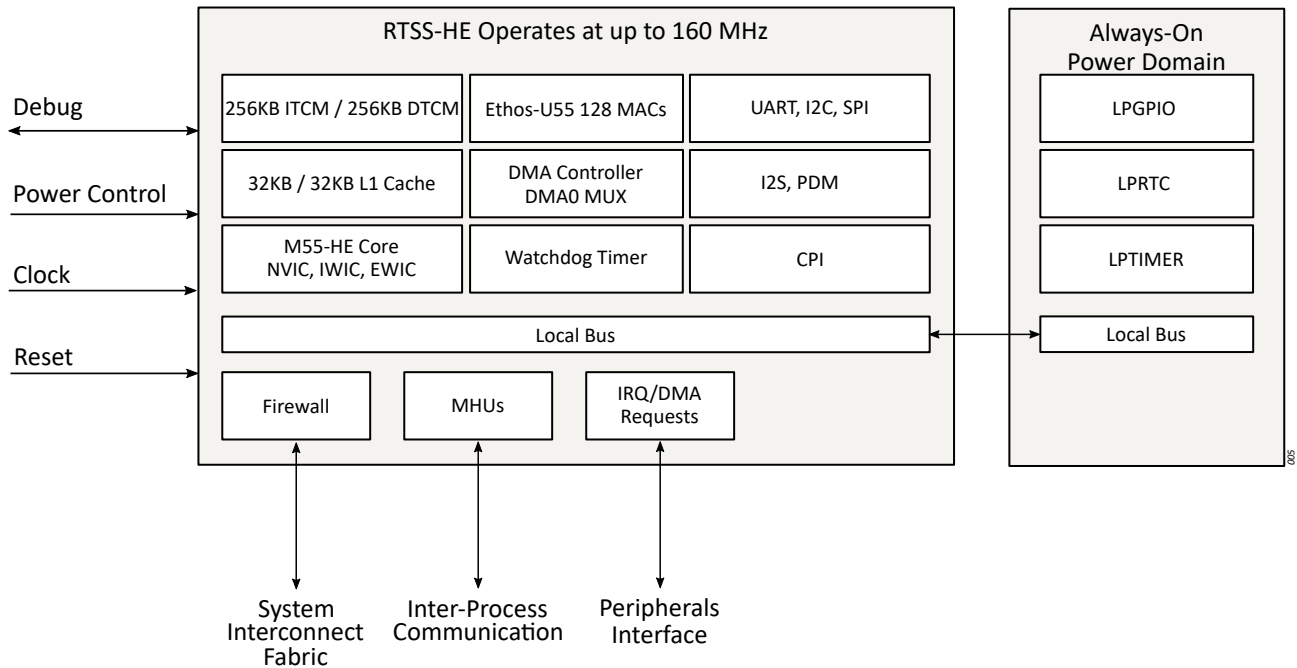
The Cortex-M55 in the RTSS-HP can operate at up to 400 MHz core clock frequency. The single-cycle 256KB Instruction Tightly Coupled Memory (ITCM) and 1MB Data Tightly Coupled Memory (DTCM) enable a highly deterministic, high-performance execution environment capable of running tight control loops with minimal jitter. The RTSS-HP also has its own Ethos-U55 microNPU ML accelerator with capacity of 256 MACs. The faster clock frequency, larger TCM capacity, and increased number of MAC(s) in the RTSS-HP provide resources for implementation of advanced neural networks and faster, more accurate inference execution.

Figure 2-4 RTSS-HP Architecture for E3-E5-E7 Devices



The implementation of the Cortex-M55 in the RTSS-HE is optimized for low current consumption while running at up to 160 MHz, and for extremely low leakage current while in STANDBY mode. This Cortex-M55 is equipped with 256KB ITCM and 256KB DTCM for fast single-cycle memory access (see [Figure 2-5](#)). The RTSS-HE resides in its own power domain and can operate standalone with minimum dependency on the other subsystems. Both the ITCM and DTCM contents can be optionally retained by a dedicated back-up power rail when the entire RTSS-HE power domain is powered off, enabling rapid wakeup. The RTSS-HE has a set of low power peripherals that can serve as a wakeup sources and that support ultra-low power application requirements. The Ethos-U55 microNPU includes 128 MACs and can be used for ML-based applications such as keyword spotting, simple image classification, anomaly detection, failure prediction, environmental sensing, and gesture recognition.

Figure 2-5 RTSS-HE Architecture



2.5 Shared Memory

The Ensemble family of devices integrates large amounts of SRAM distributed across multiple SRAM blocks, each with different performance, bandwidth, and power consumption. This is done to enable users with full flexibility to partition the memory among the various execution domains based on the needs of their application.

The different types of memory blocks and their characteristics are described in [Table 2-1](#).

Table 2-1 Shared Memory Distribution

Type	Description
Shared SRAM	The bulk of the SRAM is split between two blocks—SRAM0 and SRAM1 . They are shared with all cores. This enables the concurrent access to the memories from different execution domains.
Backup SRAM	The backup SRAM resides in a power domain powered by the VBAT power rail. This SRAM can be used to retain the system state needed to control different system wakeup scenarios. The backup SRAM is 4KB and can only hold data. The CPU cores cannot execute code from it. Read or write accesses to the Backup SRAM memory should be 32-bit wide.
Non-Volatile Memory (NVM)	The Ensemble family of devices uses non-volatile storage based on Magnetoresistive RAM (MRAM) technology. Unlike traditional flash, it allows fine-grained programming of memory sectors only 16 bytes in size. Applications can treat the MRAM as a regular RAM if the write operation consists of four 32-bit sequential words. The MRAM programming procedure is controlled by a dedicated state machine that is transparent to the application.
External Memory	The on-chip memory capacity can be expanded through one or two high-performance OSPI interfaces. OSPI supports an eXecute-in-Place (XIP) mode of operation that maps the flash memory content to an address region. When a bus master requests a read

	operation from this region, the OSPI interface translates it to a series of transactions with the external flash device and returns the result as if the read operation takes place from internal memory. This makes the access to OSPI flash transparent to the CPU cores. The XIP mode can be used to run larger software components, such as Linux, directly from external memory. This can be achieved by configuring and building the Linux kernel with XIP support, and by using a suitable filesystem, such as <i>cramfs</i>, that supports the XIP model for applications. The OSPI peripheral also supports the HyperBus protocol, enabling the expansion of on-chip RAM. Like with the XIP mode of operation, read and write bus operations are translated to HyperBus commands transparently to the bus master originating the transaction. Since adding external flash memory through OSPI exposes the data being transferred, the devices feature an AES-256 hardware decryption module on the data path between the OSPI peripheral and the system interconnect. Users can install their own AES key intended to decrypt the content of the external memory on-the-fly.
External Mass Storage	External mass storage can be added through an SDMMC controller. It supports 4-bit data interface to external SD/MMC cards, as well as an 8-bit interface for embedded MMC modules. Alternatively, the USB controller can be configured to operate in Host mode. This way, the storage capacity can be expanded using an external USB flash drive or other mass storage device.
Cache Memory	The cache memory accelerates the performance of the CPU cores in the APSS and RTSS execution domains. Devices with dual Arm Cortex-A32 cores each have 32KB L1 instruction and data cache running at up to 800 MHz. They share the common L2 cache, which is 512KB and is running at up to 400 MHz. The single core devices still utilize the full 512KB L2 cache. The Cortex-M55 cores in the RTSS execution domains have 32KB L1 instruction and data caches that are used only when accessing memory outside of the ITCM or the DTCM.
Tightly Coupled Memory	The RTSS-HP and RTSS-HE feature TCMs, providing high-bandwidth, deterministic, single-cycle access by the CPU. The TCM is split between Instruction and Data segments, each having a dedicated bus enabling their simultaneous access. The ITCM is always mapped at address 0x0000_0000 of the Cortex-M55 address space, and the DTCM is mapped at address 0x2000_0000. In addition, the TCMs are shared with the other CPU cores by being mapped to the global address space. The TCM memories and their corresponding address regions are described in Table 2-2.

The full address map is series-specific. For more information, see the series-specific Hardware Reference Manual.

[Table 2-2](#) describes the global address map from the perspective of the different processor subsystems in Ensemble E7 device.

Table 2-2 Memory Map for E7 Device

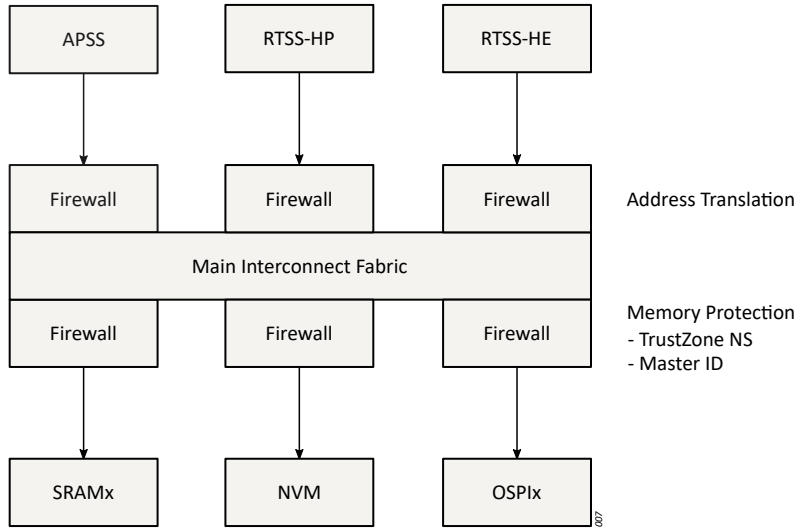
Address (hex)	Memory Size	RTSS-HP, RTSS-HE (M55)	APSS (A32)
0x0000_0000	256KB	Private ITCM Access	Boot Register
0x0200_0000	4MB	SRAM0	
0x0800_0000	2.5MB	SRAM1	

Address (hex)	Memory Size	RTSS-HP, RTSS-HE (M55)	APSS (A32)
0x1000_0000	N/A	Debug Peripherals	Debug Peripherals
0x1A00_0000	N/A	Host AON Peripherals	
0x1A02_0000	N/A	Power Control, Firewall, MHUs	
0x1A60_0000	N/A	AON Peripherals	
0x2000_0000	1MB/256KB	Private DTCM Access: 1MB (RTSS-HP); 256KB (RTSS-HE)	Not Accessible
0x4000_0000	N/A	Local Peripherals for RTSS	
0x4800_0000	N/A	Shared Peripherals	
0x5000_0000	256KB	SRAM2 (ITCM M55-HP Accessible Globally)	
0x5080_0000	1MB	SRAM3 (DTCM M55-HP Accessible Globally)	
0x5800_0000	256KB	SRAM4 (ITCM M55-HE Accessible Globally)	
0x5880_0000	256KB	SRAM5 (DTCM M55-HE Accessible Globally)	
0x6200_0000	1MB	SRAM6_A	
0x6240_0000	1MB	SRAM6_B	
0x6300_0000	512KB	SRAM7	
0x6320_0000	2MB	SRAM8	
0x6000_0000	256KB	SRAM9_A	
0x6010_0000	512KB	SRAM9_B	
0x4300_0000	N/A	Low Power Peripherals	
0x8000_0000	6MB	MRAM—On-chip Non-Volatile Memory	
0xA000_0000	256MB	Octal SPI0—External Flash Memory, HyperBus PSRAM	
0xC000_0000	256MB	Octal SPI1—External Flash Memory, HyperBus PSRAM	

The gray cells in the above table indicate areas that are not accessible.

The address ranges within SRAM0 and SRAM1 can be consolidated by the MMU configuration in the APSS, so that it appears as contiguous region. The Ensemble family of devices features Firewalls that secure transactions between master and slave ports across the main interconnect fabric (see [Figure 2-6](#)). The Firewalls connecting the bus masters in the RTSS-HP and RTSS-HE domains can do address translation of all read and write transactions. Similar to the MMU in the Cortex-A32 cores, this feature allows the Cortex-M55 CPUs or the Ethos-U55 microNPU to view fragmented SRAM blocks as a single SRAM region.

Figure 2-6 Firewall Architecture for E5-E7 Devices



The Firewalls are Arm TrustZone®-aware, and perform the role of a Memory Protection Controller for each of the shared SRAM blocks. In addition to protecting secure regions from non-secure transactions, the Firewalls can filter transactions based on specific bus master IDs. This way, applications running on the different subsystems can have private secure memory regions carved out of the globally accessible shared memory.

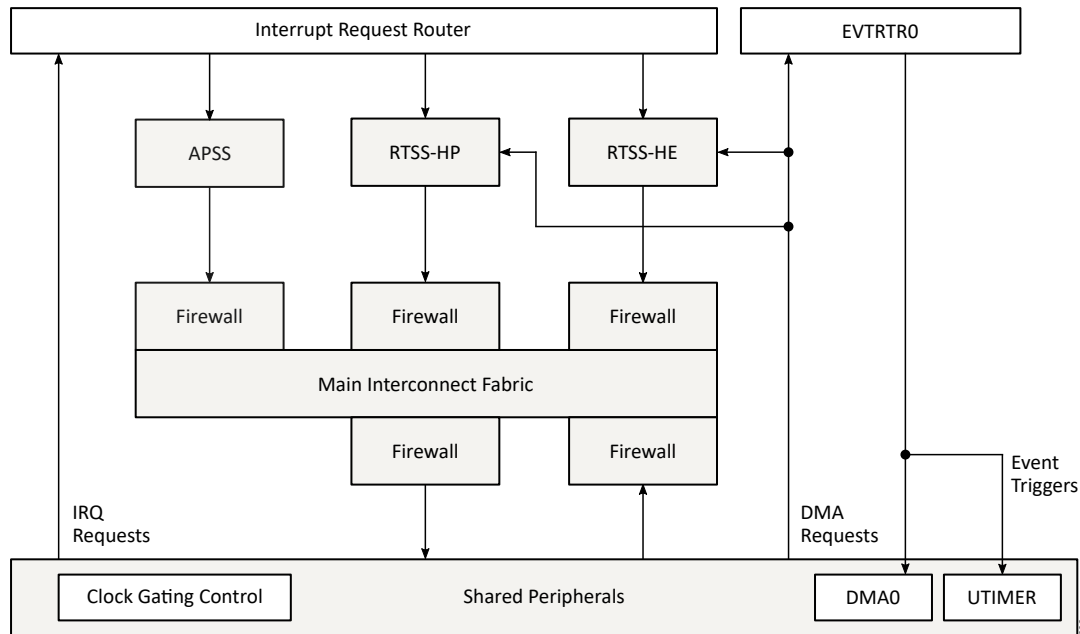
2.6 Shared Peripherals

The Ensemble family of devices integrates a rich set of peripherals, such as communication interfaces, memory expansion interfaces, display and camera controllers, timers, analog functions, and digital I/Os. For more information on the peripherals available in a particular device, refer to the *Functional Overview* section in the device series-specific Datasheet.

The family's system architecture is designed such that peripherals can be shared between the different subsystems (see [Figure 2-7](#)). This is achieved using the following components:

- **Interrupt Router:** Configures which subsystem should receive the interrupt requests coming from the shared peripherals.
- **Event Router / DMA0 Multiplexer:** Configures which DMA channel should receive the requests coming from the shared peripherals.
- **Clock Generation Controller:** Configures which of the peripherals have their clocks gated for reducing the active power consumption. In addition, it configures the clock sources and clock dividers required to meet different communication protocol specifications.
- **Firewalls:** Configure the security policy enabling access to specific bus masters. The Firewalls perform the role of a TrustZone-compliant Peripheral Protection Controller.

Figure 2-7 Protected Sharing of Peripherals for E5-E7 Devices



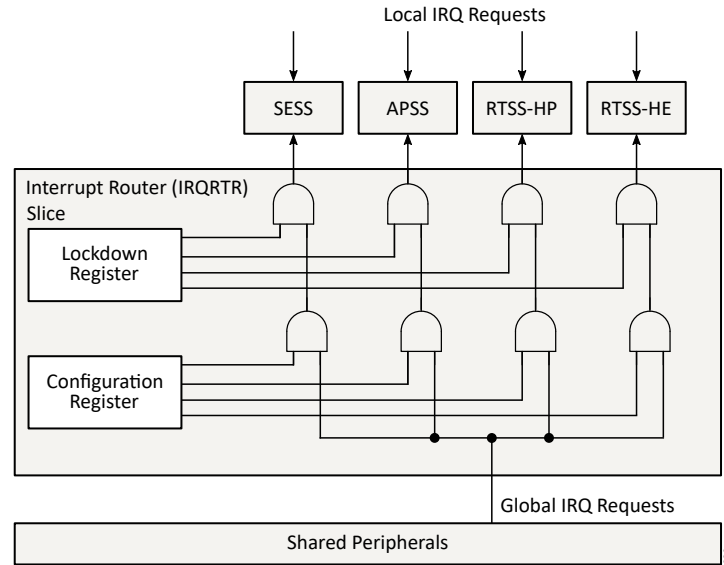
The access to above-mentioned components is restricted based on the Firewall configuration when the device is in the secure state. When the lifecycle of the device is advanced to the secured state, the association between peripherals and subsystems can only be controlled by the Secure Enclave (SE) in response to application requests. Requests are only executed if they do not contradict the security policy or another prior request.

2.6.1 Interrupt Requests Routing

The association between interrupt requests and the interrupt controllers in the individual subsystems is configurable at runtime. This function is facilitated by dedicated Interrupt Router channels with the structure shown in [Figure 2-8](#).

The figure describes how each interrupt request is distributed to the different device subsystems. The Configuration and the Lockdown registers enable different use cases including one-to-many and one-to-one associations based on the peripheral specifics and the application needs.

Figure 2-8 Interrupt Requests Distribution for E5-E7 Devices



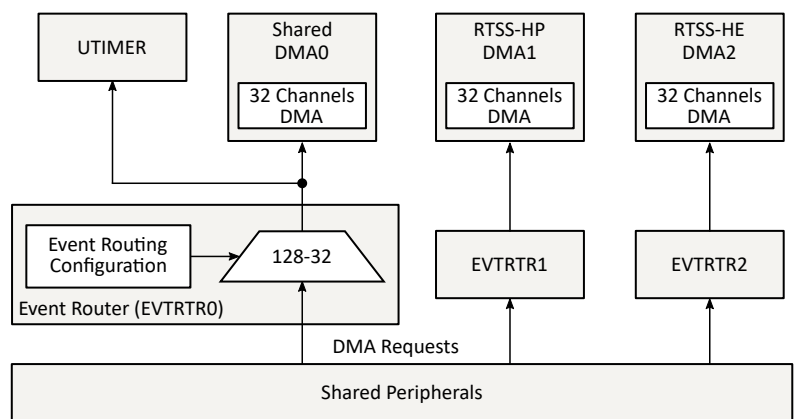
In secure mode of operation, the control of Lockdown and Configuration registers is only accessible to the SE. Applications request to set the Configuration register when they want to allocate a peripheral for use by invoking SE service.

2.6.2 Event Routing and DMA Requests Multiplexing

The Ensemble family of devices has multiple DMA controllers intended to off-load the processor data movement tasks (see [Figure 2-9](#)). High-speed peripherals such as Ethernet, USB, and SDMMC have their own dedicated DMA controllers.

The general-purpose DMA controllers (DMA0, DMA1, and DMA2), each having 32 inputs, can be served by 8 independent threads. They run at the same priority using round-robin scheduling. The threads can be configured as Secure or Non-Secure in compliance with the TrustZone technology.

Figure 2-9 General-Purpose DMA Controllers for All Devices



DMA0 expands the available 32 inputs with a dedicated multiplexer (EVTRTR0), that exposes the available DMA channels to 128 different peripherals events. These events can include requests from peripherals, such as UART or I2S, as well as GPIO transitions, or timer outputs. The first 16 multiplexed DMA0 requests are also routed as input triggers to the Universal Timer (UTIMER). This mechanism enables the implementation of

complex state machines involving the use of peripheral or time-based events that trigger data transfers and/or timer triggers.

The DMA0 controller is available for use by any of the processing subsystems. Its interrupt request lines (one for each of the 32 channels) are shared through the IRQRTR.

DMA1 and DMA2 are dedicated for use in RTSS-HP and RTSS-HE, respectively. They inherit the bus master ID of the subsystem that hosts them so that they can access the different system resources on behalf of the associated CPU core.

2.6.3 Peripherals Security

In secure state, the access to the peripheral registers is controlled by Firewalls (similarly to the access control of the shared memory regions). The Firewalls perform the role of Peripheral Protection Controller described in the TrustZone documentation. The peripheral protection is set by the SE in response to application requests to assume control over specific peripheral. This is done by preventing unauthorized access—excluding the transactions with invalid master ID and a Non-Secure (NS) side signal.

Peripherals in the Ensemble family of devices are designed to support low-power operation and to minimize energy consumption when not in use. Each peripheral instance has a clock gating circuit controlled by dedicated register bits. The clock gating control has two register stages—one controlled by the applications subsystems and another—by the SE. In secure state, clock gating changes are only enabled indirectly through dedicated service requests executed by the SE.

Ensemble family of devices uses a pin multiplexing scheme, as the number of peripherals with interface signals can exceed the number of available pins on the device package. The pin multiplexer enables the selection of up to eight different functions per pin. Like the clock gating control implementation, in secure state, the pin multiplexing is only possible by dedicated service requests executed by the SE.

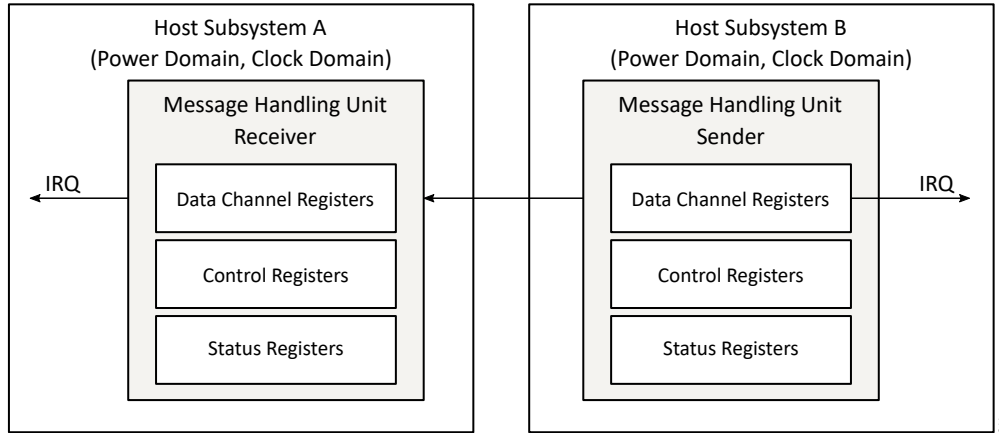
2.7 System Control

Ensemble family of devices integrates multiple application subsystems, capable of simultaneously executing different embedded applications independently. These embedded applications require mechanisms to let them communicate with one another. The MHU and the Hardware Semaphore (HWSEM) address the need for communication and synchronization in multi-core systems.

2.7.1 Inter-Process Communication with MHU

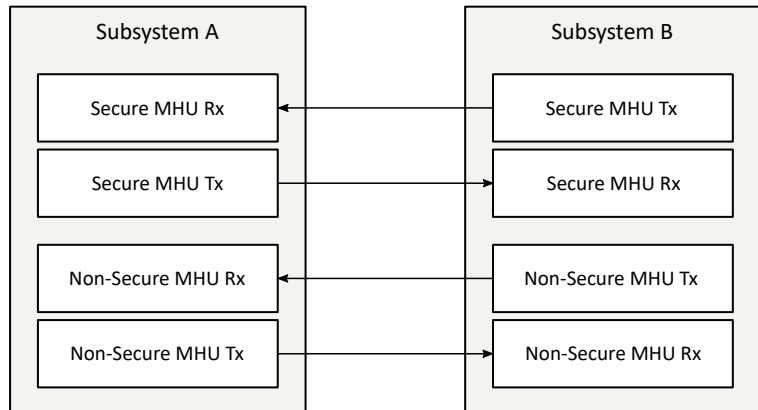
The MHUs provide interrupt-driven communication between two subsystems. They support the crossing of power and clock domains (see [Figure 2-10](#)). The MHU is a one-direction, peer-to-peer communication mechanism. The communication channel is represented by a 32-bit register which can be used to pass a single-word message or 32 single-bit messages. The interpretation of the data content is protocol specific.

Figure 2-10 Example of MHU Communication for All Devices



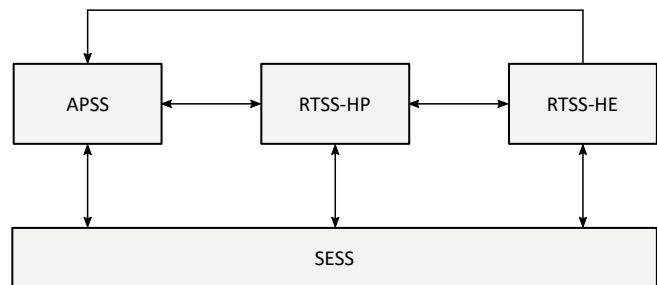
Ensemble family of devices uses two pairs of MHUs to support bi-directional IPC between processing elements. For compliance with the TrustZone technology, the number of MHUs is doubled—enabling both dedicated Secure and Non-Secure channels. In general, the IPC mechanism between any two subsystems is based on up to eight MHU instances organized in two full-duplex channels as shown in [Figure 2-11](#).

Figure 2-11 TrustZone-Compliant Use of MHUs for All Devices



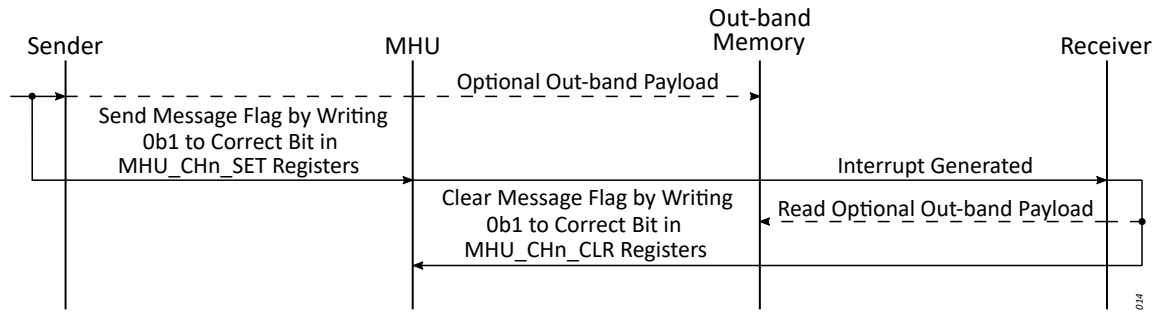
Ensemble family of devices integrates four subsystems having the IPC topology shown in [Figure 2-12](#).

Figure 2-12 IPC Topology for E5-E7 Devices



MHUs can support various transport protocols. [Figure 2-13](#) describes a simple Doorbell protocol that uses the Receiver interrupt only to send an optional payload.

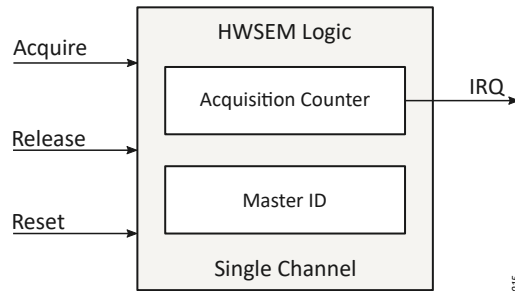
Figure 2-13 MHU Transport Protocol Example for All Devices



2.7.2 Inter-Process Synchronization with HWSEM

The HWSEM addresses the need of synchronizing the access to shared resources (memory or peripherals) by two or more independent processing elements. The device includes HWSEM operating like the counting semaphores found in most operating systems that support preemptive multi-threading. [Figure 2-14](#) represents the simplified structure of a HWSEM instance.

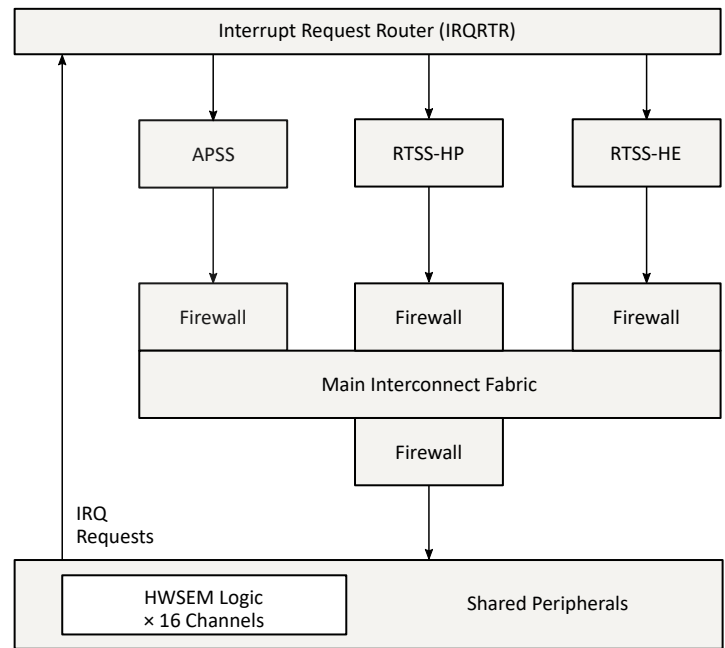
Figure 2-14 Simplified Structure of HWSEM for All Devices



The HWSEMs perform atomic operations, serializing the concurrent requests from different bus masters. This is achieved by register read/write operations resulting in counting the times the semaphore is acquired and released by a specific bus master. A HWSEM will refuse acquisition requests from other bus masters until the HWSEM is released. Once the acquisition counter reaches zero, the HWSEM release is signaled by an interrupt request (IRQ). The processors waiting to acquire the semaphore can handle this IRQ by initiating new acquisition request.

Sixteen instances of the HWSEM Logic are included among the shared peripherals. Their interrupt requests are shared with all subsystems through the IRQRTR as shown in [Figure 2-15](#).

Figure 2-15 HWSEM Shared Interrupt Requests for E5-E7 Devices



2.8 Power Management

The Ensemble family of fusion processors and microcontrollers features various independent power domains, each with a dedicated power controller coordinated by digital logic and software configuration for automatic transitions—from power-up to full GO mode, and all modes in between. This scheme achieves optimum power efficiency based on the dynamic power demand of the device at any given moment in time.

For more information on the power-saving features, refer to white paper [Alif Processors Power Saving Features](#).

2.9 System Startup and Control

2.9.1 Boot Process

The SESS performs the role of a system supervisor. After reset, the SESS is the only core that starts execution. The SESS initializes the device, sets up the security policies and performs a secure boot for the other subsystems. Once the boot is complete, the SE offers runtime services for security, crypto and system configuration.

The boot process starts with the SESS executing boot code stored in its local ROM. Its purpose is to perform a basic system initialization and to identify the system context—memory retention states and wakeup events. Based on the result, the execution flow proceeds with cold or warm boot scenarios, with or without rebooting the related execution domains.

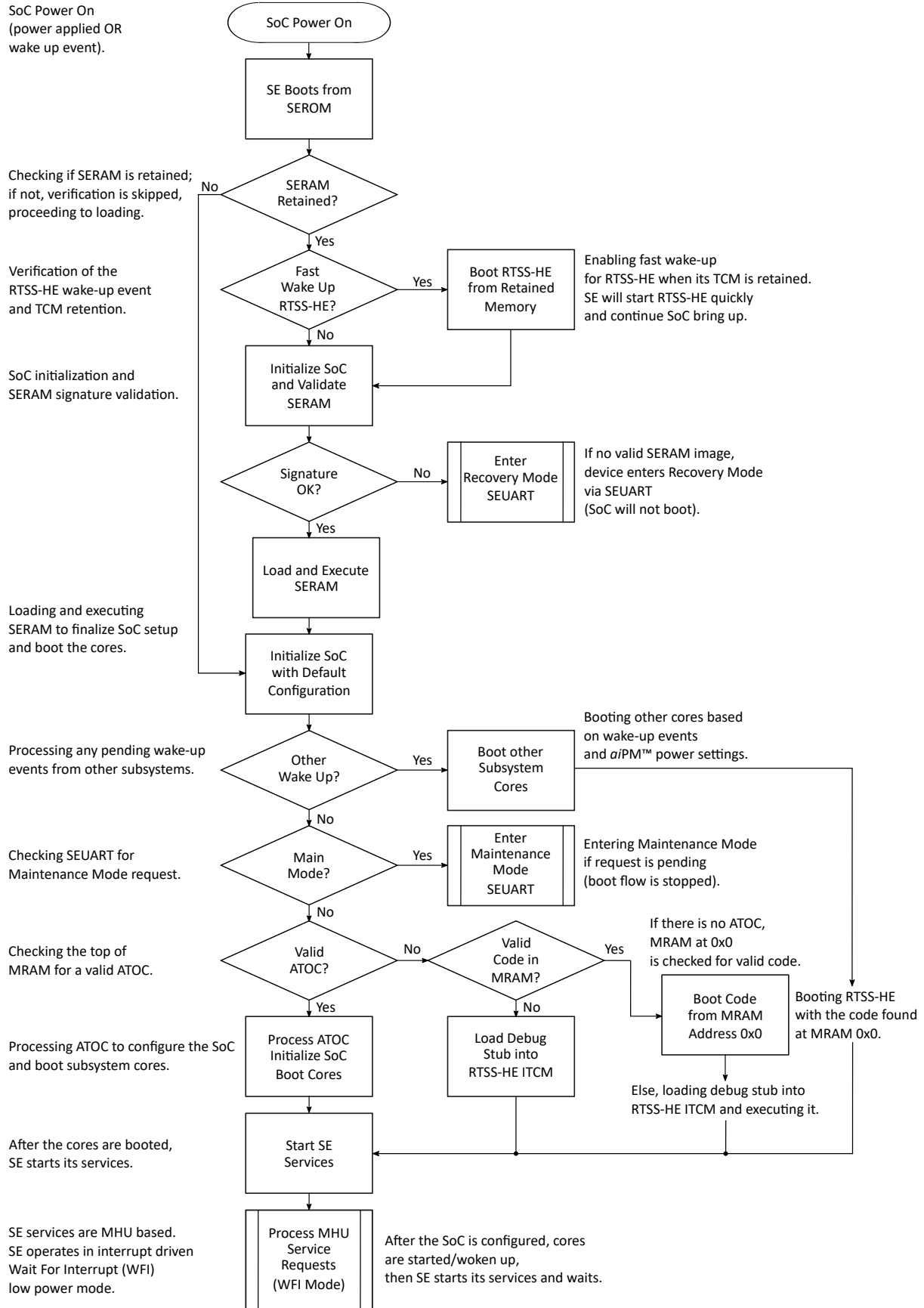
The cold boot process starts with the SESS configuring the bus infrastructure and Firewalls to enable access to the system memory and peripherals. The boot ROM code then accesses the reserved on-chip NVM for a cryptographically signed Second Stage bootloader, and after validation, it proceeds with its execution. The Second Stage bootloader in the SESS sets up the system clock generator, the PLL operation, and the security

mechanism. Next, it scans the NVM for valid Application Table of Content (ATOC) and copies memory segments to their execution address (RTSS TCMs) if needed, or executes directly from MRAM if directed (XIP mode). Finally, the SESS starts each of the subsystem execution domains, bringing them out of reset, and then enters a wait loop, expecting application service requests coming from any of the MHUs connected to the application cores.

If the Second Stage bootloader validation fails, or it does not find any application code in NVM, the SESS enters a wait loop, expecting a command stream from a dedicated SE UART. This is an abnormal state intended for system recovery (Recovery mode) following accidental NVM data corruption.

[Figure 2-16](#) describes the cold boot process execution flow.

Figure 2-16 Cold Boot Sequence



The warm boot is initiated when the processor wakes up from the lowest power-saving mode. The ROM boot code identifies the SESS RAM retention state and, in case it is valid, it continues with running the Second Stage bootloader stored there. Otherwise, it proceeds as in the cold boot scenario.

While in the STOP power mode, all subsystems are off and only a few peripherals can generate wakeup events. Usually, these are application specific events, configured and associated with a specific execution domain. The Second Stage bootloader identifies the mapping between the wakeup event and the associated execution domain, then proceeds with its boot procedure. If the execution domain has its memory retained, it is powered up and immediately started. Otherwise, the bootloader executes a cold boot scenario for that execution domain.

2.9.2 Application Deployment in NVM

Ensemble family of devices features up to 5.5MB of NVM (MRAM) that can store application code and data. The layout of the storage space is described in a dedicated memory location called System Table of Content (STOC). It contains pointers and attributes to the different blocks of application code and their attributes. The STOC is processed by the SE Second Stage bootloader as described in [Section 2.9.1 Boot Process](#).

The attributes of each binary image specify if it needs to be loaded to an SRAM region. The image loading may also be configured to include decryption and/or decompression. Once the SE loads an application image to a specific subsystem, it releases its reset signal and starts its operation.

2.10 Additional Information

Detailed device specifications are provided in the respective series-specific Datasheet.

For further detailed information about specific hardware components (CPUs, NPUs, memories, peripherals, their register settings, etc.), refer to the series-specific Hardware Reference Manual on the Alif Semiconductor [Reference Manuals](#) page.

For managing software configurations of device resources, power, pins, clocks, DMA requests, interrupts, and various other additional settings, refer to the [Alif Conductor](#) tool.

3 Secure Enclave Subsystem (SESS)

3.1 SESS Introduction

The Ensemble family of fusion processors and microcontrollers from Alif Semiconductor creates a scalable and compatible continuum of highly integrated processors for use in intelligent embedded IoT applications. It contains multiple ARM CPU cores for applications processing including Cortex-A32, Cortex-M55 and Ethos-U55 cores, along with a security subsystem, on-board memories, and peripherals.

The SESS is a security subsystem which is responsible for managing and booting the entire device in a secure manner. The SESS contains an isolated core and memories and consists of:

- Arm Cortex M0+ core
- Crypto Hardware (CC312)
- One Time Programmable (OTP) memory
- SRAM
- ROM
- SE Peripherals such as UART, Watchdog Timer (WDT), and Real-Time Counter (RTC)

The SESS initializes the device, sets up the security policies, and performs a secure boot for the other subsystems. Once the boot is complete, the SESS offers runtime services for security and system configuration.

The SESS has three software components:

- SEROM (Initial ROM code that boots the SE)
- SES (Secure Enclave Supervisor) code image
- SETOOLS (Alif Security Toolkit—Windows, Linux, or MacOS)

The SESS Arm Cortex-M0+ core is the first core to execute, which boots from the Secure Enclave Read Only Memory (SEROM) and its primary role is to load SES image from the device's internal MRAM-based NVM.

Once it has completed booting the rest of the Application CPUs, it will then provide Secure Services to the other application CPU cores.

For additional information and details of the tools and processes used to provision security credentials, how to use the credentials, how to sign images, how to create the ATOC, and how to run the tools, refer to the *Alif Security Toolkit User Guide* available on the Alif Semiconductor [Software & Tools](#) page.

Details on installing the SETOOLS can be found in the [Alif Security Toolkit Quick Start Guide](#).

Details on SE services can be found in the [SE Host Services API User Guide](#).

3.2 SEROM Bootloader

The SEROM code serves as the primary bootloader for the device, running on the Arm Cortex-M0+ core (part of the SE) immediately after a Power-on-Reset (POR) or system wakeup. It is the first component in the boot chain, with all other cores held in reset until the SESS releases them.

The main function of the SEROM bootloader is to initialize the device, load and verify the SES code (also referred to as SERAM or the second-stage bootloader) from NVM (after POR) or from RAM, and execute it. The SES image is signed and must undergo verification before execution.

3.2.1 SEROM Boot Sequence

3.2.1.1 Initialization

Execution begins in *Reset_Handler()* function. Then the initialization follows the below order:

- System power is enabled.
- The Firewalls are initialized for the SE, so that the M0+ can access all external memories. Address translations are configured.
- UART is initialized with CPU clock speed of 76.8 MHz and a baud rate equal to 115200 baud rate, and the banner message is printed.
- Function *main()* is called, the CryptoCell (CC312) is initialized, and the Lifecycle State (LCS) state is fetched. If this fails, SEROM enters into Recovery mode.
- Code waits for MRAM to be initialized. If this fails, SEROM enters into Recovery mode.
- The remaining CC312 initializations are performed. If this fails, SEROM enters into Recovery mode.
- The code boot sequence is performed by calling *BootROM_ColdBootSequence()* on the first TOC entry. If this fails, code reports error and then calls *BootROM_ColdBootSequence()* on the second TOC entry. If this fails, code reports error and then enters In-System Programming (ISP) based Recovery mode.

A cold boot sequence occurs if the device is not in Retention mode. In this scenario, SEROM will attempt to load code from the NVM (SERAM) memory. If the NVM read fails, the device cannot boot and will enter into Recovery mode.

3.2.2 BootROM_ColdBootSequence() Function

3.2.2.1 STOC Localization

As an example, the MRAM is assumed to be 6MB (0x0060_0000) in size. The last word of MRAM, located at offset 0x005F_FFC, contains the offset of the ATOC within MRAM. The function accesses this word and verifies that it is nonzero, less than or equal to the size of MRAM minus ½ megabyte, and a multiple of 16. If those conditions are not met, Recovery mode will be invoked.

3.2.2.2 Selected TOC Entry Processing

The selected TOC entry is at an offset relative to the STOC. For example, the first TOC entry is at offset 0x10 and the second TOC entry is at offset 0x20. The function validates that this TOC entry offset is valid, that is, it is nonzero, divisible by 4, and is within range. If those conditions are not met, it returns an error, thus aborting the cold boot sequence.

3.2.2.3 Certificate Chain Location and Identification

The offset in the selected TOC entry points to the first certificate of a chain. The function determines the length of this chain. Valid certificate chains are just a content certificate (length 1), a key certificate followed by a content certificate (length 2), or two key certificates followed by a content certificate (length 3). If no such sequence exists, return an error, thus aborting the cold boot sequence.

3.2.3 Certificate Chain Processing

3.2.3.1 Raw Loading

If the certificate chain only contains a content certificate and the LCS is not secure, raw loading is being requested. If the LCS state is secure, the function returns an error, thus aborting the cold boot sequence. If the content certificate indicates that the code is encrypted, the function returns an error. If the content

certificate indicates that the code is encrypted, it returns an error. If the source address or the target address are invalid, the function returns an error. Otherwise, it copies the SES code from the source address (MRAM) to the target address (SRAM). Once copied, the function calls the entry point. The primary boot sequence is complete.

3.2.3.2 Certificate Chain Verification

Assume there is either a key certificate followed by a content certificate, or there are two key certificates followed by a content certificate. The certificate chain length indicates which of these two options is presented.

3.2.3.3 Warm Boot

The function reads the retention register. If it indicates that SES code is already loaded in SRAM, the function accesses the content certificate to determine what address in SRAM is loaded and calls the entry point. The primary boot sequence is complete.

3.2.3.4 Cold Boot

For each certificate in sequence, code calls the cryptocell-boot library function *CC_SbCertVerifySingle()*. This function will compare the hash of the first key certificate's public key against what is stored in the OTP if we are in Device Manufacturer (DM) or secure LCS. It will compare the hash of the second key's public key against the value stored in the first key certificate. It will compare the hash of the content certificate against the value stored in the second key certificate. Then, it will decrypt the code if encrypted and copy it from MRAM to SRAM as specified by the content certificate. If any error is detected, code returns an error and aborts the cold boot sequence. Otherwise, it calls the entry point. The primary boot sequence is complete.

3.2.4 Error and Panic Values

If an error is returned by any function, it is reported by *main()*. If an error is so severe that processing cannot continue, Recovery mode is called and the code enters an infinite loop awaiting commands from the SE UART.

3.2.4.1 Error Status Values

Table 3-1 presents the error code values.

Table 3-1 Error Codes

Error Code Name	Value	Description
SEROM_CRYPTOCCELL_BOOT_FATAL_ERROR	0x01000000	Mask indicating a cryptocell-boot fatal error
SEROM_CRYPTOCCELL_BOOT_ERROR	0x02000000	Mask indicating a cryptocell-boot error (not fatal)
SEROM_FATAL_ERROR	0x04000000	Mask indicating a fatal error
SEROM_BASE_ERROR_MASK	0x00FFFFFF	Mask for extracting SEROM error code
SEROM_STATUS_SUCCESS	0x0	Success
SEROM_BSV_INIT_FAIL	0x1	Cryptocell-boot initialization failed
SEROM_BSV_LCS_GET_AND_INIT_FAIL	0x2	<i>BsvLcsGetAndInit()</i> returned an error
SEROM_BSV_SEC_MODE_SET_FAIL	0x3	<i>BsvSecModeSet()</i> returned an error
SEROM_BSV_PRIV_MODE_SET_FAIL	0x4	<i>BsvPrivModeSet()</i> returned an error
SEROM_BSV_CORE_CLK_GATING_ENABLE_FAIL	0x5	<i>BsvCoreClkGatingEnable()</i> returned an error
SEROM_MRAM_INITIALIZATION_FAILURE	0x6	MRAM initialization failed

Error Code Name	Value	Description
SEROM_CONTENT_CERTIFICATE_NULL	0x7	Content certificate pointer is NULL (internal error)
SEROM_ATOC_MISALIGNED	0x8	ATOC offset value is misaligned (not a multiple of 16)
SEROM_ATOC_TOO_LARGE	0x9	ATOC offset value is too large
SEROM_OFFSET_ZERO	0xA	TOC offset is zero
SEROM_OFFSET_MISALIGNED	0xB	TOC offset is misaligned (not a multiple of 4)
SEROM_OFFSET_TOO_LARGE	0xC	TOC offset is too large
SEROM_OFFSET_TOO_SMALL	0xD	TOC offset is too small
SEROM_CERTIFICATE_NULL	0xE	Certificate address is zero
SEROM_INVALID_CERTIFICATE_CHAIN	0xF	Invalid certificate chain at specified address
SEROM_INVALID_NULL_ENTRY_POINT	0x10	Entry point address is zero
SEROM_INVALID_VERIFY_IN_MEMORY_CASE	0x11	Invalid verify in memory <i>LoadVerifyScheme</i>
SEROM_INVALID_ZERO_IMAGE_LENGTH	0x12	Invalid image length specified in content certificate
SEROM_INVALID_ENCRYPTED_IMAGE	0x13	Invalid encrypted image use case
SEROM_INVALID_VERIFY_IN_FLASH_CASE	0x14	Invalid verify in flash <i>LoadVerifyScheme</i>
SEROM_IMAGE_LENGTH_TOO_LARGE	0x15	Image length specified in content certificate is too large
SEROM_RAW_IMAGE_LOADING_NOT_ALLOWED	0x16	Raw image loading not allowed in secure LCS
SEROM_INVALID_LOAD_VERIFY_SCHEME	0x17	Invalid <i>LoadVerifyScheme</i>
SEROM_CERTIFICATE_ERROR_BASE	0x20	Certificate Error base—error code
SEROM_CERTIFICATE_1_ERROR	0x21	Error encountered in first certificate
SEROM_CERTIFICATE_2_ERROR	0x22	Error encountered in second certificate
SEROM_CERTIFICATE_3_ERROR	0x23	Error encountered in third certificate

3.2.4.2 Panic Code Values

Table 3-2 presents the panic code values.

Table 3-2 Panic Codes

Panic Code Name	Value	Description
SEROM_NMI_EXCEPTION	2	NMI exception
SEROM_HARD_FAULT	3	Hard Fault exception
SEROM_MEM_MANAGE	4	MemManage exception
SEROM_BUS_FAULT	5	Bus Fault exception
SEROM_USAGE_FAULT	6	Usage Fault exception
SEROM_SUPERVISOR_CALL_EXCEPTION	11	Supervisor Call exception
SEROM_PEND_SV_EXCEPTION	14	PendSV exception
SEROM_UNHANDLED_EXCEPTION	16	Unhandled exception
SEROM_MRAM_FAILURE	30	MRAM failed to initialize
SEROM_FATAL_CC312_ERROR	40	Fatal Cryptocell error
SEROM_ABORT_PROCESSING	50	Abort Processing error

3.2.5 Security Lifecycle States

3.2.5.1 Security Lifecycle States Overview

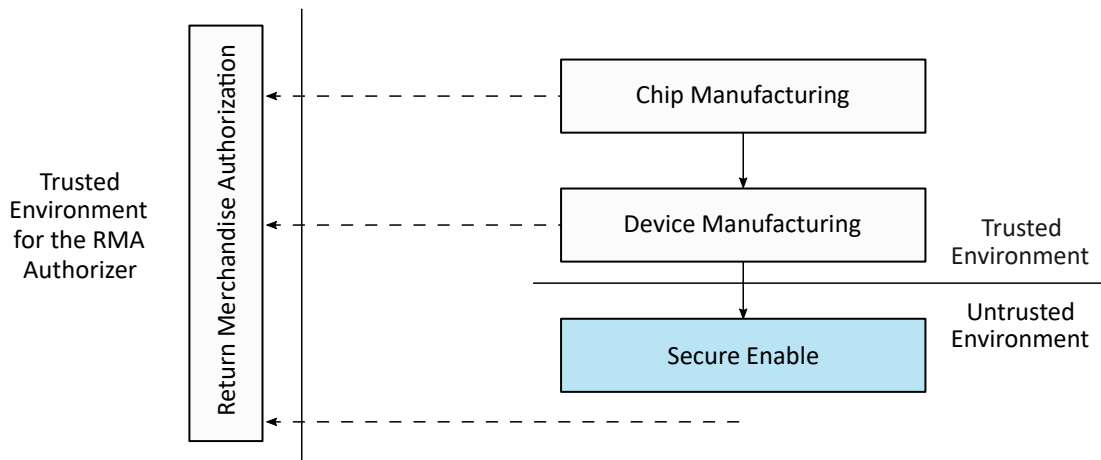
The security engine of the SE (CryptoCell-312) includes a mechanism for managing the security Lifecycle State of the device.

The LCS mechanism enables the device, both hardware and software, to behave differently in different LCSs. This mechanism protects any security assets when they have been introduced into the device and reduces the risk of IP theft and reverse engineering.

During development, the device should be in the DM state. This should allow for interactive debugging to take place and any Security features should still be enabled. Once development is completed, the software can advance the LCS to a secure state, which enables more rigorous security features to make the final product more robust.

Figure 3-1 illustrates possible LCS transitions.

Figure 3-1 LCS Transitions



3.2.5.2 Chip Manufacturing LCS

The Chip Manufacturing (CM) LCS indicates that the chip has not been assembled into a device yet and has not left the production line of the Integration Circuit Vendor (ICV).

In this LCS, all chip debugging and external testing capabilities are enabled, including capabilities that enable access to security-sensitive information. For example, chip test modes and validation functionality.

3.2.5.3 Device Manufacturing LCS

The Device Manufacturing (DM) LCS is programmed during the device assembly on the production line. In this LCS, debugging and external testing capabilities are still enabled. Internal testing capabilities, for example full-chip internal scan, are disabled.

3.2.5.4 Secure Enable LCS

The Secure Enable (SE) LCS is used for devices out of the manufacturing line and "in the field". It permits the execution of security functions but blocks all debugging and testing capabilities. Using secure boot is mandatory in this LCS.

3.2.5.5 Return Merchandise Authorization LCS

The Return Merchandise Authorization (RMA) LCS is a terminal state for devices that are returned to the ICV for analysis of fatal failures.

When a device is put into RMA LCS:

- The device loses all its existing secret keys.
- The device regains full access to all debugging and testing capabilities, as available in CM LCS.
- All cryptographic engines are usable for testing, but the Hardware Unique Key (HUK) changes for each boot phase.
- Between cold boots, the cryptographic functions of the device can be tested, but no permanent secrets can be derived from the root keys:
 - HUK is replaced with a different random value with each boot cycle. Therefore, any data that was previously saved and is protected by a key that is derived from HUK is lost. However, keys can still be derived from HUK and used until the next boot cycle.
 - *Kce* and *Kceicv* are invalidated, so that secure boot can be used only in non-encrypted mode.
 - *Kcp* and *Kpicv* are invalidated, so that provisioning can no longer be done based on the previous values.

Table 3-3 presents different LCS states.

Table 3-3 LCS States

LCS State	Debug Secure Enclave Core	Debug A32 or M55	Comments
CM	Enabled, no certificate needed.	Enabled, no certificate needed.	Device is in Alif Semiconductor factory and provisioned with Alif Semiconductor Root-of-Trust (RoT). Device never leaves factory in this state.
DM	Disabled. Can never be enabled in this LCS.	Enabled, no certificate needed.	Device will/can be provisioned by OEM with RoT. Device can be used for development in this state.
SE	Disabled. Can never be enabled in this LCS.	Enabled OR Secure: Selected cores enabled with OEM signed certificate.	Device is secure as configured by OEM. Only OEM can debug or move device to RMA state.
RMA	Enabled with Alif Semiconductor signed certificate.	Enabled, no certificate needed.	Device can only be placed into RMA state by signed OEM certificate. Then device is sent to Alif Semiconductor, where the OEM code or cores with signed certificate can be debugged.

3.3 SES Code Image

The SES image contains the main code for the SESS and performs device initialization, secure boot of the device cores, and provides run time security and device services.

3.3.1 APSS and RTSS Boot Up Sequence and ATOC

The APSS and RTSS are represented by different type of CPUs: the Cortex-A32 and Cortex-M55, respectively. These CPUs should execute application code that is booted from the NVM (MRAM).

3.3.1.1 ATOC Attributes

The ATOC is a metafile that contains the fusion device configuration options and the application images. The ATOC package contains a configuration file that specifies the default clock settings, pins' settings and security

firewall settings for the fusion processor as well as a list of image objects. For each of the image objects, there is a set of descriptors that specifies the type of image, image address in NVM, the boot mode, load/boot address.

The SESS should process this list and the associated binary objects.

Table 3-4 lists the image attributes within the ATOC.

Table 3-4 ATOC Image Attributes

Attribute	Description
Load	Load an image
Boot	Load and execute an image
XIP	Execute an image in place (from MRAM)
Deferred	Process image later via runtime request
Decrypt	Decrypt option for the image
Decompress	Un-compress option for the image

These attributes direct SESS in what to do with this image, for example boot, load, and so forth. The load addresses for the images are also contained within the ATOC.

The attributes are set into the image binaries using the SETOOLS, which create the user ATOC binary for storage into MRAM starting at address 0x0.

SESS should process the ATOC, apply the configuration options and process the images in the ATOC. The SESS does not have information of what is being booted, for example—Linux or an RTOS.

Configuration options applied by SESS:

- Firewall configuration
- Pins configuration
- Interrupt routing configuration
- Event routing configuration

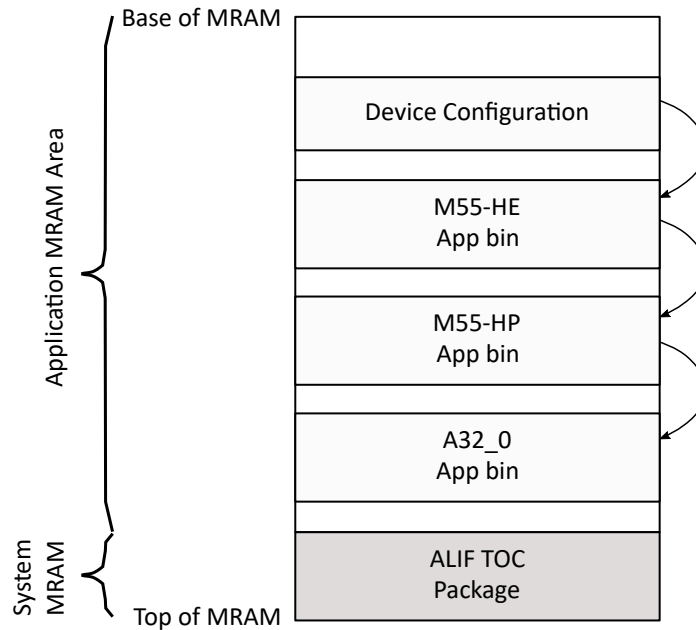
3.3.1.2 NVM (MRAM) Storage

The ATOC is programmed in the NVM (MRAM) along with the images.

NVM is divided into a System section and an Application section as shown in Figure 3-2. The Application section contains the boot code for the APSS and RTSS CPUs. The System section contains similar TOC components required for internal device peripherals.

The System TOC package contains information about the Application TOC.

Figure 3-2 NVM Storage



3.3.1.3 Boot Sequence

The SESS will process the ALIF TOC (STOC) package data in the NVM.

Figure 3-2 shows how the TOC data is processed one after the other. In this example, the device configuration is always processed first, after which the RTSS-HE and RTSS-HP images should be booted and/or loaded depending on what attributes have been set before finally moving to the APSS images. The order of CPU execution is dependent on the ordering of TOC objects in the .json file used to create this MRAM image.

If the image is marked to load, it should be copied from the NVM into a destination address, for example the ITCM of the CPU to be loaded.

If the image is marked as compressed, it should be uncompressed before loading and/or booting.

If the image is marked as deferred, it should not be verified, loaded, or booted initially. SES should expect a Secure Services call from the application CPU, signaling that image is to be loaded and/or booted.

3.3.1.4 Cold Boot

A cold boot sequence means that all the TOCs and images should be processed from NVM.

No check is done to see if CPUs had previously been loaded.

All CPUs that require booting should have the code image loaded to their TCMs, then the CPU should be reset and released causing the application CPU to start executing.

As part of the booting sequence SESS will record the CPU booted status.

3.3.1.5 Warm Boot

A warm boot sequence means that when SESS is executed it should check to see if the system is in Retention mode.

If the Retention mode is enabled, then SESS should check which CPUs were loaded from the previous boot and not reload these again. Device configurations previously loaded should be restored rather than reprogrammed, such as clocks and pins configurations.

3.4 SETOOLS

The Alif Semiconductor Security Toolkit (SETOOLS) is a set of runnable applications to assist in the building of software images for placement into the MRAM. The SETOOLS is also used for generating security material, such as certificates used to sign the application software.

For information and details of the tools and processes used to provision security credentials, how to use the credentials, how to sign images, how to create the ATOC, and how to run the tools, refer to the *Alif Security Toolkit User Guide* available on the Alif Semiconductor [Software & Tools](#) page.

The latest version of the Alif Security Toolkit is available from the Alif Semiconductor web site in three different formats:

- [Windows ZIP format](#)
- [Linux tar format](#)
- [MacOS tar format](#)

3.5 SEROM UART Console Output

Successful boot should only print the SEROM banner. Other outputs could be errors or warnings.

3.5.1 SEROM UART Banner Message

On boot up SEROM should report the banner message; this banner is printed at a very early stage of the boot process. It has the following format:

<SEROM_MAJOR_VERSION> <SEROM_MINOR_VERSION> <SEROM_PATCH_LEVEL>

Example:

```
0.47.42 REV_ID = 000B0000
```

No other information should be printed from SEROM unless there is an error.

3.5.2 SEROM Factory Data Print (REV_A1)

```
SOCID = 0000A100
```

3.5.3 SEROM Factory Data Print (REV_B)

```
REV_ID = 0001B000
```

3.5.4 SEROM UART Error and Warning Messages

[Table 3-5](#) presents the different types of SEROM UART messages.

Table 3-5 SEROM UART Messages

UART Display Message	Description
BootROM_ColdBootSequence(0, &eo): error status: 0x0200_0021	Cryptocell—boot error on first certificate
Cryptocell—boot error code: F100_0007	Cryptocell—boot error code reported
BootROM_ColdBootSequence(0, &eo): error status: 0x0000_000A	TOC1 offset is zero
BootROM_ColdBootSequence(1, &eo): error status: 0x0000_000B	TOC2 offset is zero

UART Display Message	Description
000A	
ISP_Handler—Starts	Entering ISP mode
BootROM_ColdBootSequence(0, &eo): error status: 0x0000_000B	TOC1 offset is misaligned
BootROM_ColdBootSequence(1, &eo): error status: 0x0000_000B	TOC2 offset is misaligned
BootROM_ColdBootSequence(0, &eo): error status: 0x0200_0023	Cryptocell—boot error on third (content) certificate
Cryptocell—boot error code: 0xF100_0007	Cryptocell—boot error code reported
BootROM_ColdBootSequence(0, &eo): error status: 0x0200_0022	Cryptocell—boot error on second certificate
Cryptocell—boot error code: 0xF100_000D	Cryptocell—boot error code reported

3.5.5 Hard Fault Exceptions Display

A hard fault is an exception that occurs because of an error during exception processing, or because an exception cannot be managed by any other exception mechanism. Hard faults have a fixed priority of -1, meaning they have higher priority than any exception with configurable priority (see [Table 3-6](#)).

Table 3-6 Hard Fault Exceptions Display

Hard Fault Exception	Values
MSP	0x3003_FF68
r0	0x0
r1	0x3000_1000
r2	0x0
r3	0x3003_FF7C
r12	0x0
lr	0x3000_412F
pc	0x3000_1000
psr	0x6000_0000
panic	code = 3

3.5.6 Other Exception Handler Display Messages

[Table 3-7](#) presents the different types of exception handlers.

Table 3-7 Exception Handlers

Exception Handler Display Message	Description
NMI Exception panic: code = 2	NMI exception
Memory Manage panic: code = 4	Memory Manage exception
Bus Fault panic: code = 5	Bus Fault exception
UsageFault panic code = 6	Usage Fault exception
SVC Exception panic code = 11	Supervisor Call exception

Exception Handler Display Message	Description
DebugMon	DebugMon interrupt—not fatal
PendSV Exception panic code = 14	PendSV exception
SysTick	SysTick interrupt—not fatal
Unhandled Exception panic code = 16	Unhandled exception

3.5.7 Error Status Values

For the different types of error status values, see [Table 3-1](#).

3.5.8 Panic Code Values

For the different types of panic code values, see [Table 3-2](#).

3.5.9 SEROM UART Console Configuration

[Table 3-8](#) presents the SEROM UART configuration in the console.

Table 3-8 SEROM UART Console Configuration

Parameter	Description
Speed	115.2 kbps
Data	8 bit
Parity	None
Stop bits	1 bit
Flow control	None

3.5.10 Maintenance and Recovery Mode

Should the contents of the NVM (MRAM) become corrupted or lost, the initial ROM boot will enter a Recovery mode allowing for the recovery of the SESS software in NVM.

In-System Programming (ISP) is a method by which SEROM can execute operations sent from a Host machine to the target. These operations are limited to send the revision data for the device, update MRAM contents, and reset.

These operations are carried out over the Secure Enclave UART (SE UART) using an Alif Semiconductor defined protocol. Tools on the Host machine will manage this protocol.

The ISP protocol is supported by SEROM. It uses ISP primarily as a recovery mode to restore MRAM contents.

3.6 SESS Message Handling Unit (MHU)

3.6.1 SESS MHU Overview

The SESS provides several runtime services to the application CPUs. The SESS communicates with the rest of the system using protected mechanisms, based on the MHU. The application CPUs can dispatch service requests to the SESS using the MHU. The SESS executes the service requested and responds back.

3.6.2 Service Request Mechanism

The application CPUs can dispatch service requests to the SESS. This is done using shared memory and the MHU. The messages over the MHU are used for handshaking and specifying the location of shared memory region where the service request is placed. SESS processes the service request, executes the request, and sends a response back using the same mechanism. The response includes an error code and optional data specific to the service request.

3.6.3 Runtime Services

The SESS provides several runtime services to the application CPUs.

3.6.3.1 Application Services

This section lists the services that can be requested by the application CPUs to configure and control several chip-level functions.

- *Pins Configuration Management*: The device has several pins whose functions can be dynamically reconfigured using a multiplexer. This functionality enables the application CPUs to dispatch a service request to the SESS to reconfigure a pin to a different function.
- *Interrupt Management*: This service enables the application CPUs to configure and manage chip level interrupts.
- *Event Router Management*: This service enables the application CPUs to configure and manage chip level events.
- *Clock Management*: This service enables the application CPUs to configure and manage chip level events.

3.6.3.2 Power Management Services

This section lists services that can be requested by the application CPU to optimize power consumption.

- *Stop Mode*: This service can be used to enter low-power states.
- *Wake Up*: This service can be used to wake up from low-power states.
- *DC-DC Control*: Power consumption can be optimized by using this service to control the operation of the DC-DC converter.

3.6.3.3 System Security Services

The section lists services that are related to security and that may require the involvement of the cryptographic accelerator.

- *True Random Number Generator (TRNG)*: The application CPU can request the generation of a true random number using this service.
- *Key Management*: This service enables the application CPU to configure and maintain cryptographic keys stored in NVM OTP.
- *Lifecycle State Management*: The application CPU can use this service to advance the LCS of the chip.
- *Image Verification and Loading*: The SESS, using the cryptographic accelerator, can authenticate images and load them to various memories in the chip. The application CPUs can make such requests to the SESS using this service.
- *Encryption and Decryption*: The SESS supports encryption and decryption of data. This is done with the help of the cryptographic accelerator. The application CPUs can make these requests to the SESS using this service.

4 Ensemble Application Anatomy

4.1 Application Architecture

This section describes application architectures having different memory layouts. Thanks to the multiple application processors and many different memory types (NVM, ITCM/DTCM, SRAM), the embedded software application can be optimized for performance, size, boot time or ease of debugging. The Ensemble processor platform provides a highly flexible bus interconnect that offers different bootloading and execution models. The boot process is split between the SE and the application processors. The boot process used depends on the security, performance, and memory size limitations. The execution modes differ based on the memory location containing the code.

When the application processor fetches instructions directly from the NVM, the execution mode is referred to as eXecute-in-Place (XIP). This mode is traditional for MCUs. Alternatively, the code may be copied from the NVM to the TCM memory or SRAM during the boot process, and then executed from there. This mode is typical for the TCM-based processors and MPU-based systems. User applications often use a mix of the two execution models to combine their benefits—faster boot in XIP mode and better performance when running from TCM/SRAM.

The necessity of different application scenarios stems from the fact that the Ensemble platform is a heterogeneous multi-core environment with enhanced security capabilities. The application boot process and the individual core reset are controlled by the SESS. The applications, on the other hand, can have different security and performance requirements, which affects boot model requirements and how the SE will boot these processors. The description of the different boot models references application content organized in several segments, each serving specific purposes such as:

- Code (*.text*) segment, that is Execute Only (XO) segment
- Constant (*.const*) data, Read Only (RO) segment
- Initialized (*.data*) data, Read/Write (RW) segment
- Zero Initialized (ZI) data segment
- Non-Initialized data segment

The segments are defined in the linker script (scatter file) with a name, load address, and execute address. When the load and the execute addresses are the same, the application can run in XIP mode.

NOTE

It is recommended to define at least one such XIP segment for booting. Typically, this is a code segment that also contains the vector table definition.

If both the load and the execute addresses are different, the application startup code invokes a *scatter_load()* function that copies the segments from their load locations to their execute locations. This is typically done for the initialized data segment. Certain functions can also be placed in this segment so that they can execute faster from SRAM/TCM.

To define the boot up process, the fusion processor uses the ATOC that specifies which application processor or processors to boot, the boot order, and the boot model for each application processor. The ATOC is always located at address 0x0 in MRAM and can vary in its length.

Table 4-1 summarizes the different application boot models/scenarios along with guidance for selecting one versus another.

Table 4-1 Application Boot Models

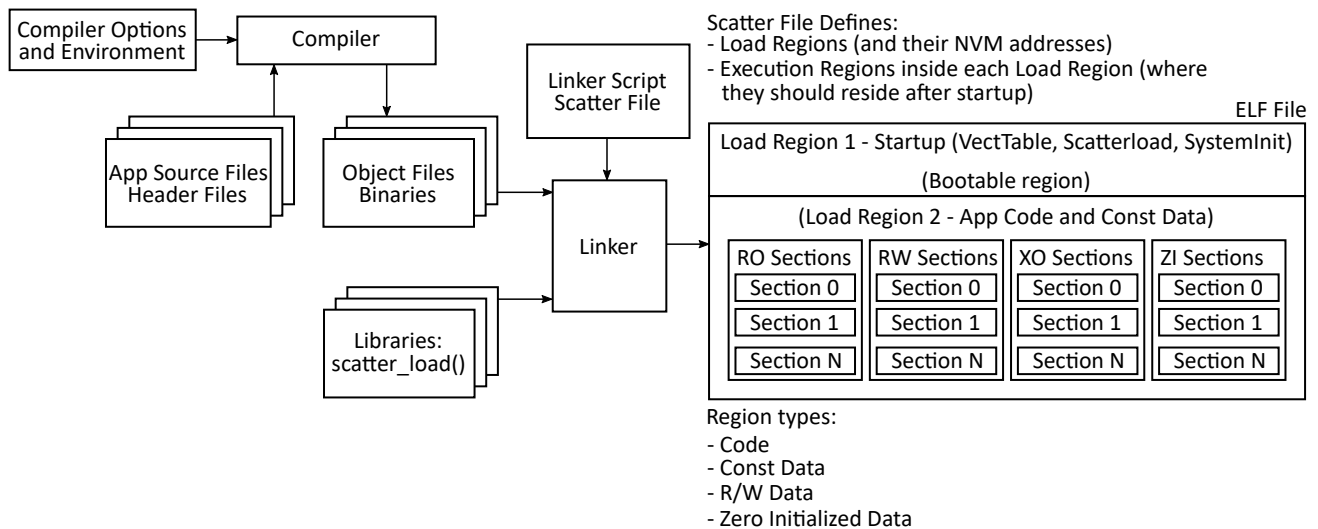
Boot Model	Boot Process	VTOR Register	Secure Boot	Execution Mode	Comments
MCU ATOC in NVM at 0x0	Step 1: SE processes the ATOC. Step 2: When secure booting, the SE processes the application certificates and verifies the application images. Step 3: SE sets VTOR register and releases the RESET of the specified application processor in ATOC. From this point, the application processor takes over the boot process. Step 4: Application core <i>Reset Handler</i> executes <i>scatter_load()</i> function to copy segments from their load address in NVM to their execute locations.	Application location in NVM	Optional	Initial XIP—the reset vector points to NVM	The SESS will verify the signature of the application in NVM. If invalid, the application processor is not released to boot. Else, the SESS will boot the application processor from NVM in XIP mode. Note: Each application NVM address has to be planned, as multiple applications may exist, and possible free space gaps in NVM may be necessary. Debug cycle is easy—application images must be processed by SETOOLS to create ATOC and then programmed to NVM.
MPU ATOC in NVM at 0x0	Step 1: SE copies only one segment (bootloader) from NVM to the SRAM/ITCM. Step 2: SE sets VTOR register and releases RESET of the application processor. Step 3: The application processor executes the bootloader with its <i>scatter_load()</i> to copy the remaining segments from NVM to ITCM/DTCM/SRAM.	ITCM or SRAM	Optional	SRAM—the reset vector points to SRAM/TCM	SE copies a bootloader segment to ITCM/SRAM. Each application load address has to be planned for individual projects—possible gaps. This approach avoids the need of XIP mode. Debug cycle is easy—application images must be processed by SETOOLS to create ATOC and signed images, ready for programming to NVM.
Advanced	Step 1: SE supports a variety of loading options that enable copying one or more NVM image segments to ITCM/DTCM/SRAM load.	ITCM or SRAM	Optional	SRAM—the reset vector points to SRAM/TCM	All code, data and constants are copied to the ITCM/DTCM/SRAM load address. For code, the load location is the same as the execute location. Application images can be efficiently packed in the NVM—no need of specific absolute addresses. Depending upon the approach used, the application size may be limited by the load memory size. The build

Boot Model	Boot Process	VTOR Register	Secure Boot	Execution Mode	Comments
ATOC in NVM at 0x0	Step 2: SE supports setting the VTOR register to boot from any load address. Step 3: Enables the SESS to perform the scatter load function instead of the application processor. Step 4: The SE also supports options for decrypting, decompressing, and signature checking the images.				process may need to include the <i>fromelf</i> utility with option to create individual images for each segment type. Debugging is more complex. Application images must be processed by SETOOLS to create ATOC and signed images ready to program to NVM.

In all models, the initial value of the VTOR register is set by the SE. The SE releases the application processor to execute (that is to boot).

Figure 4-1 presents the scatter file usage.

Figure 4-1 Scatter File Usage



Additional information on the boot process and linker files can be found in the following white papers:

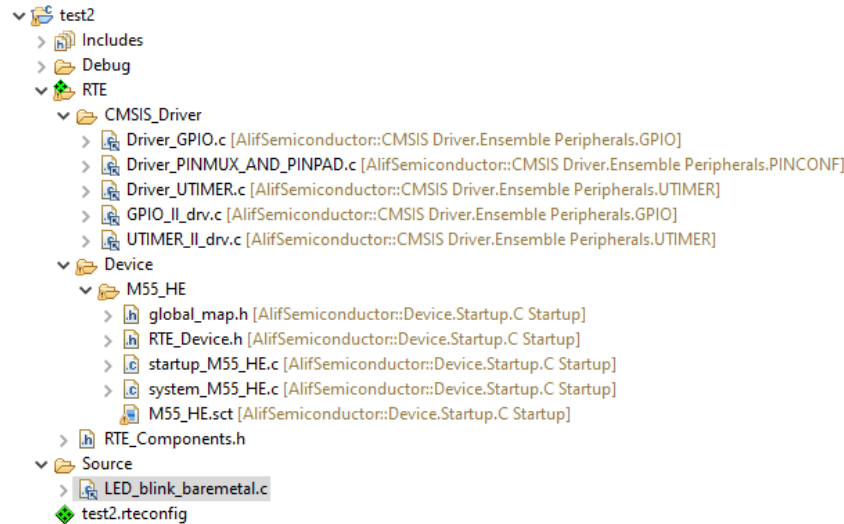
- [Diving Into Alif Semiconductor's Secure System Architecture and Secure Boot Process](#)
- [RAM Regions and Linker Files in Alif Microcontrollers](#)

4.2 Application Source Code Organization

A typical Ensemble project is structured as a collection of folders created by the new project wizard. The application source code is typically stored in a dedicated folder (*Source*) under the project root.

Figure 4-2 presents a typical project structure.

Figure 4-2 Project Structure



The project folders are as follows:

- **Includes** folder: Contains references to the locations where the Common Microcontroller Software Interface Standard (CMSIS) header files are stored. These header files are part of the CMSIS drivers and the startup code selected during project configuration.
- **Debug** folder: It is named after the currently selected project build configuration. It contains the output of the compiler (the binary objects) and the output of the linker (the executable or the library being build). This folder is created automatically and updated with every build. Selecting *Release* configuration results in folder named *Release* being added to the project structure and updated with every build command.
- The **Run Time Environment (RTE)** folder: Contains all CMSIS drivers and device specific source code imported from the CMSIS packs. The *Device* subfolder contains files specific for the CPU core targeted by the project. They include the headers with the device specific macro definitions, the vector table definition, and the startup code executed once the CPU reset is released.
- **Device** folder: Contains a subfolder having the name of the target CPU, which in turn holds all startup code, system initialization code, and the scatter file. The latter defines the application memory layout that may need to be updated as the application size grows and requires placing of the binary objects in different memory regions.
- **Source** folder: Contains the application code. In the example above, the *LED_blink_baremetal.c* contains the *main()* function invoked by the startup code. This folder should be used to store the hierarchy of application source code, possibly organized into multiple subfolders.

The CMSIS project contains one file named after the project, with the extension *rteconfig* (*test2.rteconfig* in the example above). This file contains the project specific CMSIS configuration. Double-clicking the file name opens the CMSIS configuration editor, which enables adding new components or removing the existing ones. This process updates the content of the RTE folder described above.

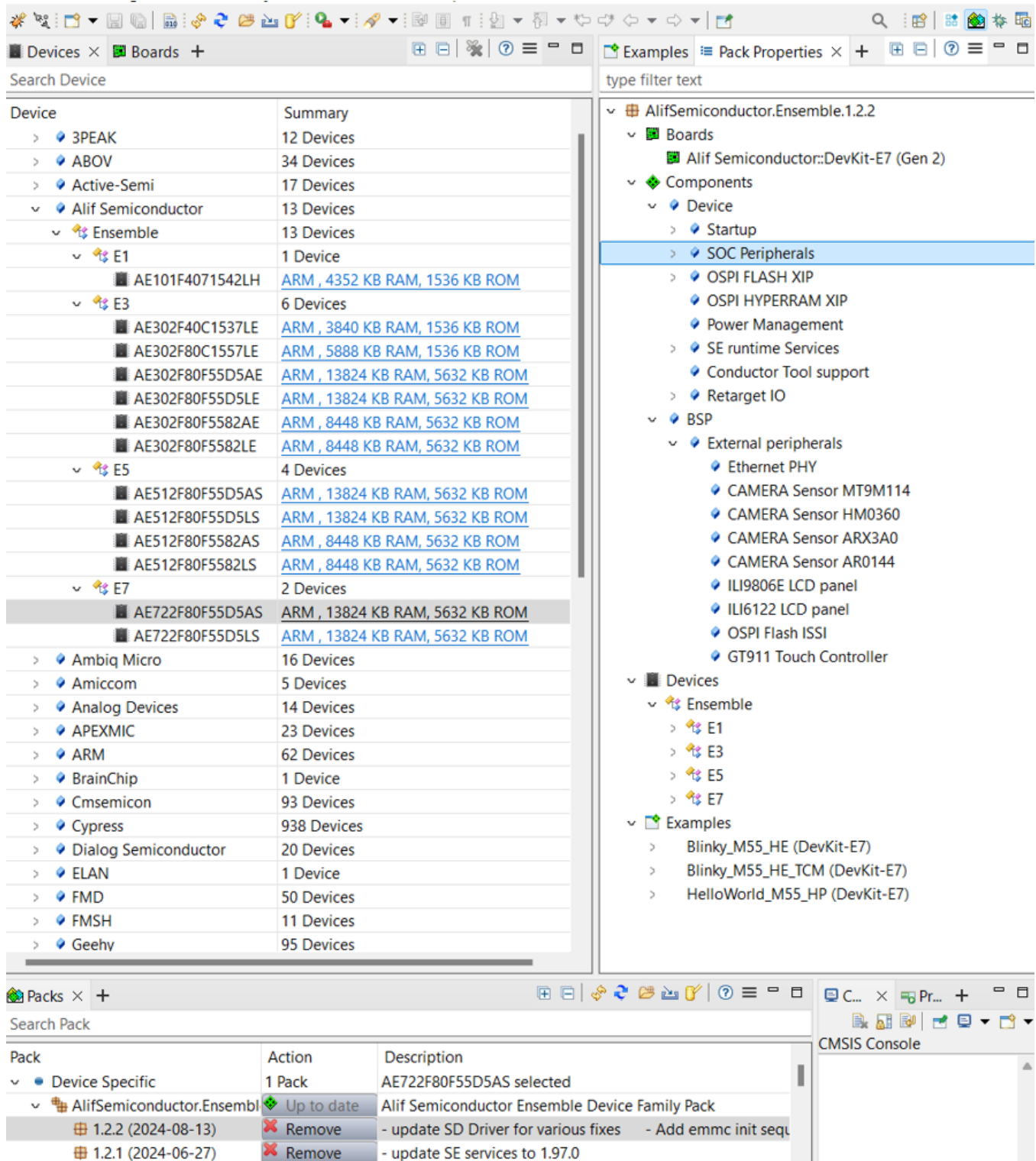
4.3 CMSIS-Packs and Header Files

The Ensemble application development is supported by application templates, device drivers, and header files organized in CMSIS-Packs. Each Ensemble device comes with a specific pack that provides a selection of device drivers that can be included seamlessly in the application code. The CMSIS-Pack includes startup

specific for the selected core and execution model (SRAM or XIP), an initial linker script (scatter file), and project settings.

Figure 4-3 presents an example of CMSIS-Pack Manager under Arm DS IDE.

Figure 4-3 CMSIS-Pack Window



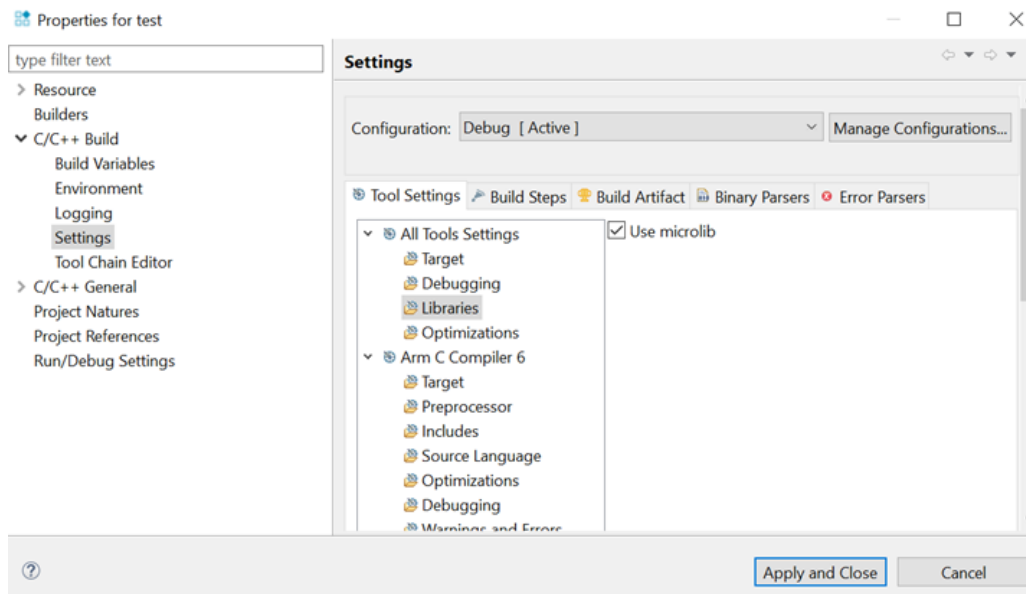
The selection of an application example automatically highlights all dependencies, such as startup code or specific driver. All warning and error messages should be cleared by following the instructions that display the missing dependencies.

4.4 Compiler Libraries

The Ensemble application development is based on C or C++ language. Traditionally, the C compilers provide a library implementing a standard set of functions. They are referenced through header files and placed in the code (.text) segment of the application memory layout.

For example, the Clang compiler comes with standard library linked to the applications by default (see [Figure 4-4](#)).

Figure 4-4 Properties for Test



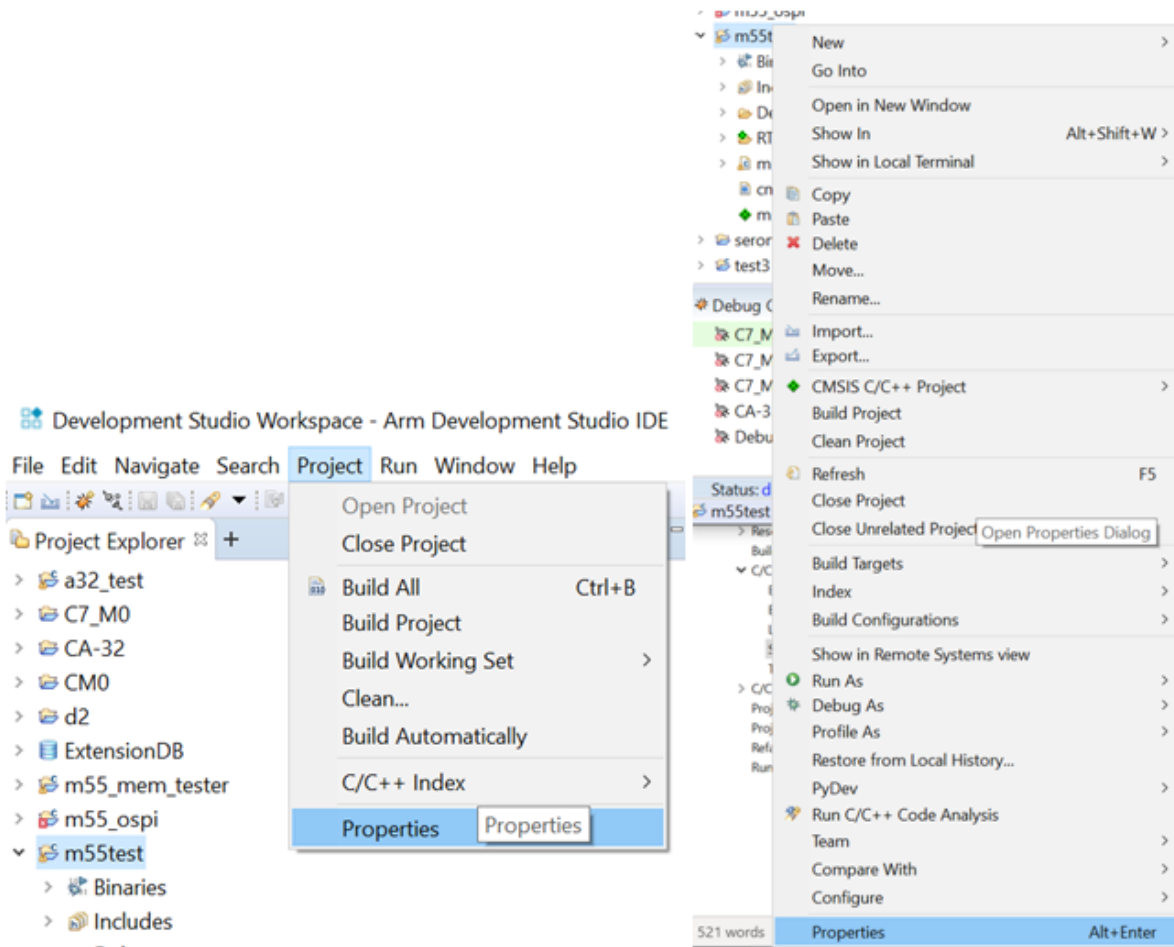
4.5 Project Settings

The project setup properties, compile, and link debug options are inseparable from the application it configures. The project templates included in the Alif Semiconductor CMSIS-Pack contain the most common settings useful for most development scenarios.

[Figure 4-5](#) through [Figure 4-8](#) are examples from Arm DS Integrated Design Environment (IDE). For particular IDE guidance, refer to the respective User Guide *Getting Started with Bare Metal Design* available on the Alif Semiconductor [User Guides & App Notes](#) page.

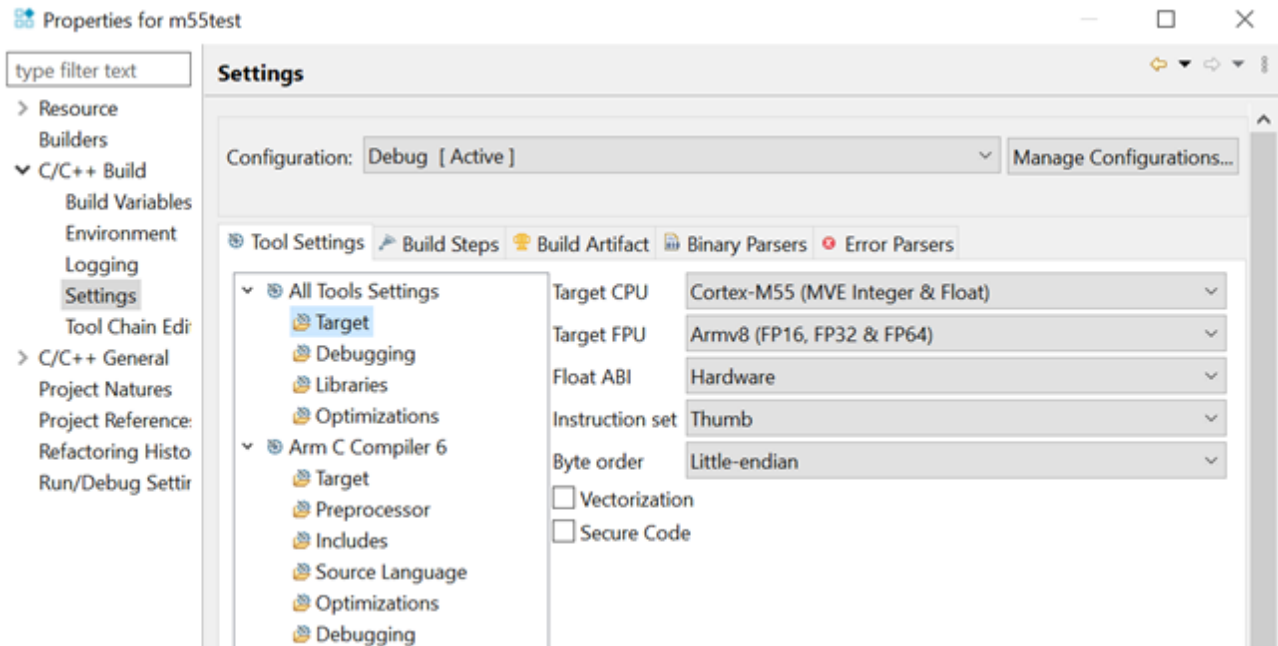
The project properties can be accessed through the main menu or through the context sensitive menu of the project name (see [Figure 4-5](#)).

Figure 4-5 Project Properties



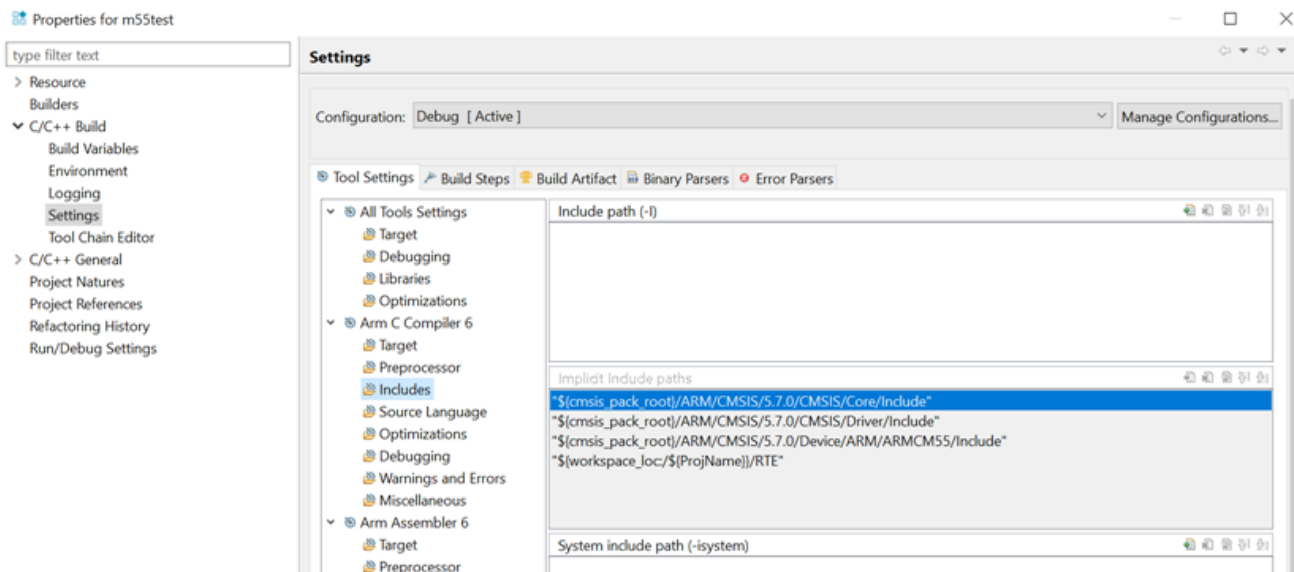
The *Settings* tab of the project *Properties* configures many compiler and linker settings, which must match the target hardware and the desired build output (see [Figure 4-6](#)).

Figure 4-6 Properties for m55test 1



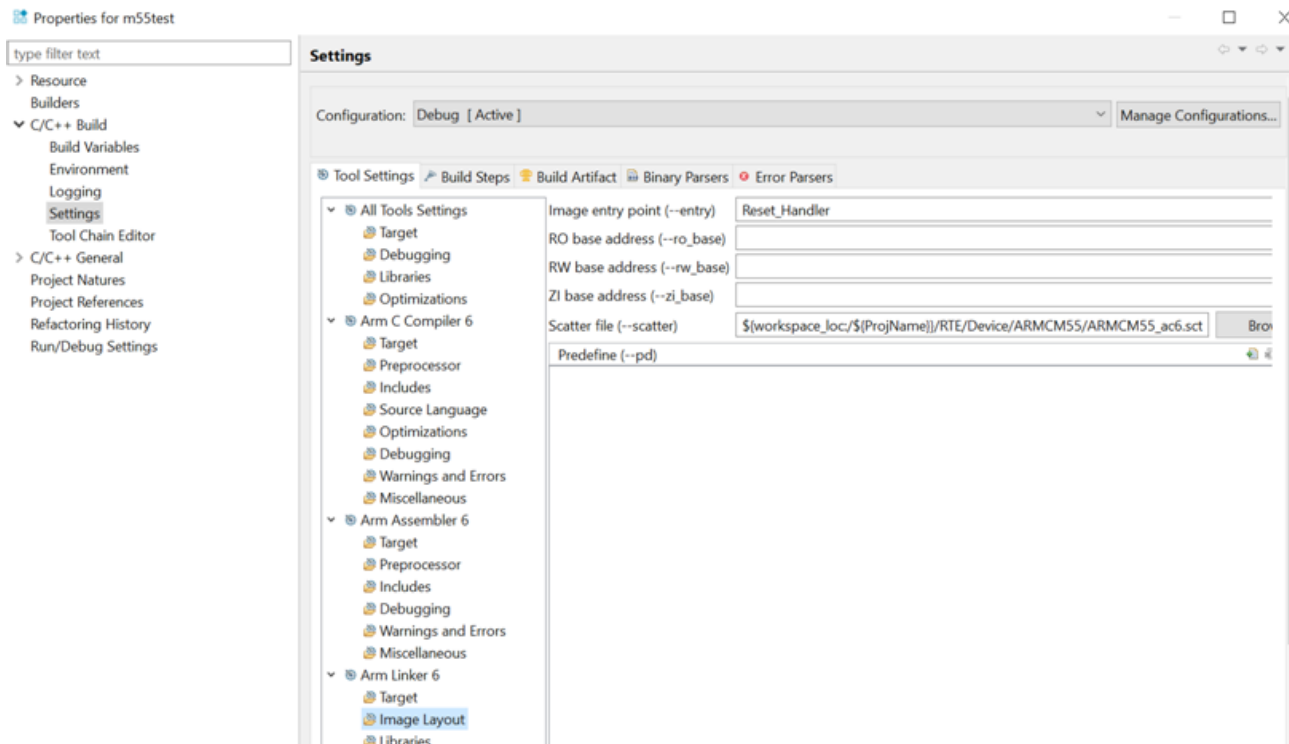
The include directories are listed in the *Includes* option of the Arm Compiler 6 menu (see [Figure 4-7](#)).

Figure 4-7 Properties for m55test 2



The linker script (scatter file) is configured in the *Image Layout* option of the Arm Linker 6 menu (see [Figure 4-8](#)).

Figure 4-8 Properties for m55test 3



4.6 Linker Script

The purpose of the linker script is to configure the operation of the linker, placing objects in specific segments—code, constants, and initialized data. In the embedded system development, the linker script also resolves the external references to address locations, describing the device memory layout.

The content of the linker script, specifically the load address of the different segments, must be coordinated with the SETOOLS configuration file when the boot mode is MCU or Hybrid. In these modes, the NVM address where the application image is stored, must match the load address definition in the linker script. In case there are several images—their address in the NVM must match each load address in the linker script.

4.7 Applications Table of Contents (ATOC)

The ATOC is a metafile used by the SE that consists of two parts:

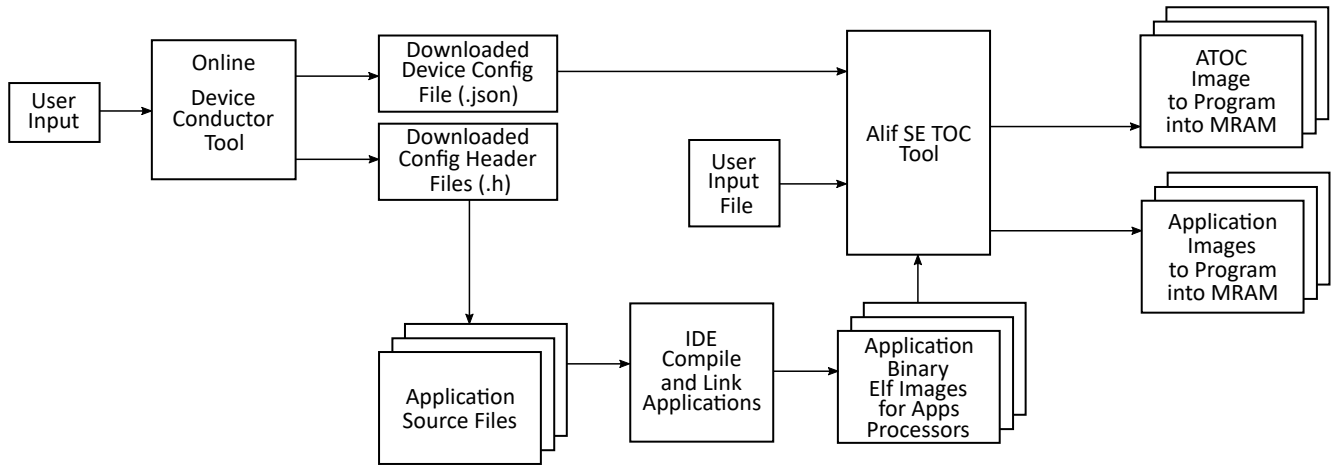
- Device configuration file
- List of the MRAM application images that execute on the application processors (see [Figure 4-9](#)).

The device configuration file specifies the fusion processor configuration settings that are applied after system reset before any application processor starts execution. This includes system clock settings, pins' settings, and system allocation (security firewall) settings.

The list of application images in MRAM provides details on how the SE will process the images using the various boot models. For each of the images, there is also a set of descriptors that specifies the type of image, image address in the NVM, the boot model, the load/boot address, and other security-related options.

The ATOC is created by the Security Toolkit, which takes input from the on-line [Alif Conductor](#) tool and the binary application images (.elf files) generated by the linker in the IDE application development tools. The ATOC tool takes user input and creates the ATOC image that is programmed to MRAM starting at location 0x0. The size of the ATOC in MRAM can be as small as 1KB, but can grow depending on the device configuration settings and the number of application images in the ATOC. The ATOC image may also be signed as well to protect its integrity.

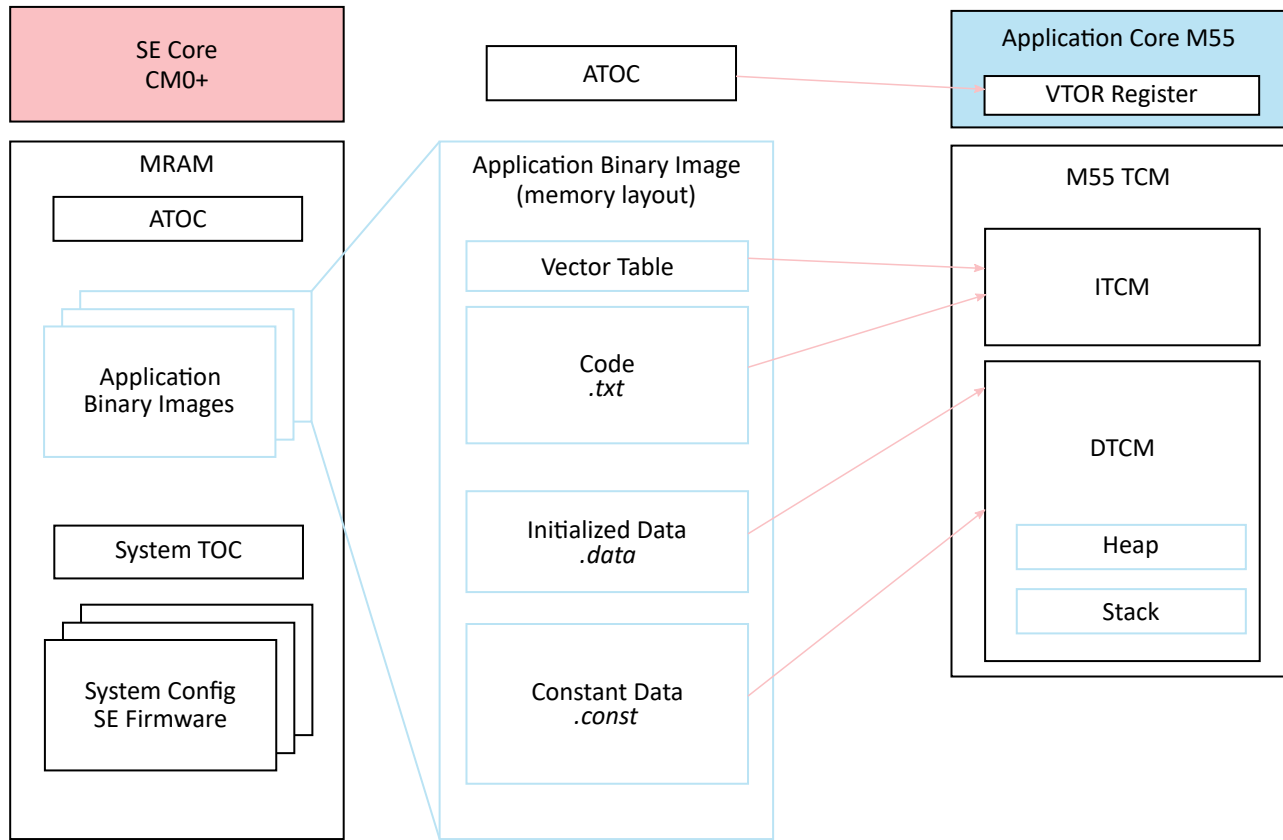
Figure 4-9 Creation of ATOC Images



The ATOC configuration depends on the selected boot model—in the case of MCU or Hybrid mode the application binary has to be placed on a location matching the load address in the scatter file. In case of overlap between different segments in NVM, the SETOOLS will flag an error.

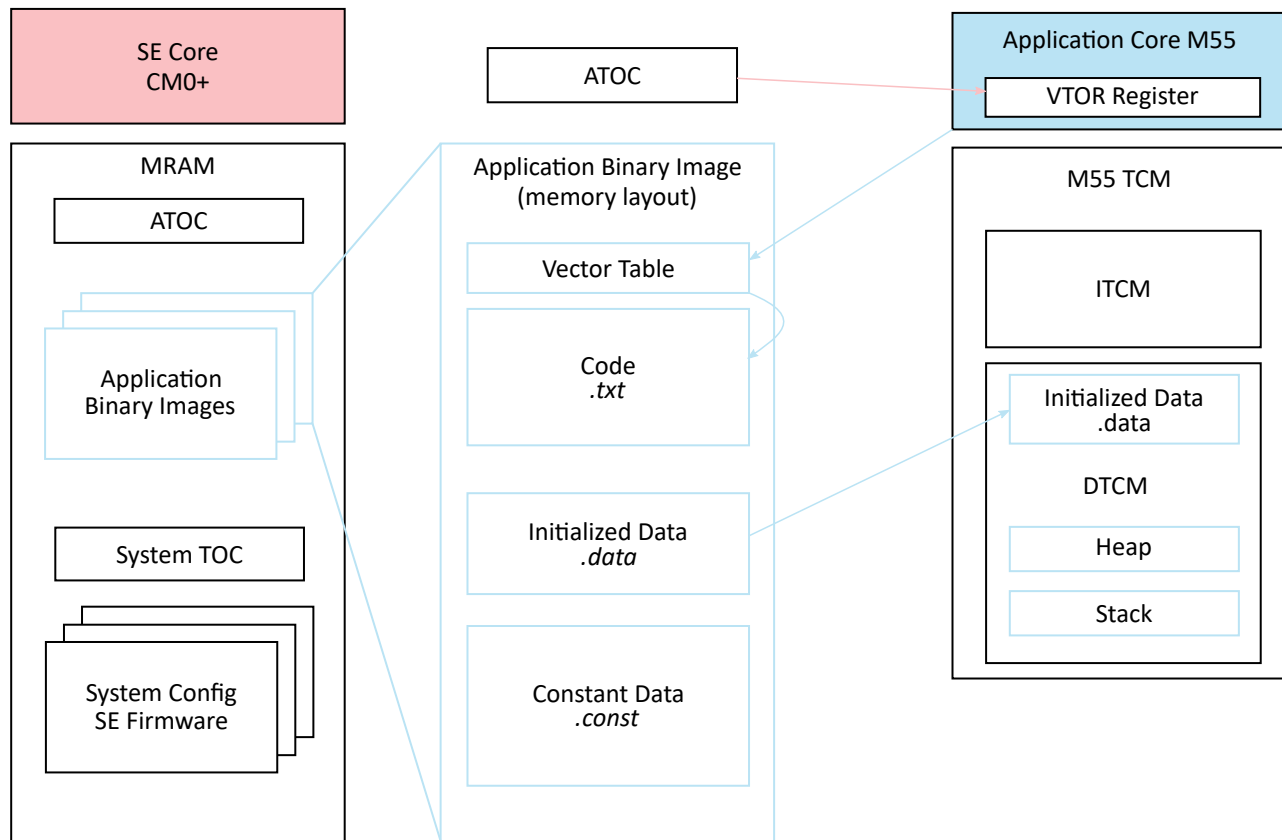
In MPU boot mode, the ATOC will contain images that can be packed one after another because their location in NVM will not need to be coordinated with the scatter file load addresses. The SE performs the data copy operation, moving the code and initialized data from the NVM memory to their execute addresses in the TCM (the red arrows in [Figure 4-10](#)).

Figure 4-10 Example of Application Loading by SE—MPU Boot Mode



In MCU boot mode, the ATOC must be defined in accordance with the execute address specified in the scatter file of each application. This will enable the XIP mode of operation. Prior to booting the application core, SE initializes its vector table pointer (VTOR register). The application core boots autonomously in XIP mode, starting from the reset vector address (the blue arrows in the [Figure 4-11](#)).

Figure 4-11 Example of Application Running in XIP—MCU Boot Mode



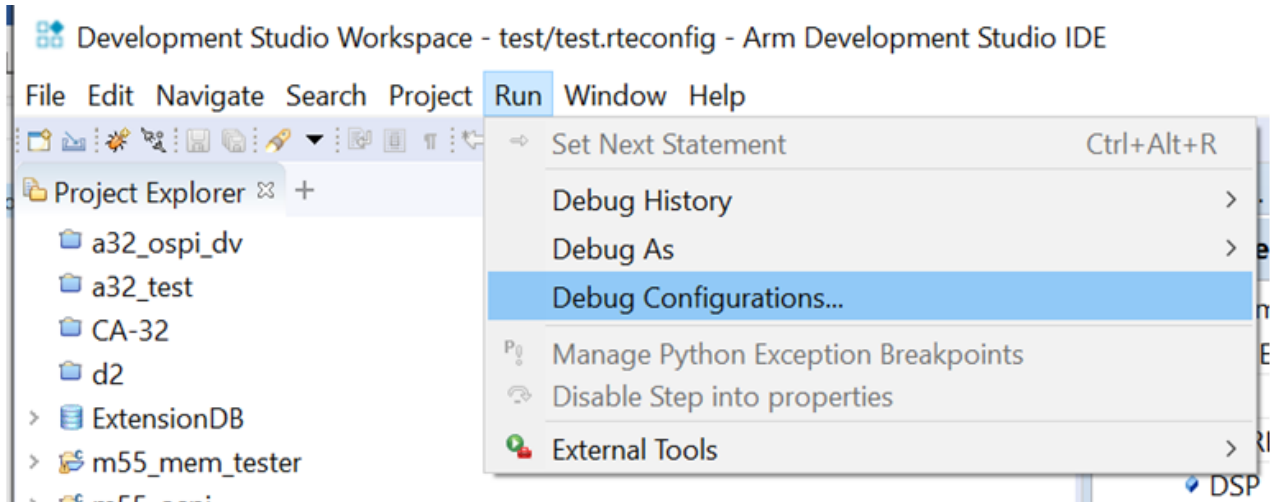
4.8 Debug Configurations and Application Reset Methods

The debug configuration sets references to the application binary image, the debug probe configuration, and a number of other attributes. Most significant among them is the reset method.

Figure 4-12 through Figure 4-14 are examples from Arm DS IDE. For particular IDE guidance, refer to the respective User Guide *Getting Started with Bare Metal Design* available on the Alif Semiconductor [User Guides & App Notes](#) page.

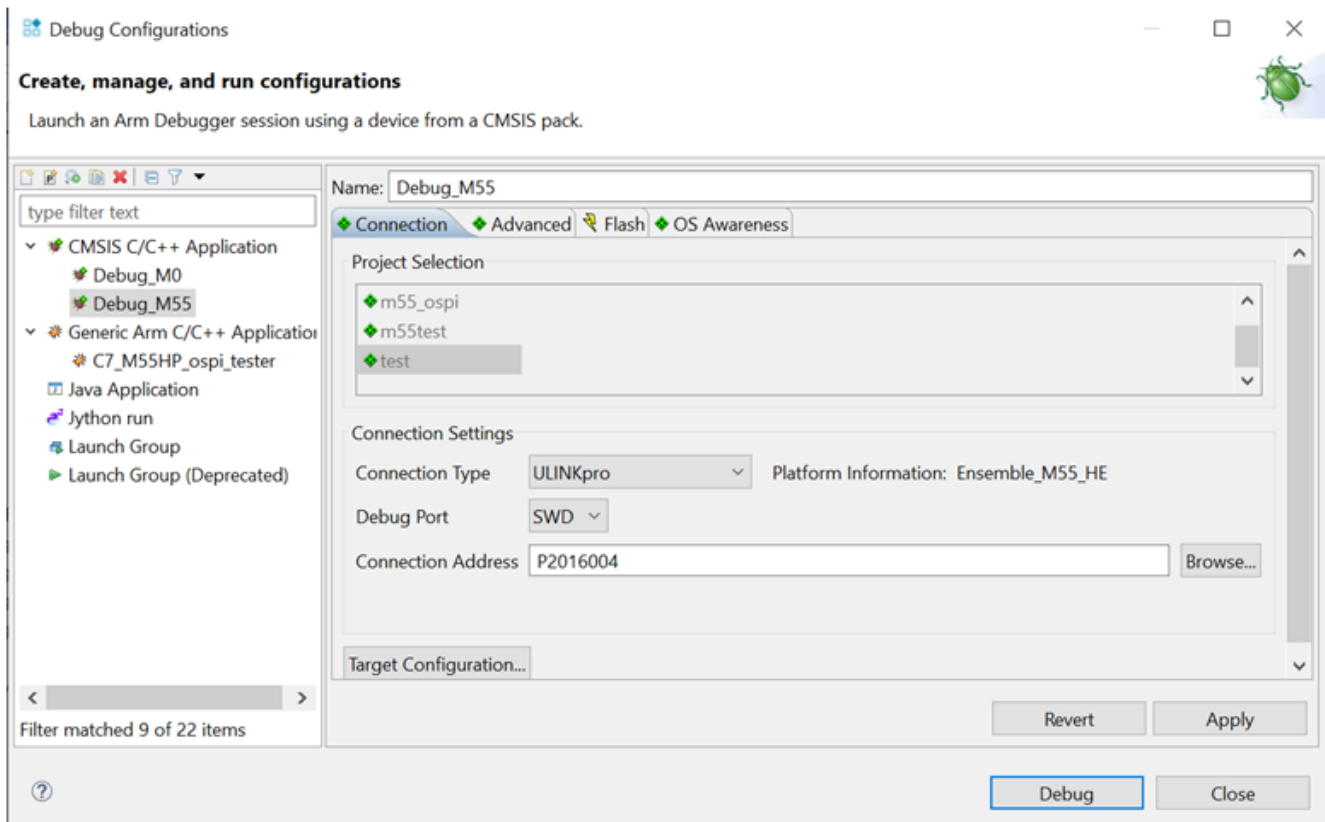
The *Debug Configurations* window can be open from the *Run* tab of the main menu (see Figure 4-12).

Figure 4-12 Debug Configurations Workspace



The *Debug Configurations* associates the application with the debug probe (ULINKpro in [Figure 4-13](#)) and the debug communication protocol (SWD or JTAG®).

Figure 4-13 Configuration with ULINKpro



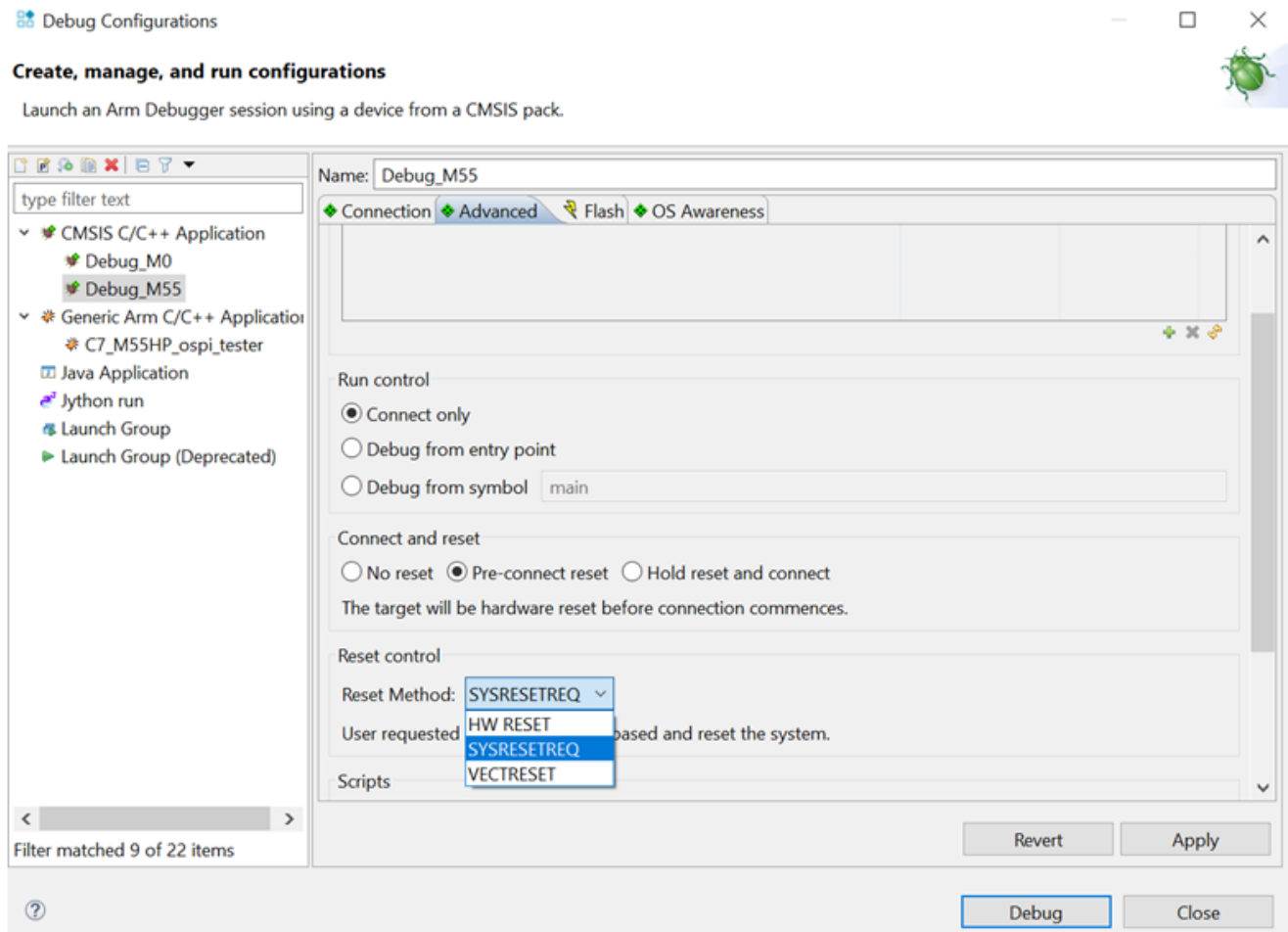
The debugging of the Ensemble applications is different from the debugging of traditional multi-core devices. The reason is that the SE is in control of the reset and boot processes. The hardware reset (through toggling the NSRST of the JTAG interface) is causing a cold restart of the whole device, not just the application being debugged. This results in loss of the debugger connection and is not desirable in most cases.

The *Advanced* tab of the *Debug Configurations* contains *Reset Control* section, which offers different reset options:

- HW RESET—asserting the NSRST signal
- SYSRESETREQ—using a system register write to cause a reset
- VECTRESET—using a processor register write to cause a reset

When debugging a single-core application, it is required to select the *SYSRESETREQ* or the *VECTRESET* option. These options skip the SE boot process if the selected boot method depends on it (see [Figure 4-14](#)).

Figure 4-14 SYSRESETREQ Reset Method



For information on debug probe usage, refer to application note [Segger Ozone and J-Link Debug](#).

5 Integrated Design Environments and Toolchains

Alif Semiconductor's Ensemble fusion processors and microcontrollers are multi-core systems consisting of Arm Cortex-A32 and Arm Cortex-M55 cores. Alif Semiconductor utilizes CMSIS-Pack to distribute peripheral drivers and GitHub for source code distribution. Design can be done using a command line, script-driven flow, or an IDE that combines code editing, project build, and debug functions.

If an IDE is used, it should support:

- Arm Cortex-A32 and/or Arm Cortex-M55 CPU cores
- CMSIS-Pack integration for Cortex-M55
- Framework for creating and building a multi-core project
- Framework for secure and non-secure projects

There are various IDEs out on the market that can fulfill these requirements and Alif Semiconductor is supported by many of them, including Visual Studio Code, Arm® Keil® MDK, IAR Embedded Workbench®, and Arm DS.

For a complete step-by-step example of installing the needed development environment and using it to create an Alif Semiconductor Ensemble application and debug it, refer to the respective User Guide *Getting Started with Bare Metal Design* available on the Alif Semiconductor [User Guides & App Notes](#) page.

6 Application Development Workflow

The application development workflow is the same as that of any CMSIS-based project development.

6.1 Create Device Configuration File

By default, an *all-access* model (all resources available to all cores) is implemented for the Alif Semiconductor devices.

The user can create an intended usage configuration for a device using [Alif Conductor](#) tool. Configuration options available to the user include CPU cores, memory, peripherals, pins, clocks, DMA, power estimation, and miscellaneous configurations.

Once the user creates the desired configuration, a configuration file can be generated. This file is then fed into the Alif Semiconductor Security Toolkit to create a binary image, which is used by the secure boot loader to configure the device.

6.1.1 DevKit and AppKit Configuration

The Ensemble E7 DevKit and AppKit come preprogrammed from the factory with a default *all-access* configuration. This configuration is set up with the kit's pin-outs and allows access to all peripherals by any of the application cores (APSS, RTSS-HP, or RTSS-HE). Clocks are programmed to the default frequencies.

6.1.2 Production Device Configuration Using the Conductor Tool

Use the graphical web-based Conductor tool to configure a chip in 3 major areas:

- Resources Configuration
- Pins Configuration
- Clocks Configuration

The [Alif Conductor](#) tool can be used after proper registration and logging to the Alif Semiconductor site <https://alifsemi.com/>.

The Help Manual provided with the tool under Main menu icon  guides on how to use each of the tool's configuration tabs.

6.1.2.1 Resources Configuration Management

An Alif Semiconductor's device consists of multiple cores, multiple memories and almost all possible peripherals for embedded world. Not all cores, memories or peripherals are required at the same time for a particular project. The Conductor tool offers a way to configure core selection and associate peripherals along with that.

Configuration options supported for any core are:

- Enabling or disabling the core
- Setting the boot address for the core
- Configuring peripheral/memory settings and assigning peripheral/memory to a core

6.1.2.2 Pins Configuration Management

The General Purpose Input Output (GPIO) pins in Alif Semiconductor's devices are multiplexed to support multiple functionalities. At any time, device pins can be configured to work in a particular functionality mode.

Peripherals selected in the *Resources* tab are automatically assigned to the default pins. If the user tries to assign the same pins to different functions, the tool will flag them as errors or warnings.

6.1.2.3 Clocks Configuration Management

Cores and peripherals in an Alif Semiconductor's device can work at different (selectable) clocks. Clocks configuration settings provide options to select the clock source for peripheral and CPU cores.

6.2 Create or Import Project Files into the IDE

After defining the device configuration, the next step is to load projects and associated CMSIS-Pack files that assist in setting up the initialization of the cores and core subsystems (interrupts, etc.). It is recommended that users start from example project files whenever possible.

After deciding on the device configuration to be used, either the default *all-access* configuration or a custom configuration, the next step is to import any needed project files into the IDE being used.

It will be necessary to import generic Arm CMSIS-Pack files needed to support the APSS and RTSS cores in the device, as well as Alif Semiconductor's specific CMSIS-Pack files with device drivers for all the available peripherals in the device.

Generic Arm packs can be found here:

- [ARM.CMSIS pack](#)
- [ARM.CMSIS-Driver pack](#)

Alif Semiconductor specific packs are:

- Pack for CMSIS device support: [alif_ensemble-cmsis-dfp](#)
- Pack for Azure RTOS support: [alif_ensemble-Azure-RTOS](#)
- Pack for Free RTOS support: [alif_ensemble-FreeRTOS-Components](#)

These packs are then combined and used to create a project in the IDE. In the respective development environment, the user can select the CPU core to be used, create the project, then go into the listing of the pack contents, and select the appropriate drivers for the peripherals to be used in the design.

Application source code can be added to a project by either adding a new file from scratch, or importing existing source files. For an initial project, it is recommended that the user imports an existing project and uses that as a model for creating further projects.

6.3 Create Debug Configuration

To have the capability to load and debug a design, it is necessary to create a debug platform definition in the tools and then define a specific debug connection. The process of creating a debug platform associates the particular debug probe connected to the host system with the board to be used.

Once a debug platform is created, the user creates a debug connection that associates that particular debug probe platform with a specific CPU core and executable file, and specifies what to do after the executable is loaded:

- Connect only
- Debug from entry point
- Debug from a specific symbol

6.4 Compile and Build Application

Once the project has been created with all the required pack libraries and user application code, the next step is to compile the code and build the complete application. This process is unique to the particular IDE being used. User source code and applications shall use the same compiler for building purposes.

Any errors noted during the project build need to be taken care of through the inclusion of the necessary library or driver elements, or changes to the user application source code.

Once the project is built with no errors, the user can start debugging the application.

6.5 Debugging a Single Core Application Via Debugger

Before committing the application NVM storage, each individual design will normally be debugged on the target CPU core using a debugger such as ULINKpro. In this case, the system code in the device will perform device setup using the default *All access* configuration, and then the user application code will be loaded into SRAM, where the CPU will begin execution.

The IDE debug tools connect PC's USB port to the target system via a JTAG connector. The debug tools can be used to program, debug, and analyze applications, along with Arm DS.

Once a debug platform has been established and a debug configuration created for the desired CPU core (A32, M55-HP, or M55-HE), the debugger can be started and the user can go through the process of looking at expected output and setting breakpoints if necessary.

If changes are needed to the design, the user can stop the debugger, edit the source code, re-build the design, and then create a new debug connection, and continue the debugging process.

This debug process would be used on the designs for each CPU core to be used in the final application until a set of functional binary executables is available.

6.6 Using NVM Storage

The SETOOLS package allows the user to provision the device. This includes programming MRAM with application binaries for the Arm Cortex-A32 and Arm Cortex-M55 embedded processors available for user applications.

The tools can interface to the device using a ULINKpro debugger connected to the JTAG port, or optionally through a serial port connection to the SE UART, which connects to the SESS (ISP mode). For beta devices (A1 silicon), only ISP mode is supported.

These tools operate on binary executables that are *boot-from-flash* ready and can be programmed into ROM, so it will be necessary to convert the output of the design tools you are using into binary form. Use the appropriate conversion tool for the particular design environment.

6.6.1 Generate New RoT if Necessary

The RoT is created with *app-gen-rot.py*. This operation only needs to be performed once, unless there is a need to recreate the RoT with a new set of keys. The release packet contains a sample RoT with corresponding keys, and certificates that can be used for experimentation. For actual device provisioning, the RoT must be re-generated to create unique keys.

6.6.2 Creating Application Package Binary TOCs

Edit the *build/app-cfg.json* file (or a *.json* file with other name) with images to include in the desired *AppTocPackage.bin* image and copy the binaries into the *build/* directory.

The supplied version of *app-cfg.json* has a default load addresses for each CPU core. If the user is going to let the tools decide where to store the applications in MRAM, it is only needed to specify the binary file name for each CPU and set the *load* and *boot* flags as desired.

Run *app-toc.py* to generate the application package, which will be written out as *AppTocPackage.bin* to the *build/* directory.

6.6.3 Programming the MRAM Memory

Use `app-write-mram.py` to upload the `AppTocPackage.bin` image into the MRAM. On successful completion of the MRAM programming, the program will reset the board to reboot the chip. The first time you run this tool, it will take you through a discovery mode to establish the proper debug connection.

This entire process is described in detail in the *Alif Security Toolkit User Guide* available on the Alif Semiconductor [Software & Tools](#) page.

6.7 Additional Information

For instructions and tools to facilitate development of applications on Ensemble processors refer to [AlifSemi Ensemble SDK](#).

7 Supported Software Device Drivers

7.1 List of Device Drivers Supported in the APSS and RTSS

The following are the list of peripherals supported in the APSS and RTSS:

- Message Handling Unit (MHU)
- Hardware Semaphore (HWSEM)
- Ethernet (ETH)
- Low-Power Real-Time Counter (LPRTC)
- Digital-to-Analog Convertor (DAC)
- Octal SPI—NOR Flash
- APSS Watchdog Timer (WDT_APSS)
- USB Device
- USB Host
- Dual-Role USB
- Secure Digital / MultiMediaCard (SDMMC)
- Camera Parallel interface (CPI)
- MIPI Camera Serial interface (CSI)

For a full list of supported device drivers refer to [Ensemble devices CMSIS Design Family Pack \(DFP\)](#).

For more practical examples of using device drivers with the Alif Semiconductor platforms, refer to the following resources:

1. GitHub Repository: Sample Templates and Use Cases
 - [Alif Semiconductor GitHub Repo](#)
2. Application Note AAPN0026
 - [CMSIS Examples and Templates](#)

7.1.1 Message Handling Unit (MHU)

7.1.1.1 MHU Overview

The MHUs facilitate interrupt-driven communication between two subsystems. Each MHU has two halves namely:

- A Sender frame or Sender
- A Receiver frame or Receiver

The communication between two subsystems is made possible with one subsystem having the Sender, and the other subsystem having the Receiver of a single MHU.

In Ensemble family of devices, MHUs are implemented in pairs to allow for full-duplex communications, with Sender and Receiver being reversed in the MHU pair, that is, each Subsystem A and Subsystem B has Sender and the Receiver of a single MHU, where 4-byte data sent by Sender of subsystem A is received by Receiver of Subsystem B; similarly, 4-byte data sent by Sender of subsystem B is received by Receiver of Subsystem A. By this way full-duplex communication between Subsystem A and Subsystem B is achieved using two MHUs.

For compliance with the TrustZone technology, each subsystem has two MHUs, Secure MHU (MHU0) and Non-secure MHU (MHU1) for secure and non-secure communication, respectively.

7.1.1.2 Inter-Process Communication (IPC) with MHU

As MHU facilitates communication between two subsystems, this makes it ideal to use for IPC. An MHU is made up of two memory-mapped register frames. The first frame is used by the Sender, the second frame is used by the Receiver. Inside each frame, there are:

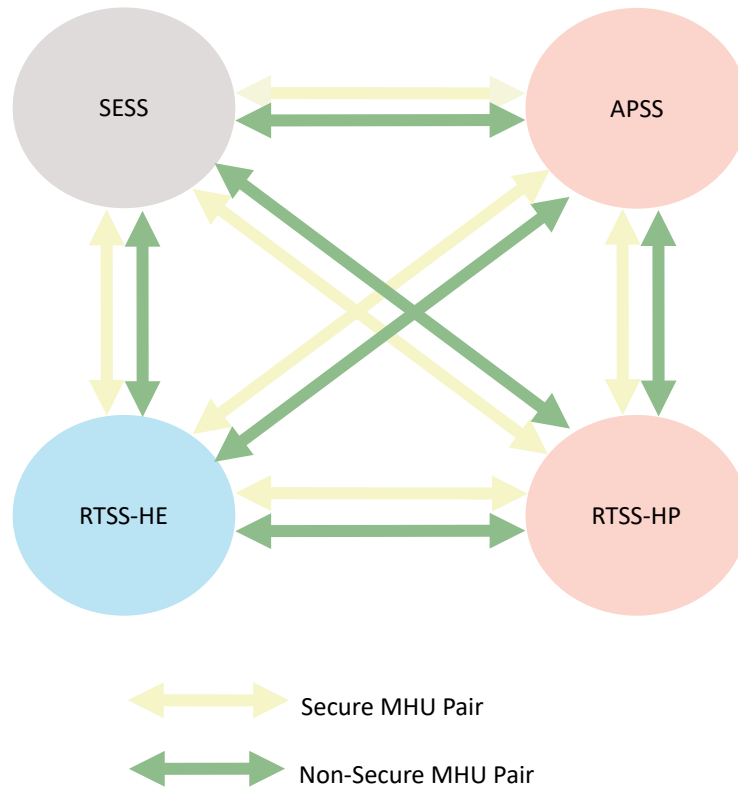
- Two channels that the Sender uses to send transfers to the Receiver in the Ensemble family of devices. Each channel supports 4-byte data transfer.
- Control and identification registers.

7.1.1.3 IPC Topology Using MHU

In Ensemble family of devices, both secure communication and non-secure communication via MHUs are supported between the following cores (see [Figure 7-1](#)):

- APSS and RTSS-HP
- APSS and RTSS-HE
- APSS and SESS
- SESS and RTSS-HP
- SESS and RTSS-HE
- RTSS-HP and RTSS-HE

Figure 7-1 MHUs Among the Processor Entities



There are 12 unidirectional MHUs (highlighted in yellow) used across SESS, APSS, RTSS-HP, and RTSS-HE for secure communication, and similarly, 12 unidirectional MHUs (highlighted in green) used for non-secure communication. Each pair (one MHU at Subsystem A, other MHU at Subsystem B) of MHUs supports full-duplex (2-sided) communication between two subsystems.

7.1.2 Hardware Semaphore (HWSEM)

7.1.2.1 HWSEM Overview

A need for synchronization while accessing a shared resource (memory or peripheral) by two or more independent processing subsystems is fulfilled by the HWSEM. By accessing shared resources with synchronization, the possibility of race conditions, deadlock, abnormal behavior among independent processing subsystems can be avoided.

Requests from multiple bus masters to access a shared resource are made atomic and serialized to avoid conflicts and race conditions. This is achieved by making processing subsystems wait for the HWSEM interrupt to occur before locking/acquiring HWSEM. The locking/acquiring of HWSEM by a specific processing subsystem is done by writing its master ID (4 bytes of data) into the HWSEM registers before accessing a shared resource. The processing subsystem holding the HWSEM can lock it multiple times, and a corresponding counter gets incremented. Any attempt to lock/acquire the same HWSEM by any other bus master is refused or blocked until the processing subsystem holding the HWSEM is released. The counter value is decremented upon every HWSEM release. If the counter value reaches zero, then HWSEM release/free is signaled through an interrupt. The other processing subsystems waiting to acquire the HWSEM can handle this interrupt by trying to acquire the HWSEM.

In Ensemble family of devices, there are up to 16 instances of HWSEM, which are included among the shared peripherals. The interrupt requests of these HWSEMs are shared across the multiple processing subsystems.

7.1.2.2 HWSEM Design

The locking/acquiring, unlocking/releasing of HWSEM are controlled by register write operations. [Table 7-1](#) shows the register base address of each HWSEM.

Table 7-1 HWSEM Base Addresses

Module Name	Register Base Address (BASE_ADDR)
Hardware Semaphore 0 (HWSEM0)	0x4902_E000
Hardware Semaphore 1 (HWSEM1)	0x4902_E010
Hardware Semaphore 2 (HWSEM2)	0x4902_E020
Hardware Semaphore 3 (HWSEM3)	0x4902_E030
Hardware Semaphore 4 (HWSEM4)	0x4902_E040
Hardware Semaphore 5 (HWSEM5)	0x4902_E050
Hardware Semaphore 6 (HWSEM6)	0x4902_E060
Hardware Semaphore 7 (HWSEM7)	0x4902_E070
Hardware Semaphore 8 (HWSEM8)	0x4902_E080
Hardware Semaphore 9 (HWSEM9)	0x4902_E090
Hardware Semaphore 10 (HWSEM10)	0x4902_E0A0
Hardware Semaphore 11 (HWSEM11)	0x4902_E0B0
Hardware Semaphore 12 (HWSEM12)	0x4902_E0C0
Hardware Semaphore 13 (HWSEM13)	0x4902_E0D0
Hardware Semaphore 14 (HWSEM14)	0x4902_E0E0
Hardware Semaphore 15 (HWSEM15)	0x4902_E0F0

The following register offsets are used to control the state of each HWSEM:

- BASE_ADDR + 0x0 offset
 - Write operation—The processing subsystem writes his master ID to acquire/lock HWSEM. If granted, HWSEM count at BASE_ADDR + 0x4 will increment.
 - Read operation—The processing subsystem reads the current ID of the granted processing subsystem.
- BASE_ADDR + 0x4 offset
 - Write operation—The processing subsystem writes his master ID to release semaphore, if the written master ID matches with the ID at BASE_ADDR + 0x0, then the HWSEM count will decrement.
 - Read operation—The processing subsystem reads the latest HWSEM count.

7.1.2.3 Inter-Process Synchronization with HWSEM

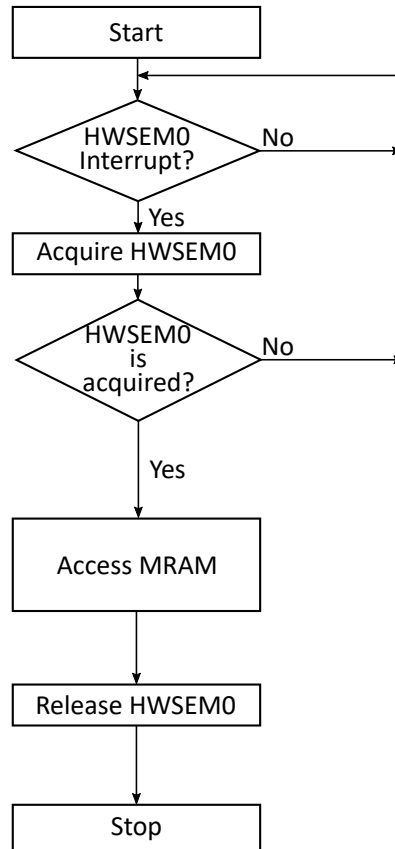
The primary use case of using HWSEM is to achieve inter-process synchronization while accessing a shared resource by a multiple independent processing subsystem. Having synchronization while accessing shared resources eliminates race conditions, deadlocks, and abnormal behavior among different subsystems. Each subsystem should follow the steps below to achieve inter-process synchronization while accessing shared resources:

1. The processing subsystems wait for the HWSEM interrupt to occur.
2. Acquire a specific HWSEM before accessing a shared resource, by writing its master ID into BASE_ADDR + 0x0 offset.
3. Check whether the HWSEM acquisition is successful by reading the contents of BASE_ADDR + 0x0 offset with the written master ID. If the contents match, then the HWSEM acquisition is successful.
4. Optionally multiple threads can acquire the same HWSEM by writing master ID into BASE_ADDR + 0x0 offset. Check the HWSEM count by reading contents at BASE_ADDR + 0x4 offset.
5. Access the shared resource.
6. Unlock HWSEM by writing its master ID into BASE_ADDR + 0x4 offset.

7.1.2.4 HWSEM Example Scenario

The HWSEM0 is used to achieve synchronization while accessing the MRAM (non-volatile memory—a shared resource) by multiple processing subsystems. Each subsystem must lock the HWSEM0 before accessing the MRAM to avoid race conditions and data integrity issues. [Figure 7-2](#) shows how each subsystem should access the MRAM using HWSEM0 without having race conditions and data integrity issues.

Figure 7-2 Flow Chart for Accessing Shared Resource MRAM Using HWSEM0



7.1.3 Ethernet (ETH)

7.1.3.1 ETH Overview

Ethernet is a traditional technology for connecting devices in a wired Local Area Network (LAN) or Wide Area Network (WAN), enabling them to communicate with each other via a protocol, a set of rules, or a common network language.

Ethernet lies in the lower layers of the Open Systems Interconnection (OSI) model as shown in [Figure 7-3](#).

It facilitates the operation of physical and data link layers. The OSI model, which is composed of seven layers, illustrates how various communication protocols work together.

The Ethernet MAC is the media access controller. The Ethernet MAC is defined by the IEEE-802.3 Ethernet Standard. IEEE 802.3 is a working group and a collection of Institute of Electrical and Electronics Engineers (IEEE) standards produced by the working group defining the physical layer and data link layer's Media Access Control (MAC) of wired Ethernet. The Ethernet MAC implements the data-link layer in the OSI model.

Ethernet PHY handles the physical layer of the OSI network model (see [Figure 7-4](#)). It implements the physical layer portion of the Ethernet. Its purpose is to provide analog signal physical access to the link.

Figure 7-3 OSI Model

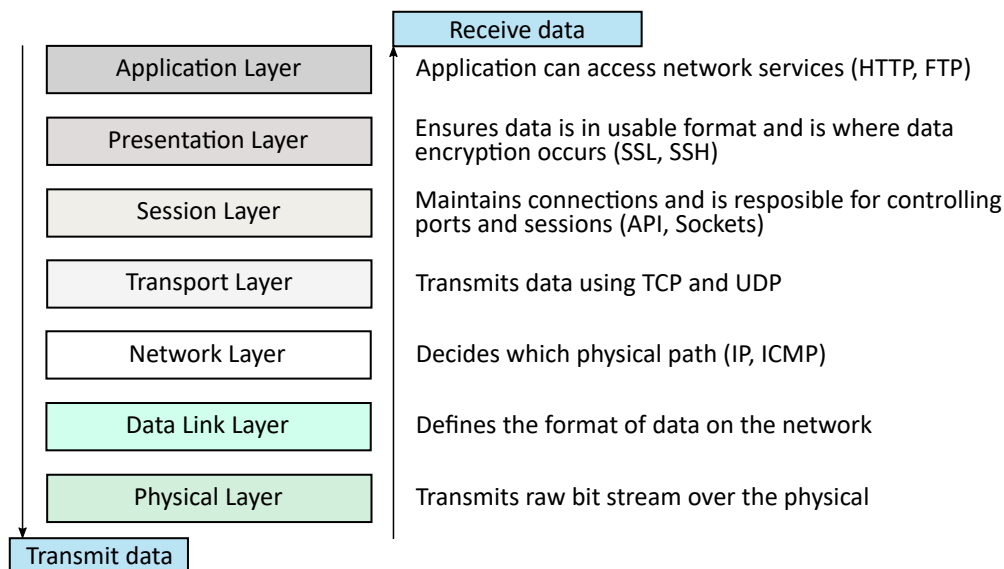
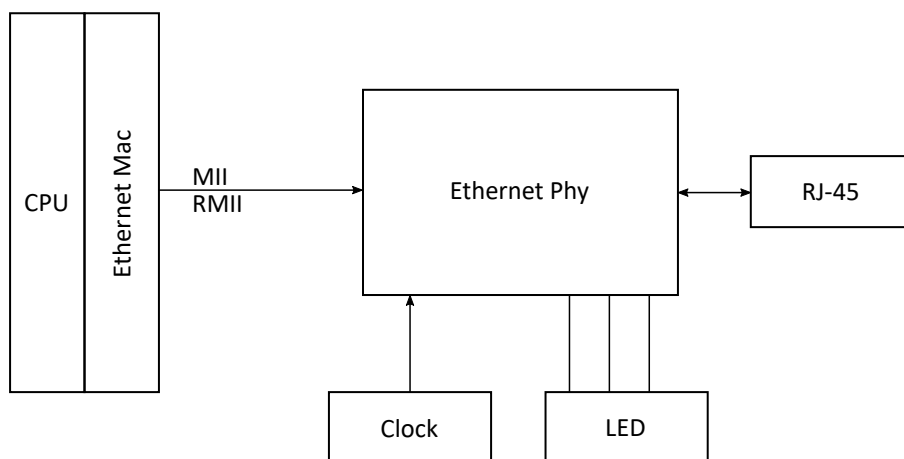


Figure 7-4 Ethernet PHY Environment



The ETH controller in the device functions as the Ethernet MAC. The ETH is compliant with the IEEE 802.3-2015 Standard and can be used in applications such as switches, network interface cards, and data center bridges and nodes. The ETH controller enhances Ethernet performance by significantly increasing the number of transmit and receive channels, adding support for new features in TCP/IP offloading, and enabling data center bridging. The ETH controller supports various interfaces like Native interface, AHB interface, AXI interface, and PHY interface.

The PHY interface provides the following capabilities:

- Gigabit Media Independent Interface (GMII)
- Reduced Gigabit Media Independent Interface (RGMII)
- Serial Gigabit Media Independent Interface (SGMII)
- Ten Bit Interface (TBI)
- Reduced MII (RMII)
- Serial MII (SMII)

- Reduced TBI (RTBI)
- Reverse MII (RevMII)

Features supported in the driver:

- Speed support for 10/100 Mbps
- Support for RGMII PHY configurations
- Support for common *Ethtool* queries
- NAPI support
- Full/Partial checksum offload support
- Multi-queue support
- Transport Layer Security (TLS)

Table 7-2 shows the register base address of the Ethernet.

Table 7-2 Ethernet Base Addresses

Module Name	Register Base Address (BASE_ADDR)
ETH	0x4810_0000

7.1.4 Low-Power Real-Time Counter (LPRTC)

7.1.4.1 LPRTC Overview

The Low-Power RTC is a programmable peripheral. The LPRTC is a clock source function used by systems that measure time, even when the main device CPU is powered off. LPRTCs are present in almost any electronic device that needs to keep the accurate time of day. LPRTCs often have an alternate source of power, allowing them to keep time even the primary source of power is off or unavailable.

The LPRTC module supports the following main features:

- Incrementing counter and a comparator for interrupt generation
- Prescaler counter
- Free-running PCLK

Some applications of the LPRTC are:

- Real-time clock—to keep track of the current time
- Long-term, exact chronometer—to keep track of time from now up to 136 years in the future (when clocked with a 1 Hz clock signal)
- Alarm function—to generate an interrupt after a programmed number of cycles
- Long-time, base counter—clocked with a kHz range clock signal

Table 7-3 shows the register base address of the LPRTC.

Table 7-3 LPRTC Base Addresses

Module Name	Register Base Address (BASE_ADDR)
LPRTC	0x4200_0000

The LPRTC module is driven by a 32.768 kHz clock. The Linux driver for LPRTC located in the *drivers/rtc* path of the Alif Semiconductor Linux tree, plugs into the LPRTC framework with *rtc_register_device()* *api* call. This driver provides the following features:

- Read and set time using the *hwclock* utility
- Set alarms and get interrupts when the alarm time has elapsed.

7.1.5 Digital-to-Analog Convertor (DAC)

7.1.5.1 DAC Overview

DAC is a module that converts a digital signal to an analog signal. The number of DAC bits is typically chosen based on the desired resolution and accuracy for applications within the given system. The Ensemble family of devices contains up to two 12-bit DAC modules, each having the following features:

- Dual-channel DAC converter controlled independently
- 5 MHz sampling rate with minimum 50 ns t_{on} and minimum 150 ns t_{off} pulse signal
- Conversion of unsigned binary or two's complement digital data
- Retiming circuit supporting an option to load digital data from registers
- Programmable current, load resistance and load capacitance
- Maximum current consumption up to 1 mA
- Required 0.9 V external reference voltage and 10 μ A reference current

7.1.5.2 DAC Design

Table 7-4 shows the register base address of the DAC.

Table 7-4 DAC Base Addresses

Module Name	Register Base Address (BASE_ADDR)
DAC120	0x4902_8000
DAC121	0x4902_9000

The following register offsets are used to control the state of each DAC.

- BASE_ADDR + 0x0 offset
 - Write operation—This register is the control register, and a specific value must be written to enable DAC120 and DAC121. 0x0E31_C7FD needs to be written to DAC120 to enable it. 0x0631_C7FD needs to be written to DAC121 to enable it.
 - Read operation—Read back value written to this register.
- BASE_ADDR + 0x4 offset
 - Write operation—The 12 lower bits are the digital input value that is converted to the analog output value
 - Read operation—Readback the value written to this register.

The kernel driver for the DAC module is in the *drivers/iio/dac/ensemble.c* location in the Alif Semiconductor Linux kernel source. The driver registers itself with the IIO subsystem in Linux, and provides the capability to set and read back values to the DAC. Since it is a 12-bit DAC, the value that can be set ranges between 0 and 4095. The voltage on the output pin ranges from 0 V to 1.3 V.

7.1.6 Octal SPI (OSPI)

7.1.6.1 OSPI Overview

OSPI is a serial synchronous communication protocol that enables communication with serial memories, including external single, dual, quad, or octal SPI memories. The external memory device can be memory-mapped, allowing any system to access it like internal memory. The Octal Flash memory devices are designed for use in IoT, smart devices, and embedded processors that demand high performance. An octal flash memory is for code storage in embedded systems because it supports XIP. This functionality is designed to

execute code directly from the Flash memory in sync with the cache controller, without needing to first copy the code to a RAM. OSPI is the interface to the NOR Flash memory device.

OSPI is configured to operate in standard and enhanced modes of Motorola SPI protocol. In standard mode, it acts as a four-wire, full-duplex serial master, with four possible combinations for the serial clock phase and polarity. In enhanced mode, SPI can be configured to operate in Dual, Quad, or Octal mode.

OSPI acts as an interface to Flash memories in two modes:

- Regular read and write
- XIP set to RO mode

When OSPI is configured in XIP mode, the incoming data can be optionally decrypted on-the-fly via a dedicated hardware AES engine.

7.1.6.2 OSPI Design

Table 7-5 shows the register base address of the OSPI.

Table 7-5 OSPI Base Addresses

Module Name	Register Base Address (BASE_ADDR)
OSPIO	0x8300_0000
OSPI1	0x8300_2000

In order to boot the Linux kernel from the OSPI flash memory, first the kernel needs to be written in regular read-write mode, then the driver in *bl32.bin* sets the OSPI to XIP mode, which makes it possible to boot Linux.

Registers related to XIP mode set on OSPI are:

- OSPI_CTRLR0[DFS] bit field sets the Frame format
- OSPI_SPI_CTRLR0 sets the Address length, Wait Cycle, Transaction type
- OSPI_XIP_WRAP_INST sets the op-code for Burst read
- OSPI_XIP_INCR_INST sets the op-code for Regular read

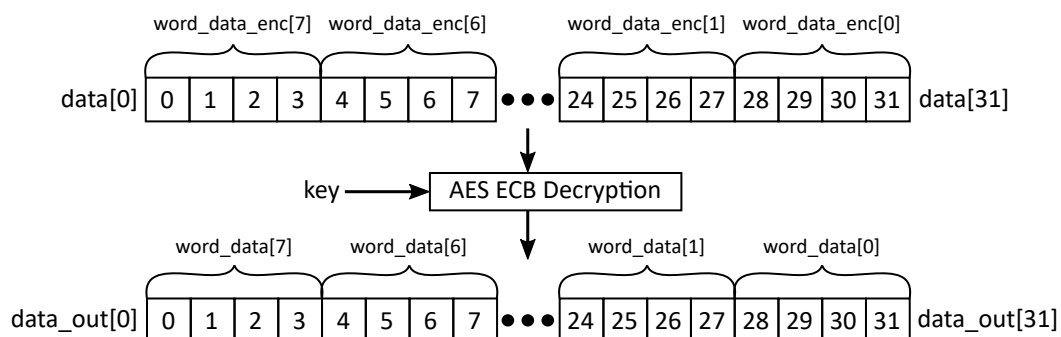
7.1.6.3 OSPI with AES

The Advanced Encryption Standard (AES) is the algorithm used for block encryption. In the device, the size of an AES block is 256 bits (32 bytes), and it uses the Electronic Code Book (ECB) mode.

The user can utilize this feature by writing an encrypted image or a binary to the Flash.

Decryption is done through a hardware AES block, see Figure 7-5.

Figure 7-5 AES ECB Decryption



A Python script is employed to encrypt the Linux XIP kernel image and other binaries.

Once the OSPI is programmed to decrypt data by providing 256-bit key, the decryption function should be invisible to the user. This is all taken care of by the same driver in *bl32.bin* that sets up the OSPI—NOR Flash for XIP.

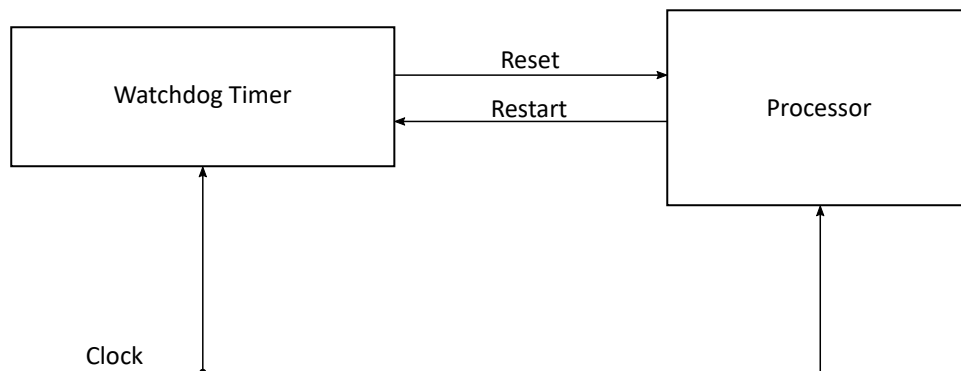
7.1.7 Watchdog Timer (WDT_APSS)

7.1.7.1 WDT_APSS Overview

Most embedded systems need to be self-reliant. It's not usually possible to wait for someone to reboot them if the software hangs or the system crashes. Some embedded designs, such as space probes, are simply not accessible to human operators. If their software ever hangs, such systems are permanently disabled. In other cases, the speed at which a human operator might reset the system may be too slow to meet the product's uptime requirements.

The WDT is a hardware component that can be used to automatically detect software anomalies and reset the processor if any occur. The WDT is based on a counter that counts down from some initial value to zero. The embedded software sets the counter's initial value and periodically restarts it. If the counter ever reaches zero before the software restarts it or refreshes the counter, the software is presumed to be malfunctioning and the processor's reset signal is asserted. The processor (and the embedded software it's running) is restarted as if a human operator had cycled the power. [Figure 7-6](#) shows the operation of the WDT module.

Figure 7-6 WDT Operation



The Arm Server Based System Architecture (SBSA) WDT (or also WDT_APSS) configured in Linux supports two stages—single-stage and two-stage WDT—and it is set up by the module parameter *action* defined in the driver. WS0 and WS1 signals define the single- and the two-stage WDTs.

The Arm SBSA generic WDT has two-stage timeouts: the first signal (WS0) alerts the system via an interrupt, while the second signal (WS1) triggers a real hardware reset.

In the single-stage mode, when the timeout is reached, the system is reset by WS1 signal. The first signal (WS0) is ignored.

In the two-stage mode, when the timeout is reached, the first signal (WS0) triggers panic. If the system encounters an issue and cannot be reset by panic or refreshed by accessing watchdog registers through an application or driver, then the second stage is reached, and system is reset by WS1. [Figure 7-7](#) shows the working of two-stages WDT.

Figure 7-7 WDT 2-stage Operation

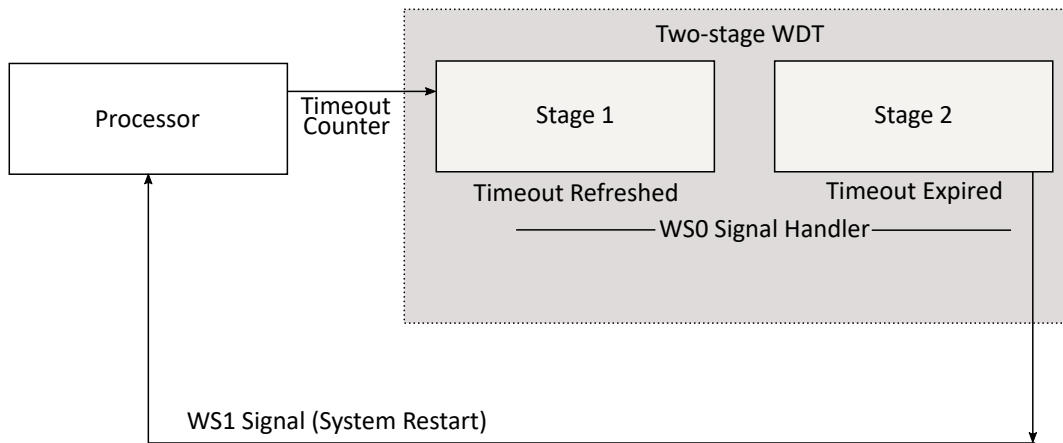


Table 7-6 shows the base addresses of Refresh and Control register set of the WDT module.

Table 7-6 WDT_APSS Base Addresses

Module Name	Register Base Address (BASE_ADDR)
WDT Control register (WDT_AP_CTRL)	0x1A30_0000
WDT Refresh register (WDT_AP_REFRESH)	0x1A31_0000

7.1.8 USB Device

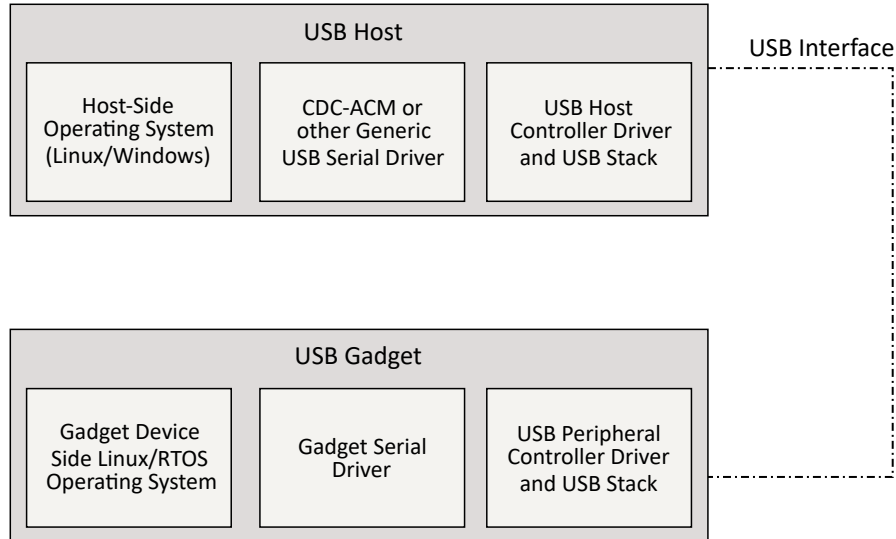
7.1.8.1 USB Overview

Universal Serial Bus is an industry-standard external serial bus interface for connecting peripheral devices to an embedded device or computer. Certain devices have USB Device Controllers or Dual-Role Controllers that can operate in either Host mode or Device mode.

A Device mode cable allows connection to a USB host, such as a PC, and it can be either USB-B or USB-C cable. When used in this mode, the device needs to have a gadget driver loaded, which implements the required attributes for the device type, such as CDC-ACM, CDC-ECM, and CDC-MS devices. These devices are categorized as Composite devices with more than one functionality. To configure or use a USB Device as a serial device gadget, a *g_serial* gadget driver in the kernel is required.

Figure 7-8 shows the components required for a USB Host and USB Device to function as a USB serial gadget and its related class components in the USB Host. The USB interface is the physical connection between USB Host and USB Device, using either USB-B or USB-C cable.

Figure 7-8 USB Device Used as a USB Serial Gadget



7.1.8.2 Dual-Role USB

A Dual-Role USB controller works as either USB Host or USB Device mode, depending on the requirement. To configure the target as a Peripheral Gadget or Device, the USB controller is set to peripheral mode along with the gadget driver according to the class requirements.

The main features of the Dual-Role USB controllers are:

- Dual-Role Device (DRD) capability. Can be configured as a Host or Device
- The same programming model for SuperSpeed (SS), High-Speed (HS), Full-Speed (FS), and Low-Speed (LS)
- Internal DMA controller
- LPM protocol in USB 2.0

The USB controller is configured as a high-speed USB 2.0 serial gadget and once connected through a USB-B micro end to the target board and USB-A end to the Linux host, it is detected as a CDC ACM device, with the serial device `/dev/ttyACM0` on the host machine and `/dev/ttyGSO` on the target machine. USB uses master-slave protocol and all transactions should be initiated by the USB Host.

Table 7-7 shows the base addresses of USB controller and gadget driver.

Table 7-7 USB Base Addresses

USB Controller and Gadget Driver Name	Register Base Address (BASE_ADDR)
USB	0x4820_0000
<i>g_serial</i>	N/A

7.1.9 USB Host

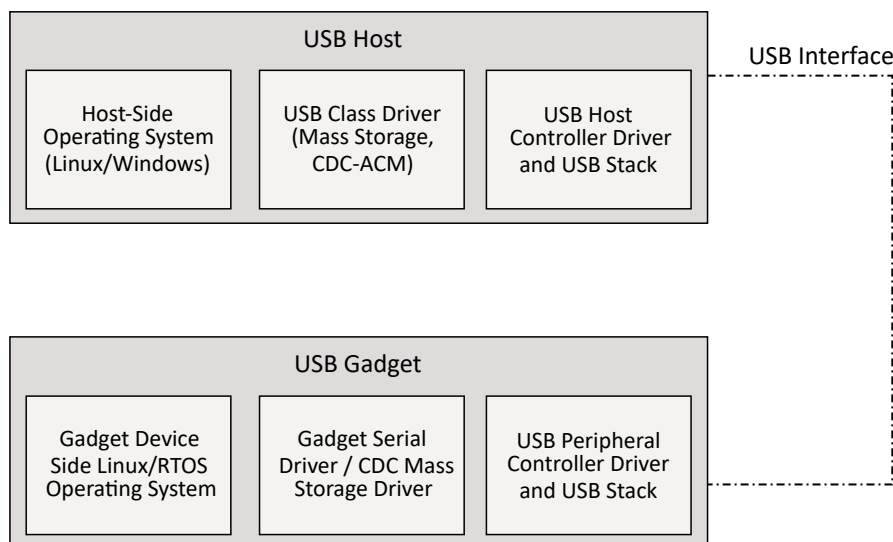
7.1.9.1 USB Host Overview

Standard USB uses a Host-Device architecture; a Host acts as a controller device for the entire bus, and a USB Device acts as a follower device. In standard USB, devices must assume one role or the other, with computers generally set up as Hosts, while, for example, printers normally function as Devices or Peripherals role.

Embedded targets can be configured in a Dual-Role, and they can be either Host or Device depending upon

the requirement. When a device is plugged into the USB bus, the master Device, or Host, sets up communications with the device and handles service provisioning. The Host's software enables or does the needed data-handling, such as file managing or other types of data communication or function. In order to communicate with a device, the Host must be aware of the device's class, such as a mass storage device or a CDC ACM device. [Figure 7-9](#) shows components in USB Host and USB Device drivers.

Figure 7-9 Components in USB Host and USB Gadget



7.1.10 USB Host or Device Controller

A Dual-Role USB controller operates as either USB Host or USB Device mode, depending on the requirement. To configure the target in Host mode, the USB controller is configured as a Host along with the Class driver, according to the type of device to be connected.

The main features of the Dual-Role USB controller are:

- DRD capability. Can be configured as a Host or Device
- Same programming model for SuperSpeed (SS), High-Speed (HS), Full-Speed (FS), and Low-Speed (LS)
- Internal DMA controller
- LPM protocol in USB 2.0

The USB controller is configured as high-speed USB 2.0 Host if any device is connected through a USB-A end on the target board and USB-B micro end on the Linux gadget. The connected USB Device is detected as either a CDC ACM device or a mass storage device, depending on the class of the connected device and the availability of its driver on the USB Host side. A mass storage device is detected as a storage device with mount points `/dev/sda*`, while the ACM class device is detected as `ttyACM0`. As USB is master-slave protocol all transactions are initiated by USB Host.

[Table 7-7](#) shows the base addresses of the USB controller registers.

Table 7-8 USB Controller Base Addresses

USB Controller and Class Driver Name	Register Base Address (BASE_ADDR)
USB	0x4820_0000
Mass storage / CDC ACM	N/A

7.1.11 Secure Digital / MultiMediaCard (SDMMC)

7.1.11.1 SDMMC Overview

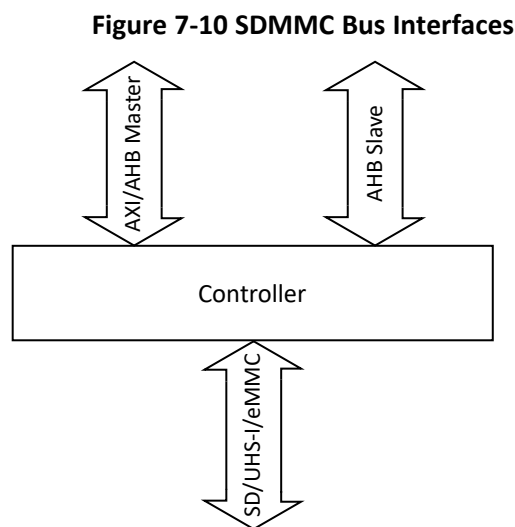
SDMMC is a configurable Secure Digital (SD) host controller module, designed to communicate with memory cards targeted for the mobile/portable markets.

The Secure Digital / MultiMediaCard (SDMMC) module is a highly configurable and programmable, high-performance mobile storage host controller, featuring AXI/AHB as the bus interface for data transfer.

SDMMC supports 4-bit data bus width, and Ultra High-Speed Interface (UHS-I) interface towards the physical layer in the legacy SD modes.

7.1.11.2 SDMMC Interfaces

Figure 7-10 shows the bus interfaces of the SDMMC.



7.1.11.2.1 Slave Interface

The SDMMC supports the AHB slave interface, designed to integrate with the AMBA High-performance Bus (AHB) on the application side. This is a slave interface to host CPU and is used for programming host controller registers.

7.1.11.2.2 Master Interfaces

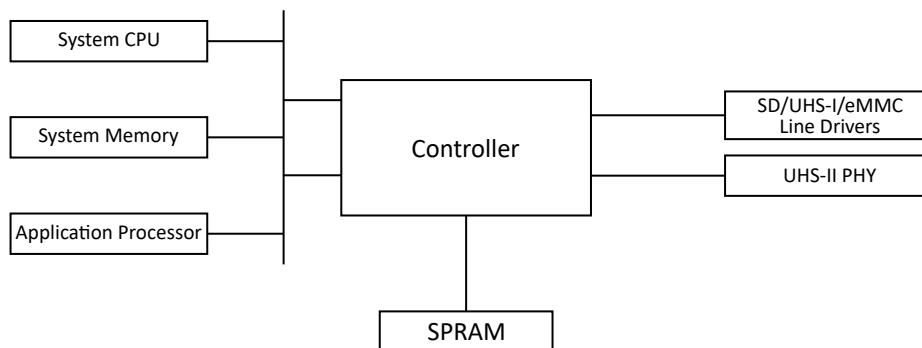
SDMMC supports configurable master interfaces. Either AHB or AXI can be selected as the protocol for the bus master interface.

- **AHB master interface**—The AHB master interface is designed to integrate with the AHB bus master interface unit that is compliant with the AMBA 2.0 Specification. The AHB master interface is used to translate Read/Write transfers initiated by the DMA/CQE (using a generic master interface) into AHB compliant transfers on the system bus. The AHB interface transfers the data to and from the system memory through AHB master interface.
- **AXI master interface**—The AXI master interface is designed to integrate with AMBA AXI bus on the application side. The AMBA AXI 3 protocol for the master interface is supported. The AXI master interface is used to translate Read/Write transfers initiated by the DMA/CQE (using a generic master interface) into AXI 3-compliant bus transfers. The AXI interface transfers the data to and from the system memory through the AXI master interface.

7.1.11.3 System-Level Block Diagram

SDMMC bridges the device bus and UHS-II PHYs/Line drivers. SDMMC also includes RAM interfaces for data storage. The descriptor caching and register caching are done internally. The data storage memory must be instantiated external to SDMMC, see [Figure 7-11](#).

Figure 7-11 SDMMC Block Diagram



The SDMMC module supports the following main features:

- SD card interface:
 - 4-bit interface
 - Default Speed (DS), High-Speed (HS), SDR12, SDR25, and SDR50 speed modes
 - SDIO read wait
 - SDIO card interrupts in both 1-bit and 4-bit modes
 - Wake up on card interrupt
 - Supports the command queuing engine
 - ADMA data transfer types for SD
 - SD memory compliant with SD HCI specification
 - Host Version 4. SD Host Controller Specification Version up to 4.20.

SD card clock frequencies for different speed modes are shown in [Table 7-9](#).

Table 7-9 SD Card Clock Frequencies

SD Speed Mode	Clock Frequency for CCLK_TX/CCLK_RX (in MHz)
SDR12	25
SDR25	50
SDR50	100

NOTE

Clock frequencies are validated for SDR12 and SDR25 modes only.

[Table 7-10](#) shows the base addresses of SDMMC module registers.

Table 7-10 SDMMC Module Base Addresses

Module Name	Register Base Address (BASE_ADDR)
SDMMC	0x4810_2000

7.1.12 Camera

7.1.12.1 Camera Overview

The Camera controller receives pixel data from external camera sensors and writes it to the on-chip memory. The controller supports single frame capture mode and continuous/time-lapse operation mode.

There are two video data input paths supported (see [Figure 7-12](#)):

- Parallel video data input path
 - External Camera → Camera Parallel Video Input Interface → Camera Controller → Memory
- Serial video data input path
 - External Camera → MIPI D-PHY RX → MIPI CSI-2 Controller → Camera Controller → Memory

NOTE

The two data paths cannot be used concurrently. The Camera controller can process only one video input at a time.

Figure 7-12 Camera Interfaces

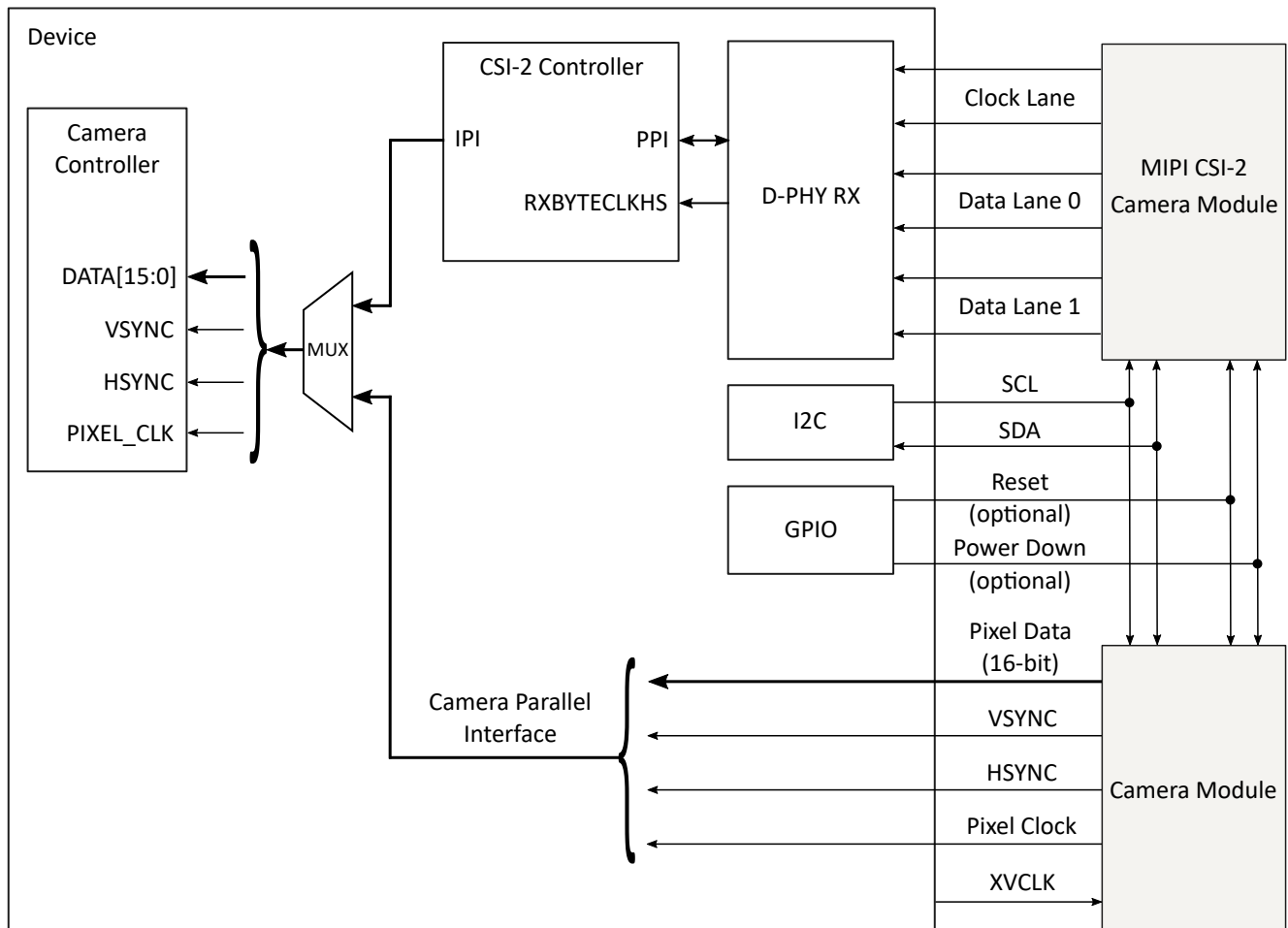


Table 7-11 shows the base addresses of Camera registers to access.

Table 7-11 Camera Base Addresses

Module Name	Register Base Address (BASE_ADDR)
CPI	0x4903_0000
CSI	0x4903_3000

7.1.12.2 Camera Parallel Interface (CPI)

The digital CPI enables a connection to CMOS sensors and camera modules with an 8 to 16-bit wide parallel interface. The data transfer rate is limited by the maximum supported pixel clock and data bus width.

The CPI supports the following main features:

- Configurable 8-, 10-, 12-, 14- or 16-bit data bus width
- Up to 100 MHz pixel clock
- Horizontal and vertical synchronization signals
- Pixel clock output to external camera module

7.1.12.3 MIPI Camera Serial Interface 2 (CSI)

The MIPI Camera Serial Interface 2 controller implements the MIPI CSI-2 protocol on the host side, facilitating data reception from a MIPI CSI-2 compliant camera sensor. A MIPI D-PHY receiver acts as the physical layer.

The CSI controller supports the following features:

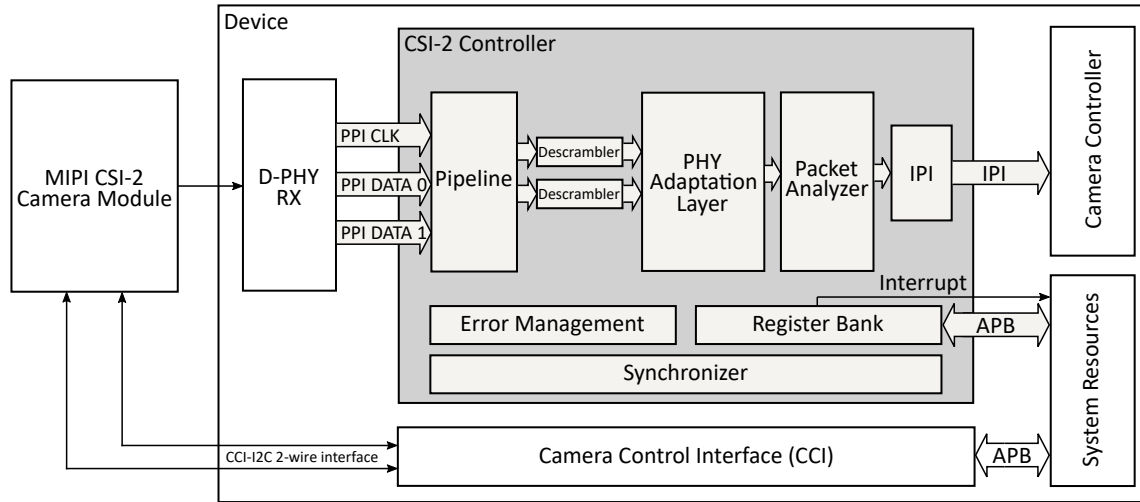
- Compliant with MIPI Alliance standards:
 - MIPI Alliance Specification for Camera Serial Interface 2 (CSI-2), Version 1.2, January 2014
 - MIPI Alliance Specification for D-PHY, Version 1.2, September 2014
- Up to two D-PHY RX data lanes
- Up to 2.5 Gbps per lane in D-PHY
- Dynamically configurable multi-lane merging
- Long and short packet decoding
- Timing accurate signaling of frame and line synchronization packets
- Several frame formats
 - General frame or digital interlaced video with or without accurate sync timing
 - Data type (packet or frame level) and virtual channel interleaving
- All primary and secondary data formats:
 - YUV420: 8-bit (legacy)/8-bit/10-bit/8-bit (CSPS)/10-bit (CSPS)
 - YUV422: 8-bit/10-bit
 - RGB: RGB888/RGB666/RGB565/RGB555/RGB444
 - RAW: RAW6/RAW7/RAW8/RAW10/RAW12/RAW14/RAW16
 - Generic 8-bit long packet data types
 - User-defined byte-based data
 - Embedded data (RAW16)
- Error detection and correction
 - PHY level
 - Packet level
 - Line level
 - Frame level

The CSI controller has the following interfaces:

- Image Pixel Interface (IPI) that provides:
 - 16-bit output parallel bus operating at pixel clock rate and delivering 1 pixel per clock cycle
 - Vertical and Horizontal timing accurate video synchronization signal
- APB slave bus for configuration purposes
- Interface with MIPI D-PHY RX following the PHY Protocol Interface (PPI), as defined in the MIPI Alliance Specification for D-PHY
- Optional support for Camera Control Interface (CCI) through the use of existing I2C master at system level

Figure 7-13 presents a simplified diagram of the MIPI CSI-2 implementation in the device.

Figure 7-13 MIPI CSI-2 Implementation⁽¹⁾



1. Contains Synopsys proprietary information. Used with permission.

The main functional blocks of the MIPI CSI-2 controller are as follows:

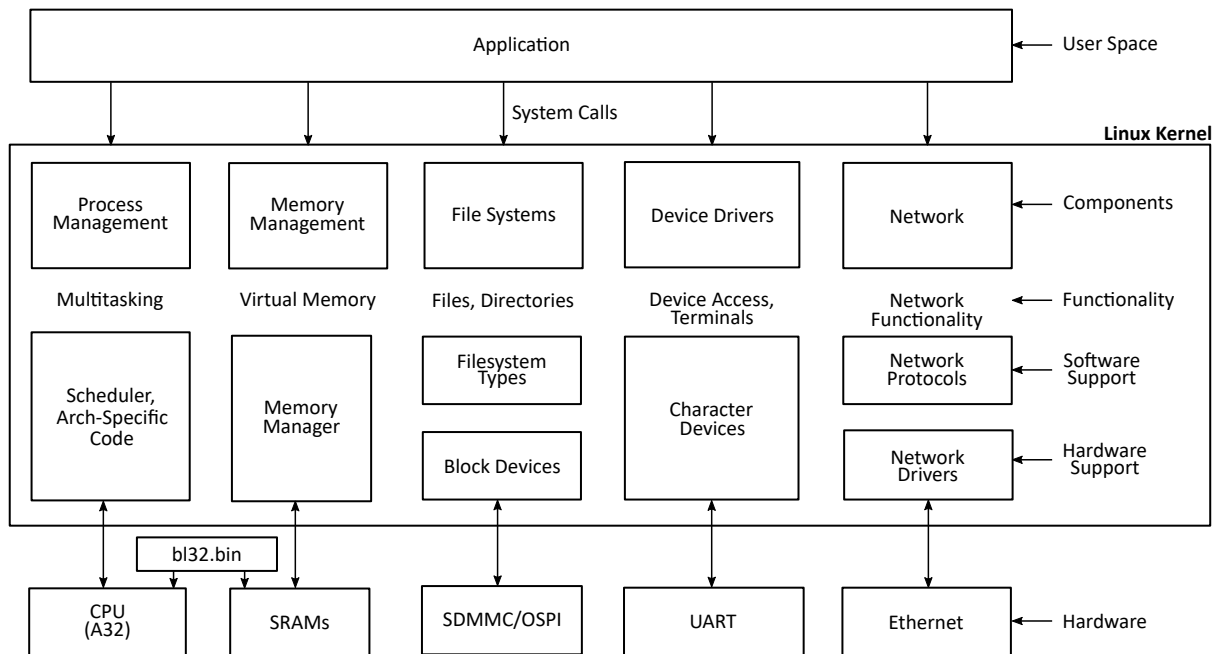
- **MIPI D-PHY RX:** Implements the physical link layer
- **Pipeline:** Registers the PPI high-speed related signals avoiding possible device timing constraints.
- **Descrambler:** If enabled, this block converts the PPI scrambled data to its original values by using the same Linear-Feedback Shift Register (LFSR) seeds used by the transmitter.
- **PHY adaptation layer:** It is responsible for managing the PHY interface, including PHY error handling.
- **Packet analyzer:** If required, data lane merging is implemented, together with header decoding, error detection and correction, frame size error detection, and CRC error detection.
- **Image Pixel Interface (IPI):** Conveys unpacked pixels and generates timing accurate video synchronization signals.
- **Error management:** Notifies and monitors the error conditions on the CSI-2 link.
- **Register bank:** Provides access to configuration and control registers.
- **Synchronizer:** This module contains all the synchronizers used to implement CDC functions.
- **Camera Control Interface (CCI):** I2C interface for camera configuration (optional).

8 Linux Software Development Kit (SDK) Setup

8.1 Linux SDK Architecture Overview

Figure 8-1 shows the architecture overview of the Linux Software Development Kit (SDK).

Figure 8-1 Linux SDK Architecture Overview



The *bl32.bin* is a secure stage 3-2 bootloader, which makes Linux kernel execute after performing the following tasks:

- A very early platform-specific architectural setups, such as the platform-specific memory map
- Ensures the MMU is initialized and enabled
- Initializes the GICv2 driver for interrupts
- Enables the system-level generic timer
- Configures the external OSPI NOR flash XIP mode

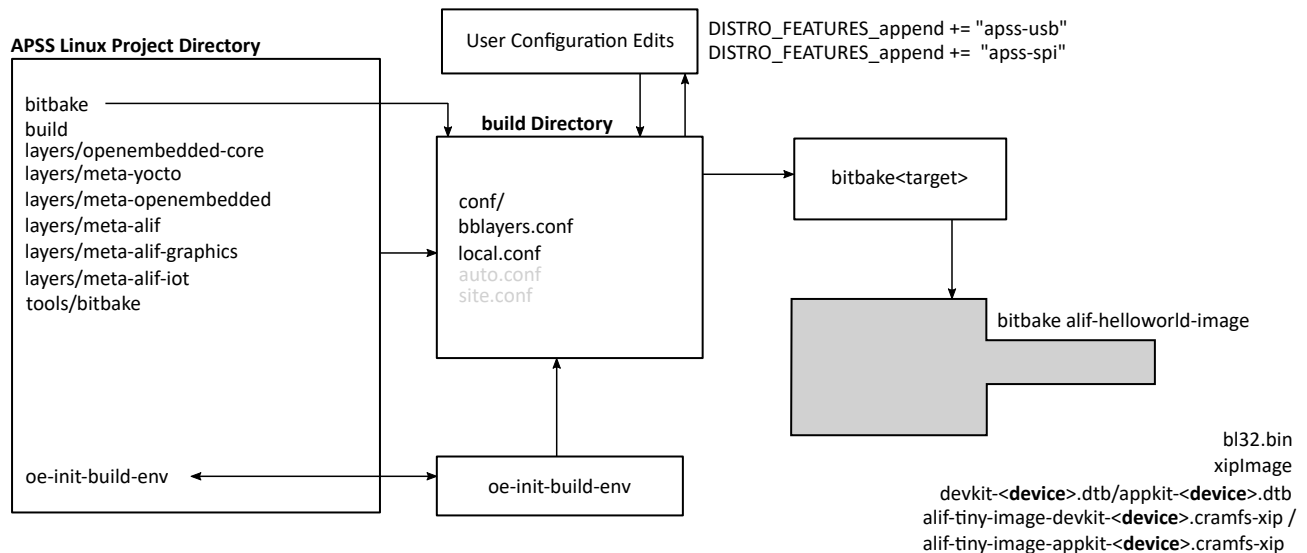
The Linux kernel is the main component of a Linux Operating System (OS) and serves as the core interface between device hardware and user-space applications. The requests from user-space applications are serviced by the Linux kernel and its drivers efficiently through system calls.

The scheduler and any architecture-specific code of the Linux kernel schedule user-space processes to execute on the Cortex-A32 CPU in the APSS. The memory manager of the Linux kernel sets up memory management by mapping the physical addresses of SRAMs (Static Random-Access Memory) to a larger virtual address, which allows multiple and large user-space applications to run seamlessly. Various Linux device drivers allow access and control over the functionality of the various peripheral devices, such as UART, Ethernet, and USB. The Linux filesystem is used to handle the data management of the storage. It helps arrange the files and directories on storage mediums, such as SD card and OSPI NOR flash. It manages the file name, file size, creation date, and various other information about a file. The Linux kernel mounts the filesystem, which is a structured and hierarchal collection of files, and which contains the root directory (/)

and its subdirectories. The user-space applications and the system libraries required to run the applications are also part of the filesystem.

Figure 8-2 shows the build flow of the APSS Linux project for creating images for the APSS.

Figure 8-2 APSS Linux Project Build Flow



The APSS Linux project uses the Yocto project, an open-source project focused on creating embedded Linux distributions to build images for the APSS in the DevKit/AppKit device. It uses the OpenEmbedded build system and the *BitBake* tool to create Linux images. OpenEmbedded is a build automation framework and cross-compilation environment used to create Linux distributions for embedded devices. *BitBake* is a generic task execution engine that allows shell and Python tasks to be run efficiently and in parallel, while working within complex inter-task dependency constraints.

The following are the output images for the APSS generated from the APSS Linux project:

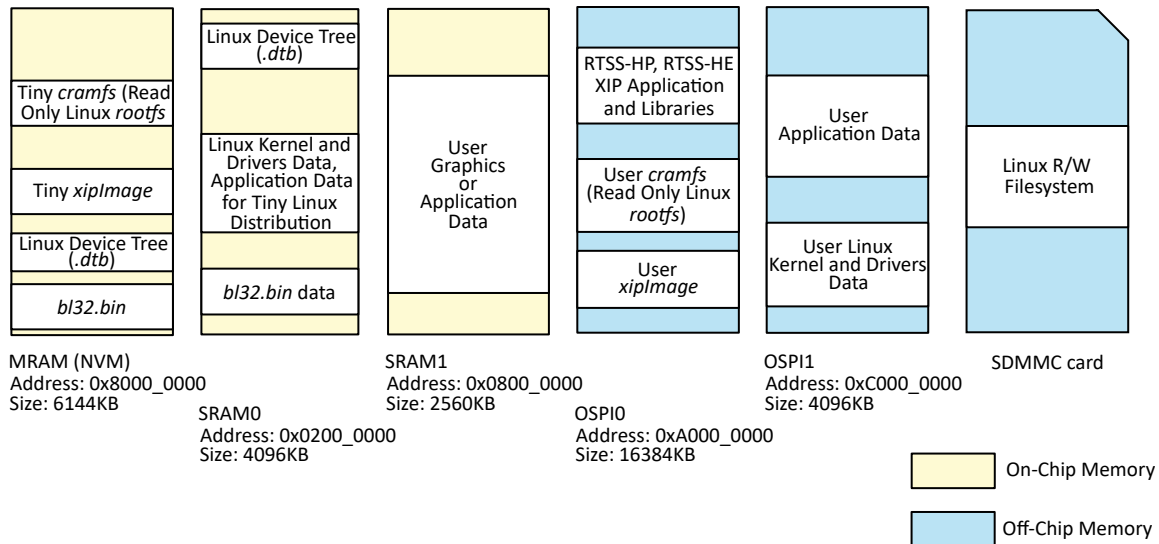
- **bl32.bin**: The secure stage 3-2 bootloader, gives control to execute Linux kernel.
- **devkit-<device>.dtb/appkit-<device>.dtb**: The Linux device tree used by the kernel, for example devkit-e7.dtb/appkit-e7.dtb.
- **xiplmage**: The Linux kernel image used in the APSS of the DevKit/AppKit device is an XIP image (*xiplmage*). The XIP is a method, where the Linux kernel and applications are allowed to run from non-volatile storage directly addressable by the CPU, such as NOR flash, instead of copying the code to RAM.
- **Filesystem**: A structured and hierarchal collection of files, which can be of the following formats:
 - **cramfs-xip**: The *cramfs* is a read-only Linux filesystem designed to be simple, small, and efficient in compression. The *cramfs-xip* filesystem can flag individual files/applications to be run as XIP directly from the storage medium, assuming the filesystem image was stored in linear accessible memory, such as NOR flash or ROM. This filesystem can be directly mounted from an NVM to boot a Tiny Linux Distribution or run custom user-space applications on the APSS.
 - **ext4**: The *ext4* is the default filesystem for many Linux distributions. A requirement for read-write (R/W) filesystem is fulfilled by *ext4*. An external memory device, such as SDMMC, is programmed with an *ext4* filesystem to boot a user-specific read/write Linux distribution.

For more details on the APSS Linux project, refer to User Guide *Getting Started with Linux* available on the Alif Semiconductor [User Guides & App Notes](#) page.

8.2 Memory Architecture Used by the APSS

Figure 8-3 gives an overview of the memory architecture used by the APSS.

Figure 8-3 APSS Memory Architecture Overview



The MRAM (a non-volatile memory) is used to store the APSS images for booting the APSS across multiple Power-on-Reset (POR). The stage 3-2 bootloader (*bl32.bin*) and Linux device tree (*devkit-<device>.dtb/appkit-<device>.dtb*) are stored and executed from the MRAM during the APSS boot up. The *xiplmage* and read only *rootfs* (in *cramfs-xip* format) can also be stored and executed directly from MRAM for use cases that do not require huge code and data spaces.

The data space required by the Linux kernel and its device drivers is allocated in on-chip SRAM0. The *devkit-<device>.dtb/appkit-<device>.dtb* is loaded from MRAM to SRAM0 before executing *xiplmage*. As the size of SRAM0 is limited to 4MB, only Tiny Linux or custom user-space applications, which require less data space, can be executed on SRAM0.

The SRAM1 is used as data space for user-specific graphics and user-space applications.

The OSPI0 NOR flash memory is used to store custom images, such as *xiplmage* and *rootfs* (*cramfs-xip*), that do not fit in MRAM. These images can be user-specific, with larger code sizes and greater data space requirements.

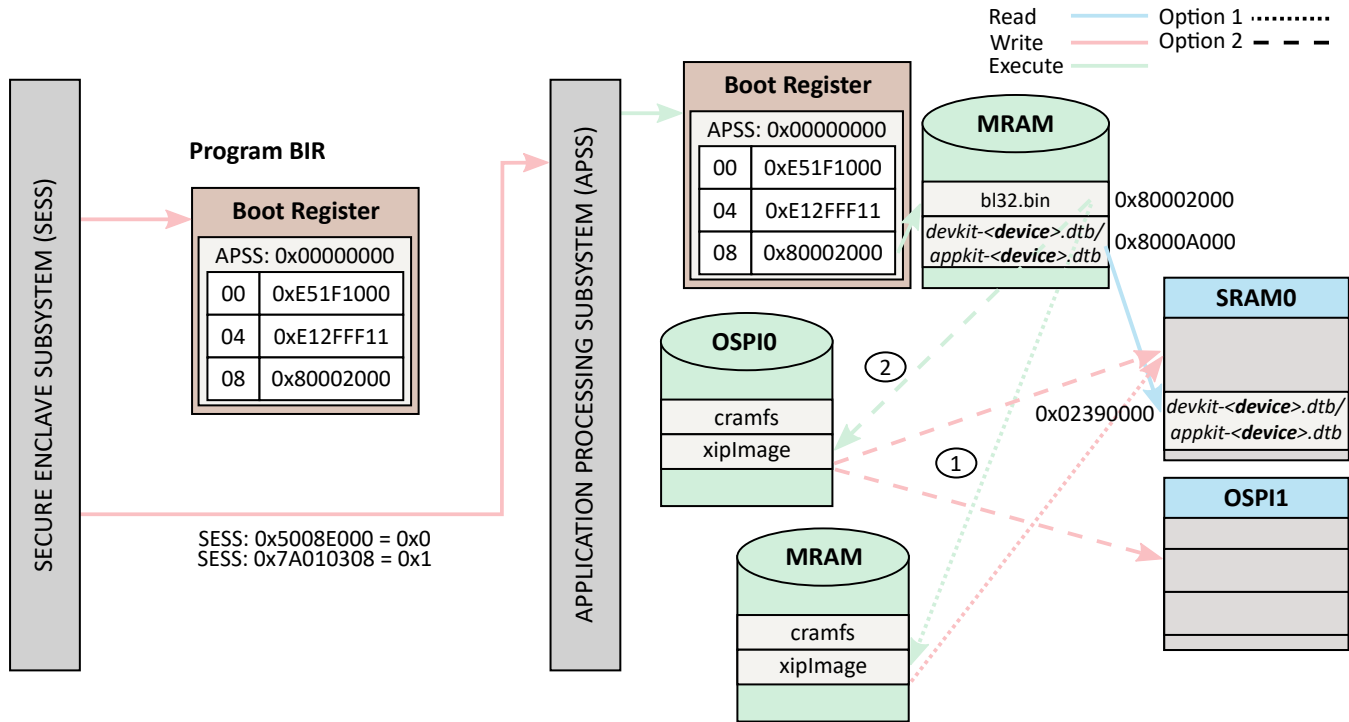
The OSPI1 HyperRAM is used as data space for *xiplmage* and user-specific applications that require more data space than the available SRAM0 and SRAM1.

If a use case needs a Linux read/write filesystem, such as *ext4*, then an SDMMC card can be used to boot user-specific Linux distribution.

8.3 APSS Boot Up

Figure 8-4 shows the sequence of events while booting the APSS from the SESS.

Figure 8-4 APSS Boot Up Events



The binaries required to boot the APSS are programmed into MRAM using the ATOC. For more information about the tools, refer to the *Alif Security Toolkit User Guide* available on the Alif Semiconductor [Software & Tools](#) page.

The MRAM must contain *bl32.bin* and *devkit-<device>.dtb/appkit-<device>.dtb*, while *xiplmage* and *cramfs-xip* are included optionally, depending on the availability of free space.

The SESS programs the Boot Instruction Register (BIR) of the APSS to start executing *bl32.bin* from the MRAM address 0x8000_2000 when the APSS is woken up. After programming BIR, SESS wakes up the APSS by writing 0x0 and 0x1 into Host System Reset Control Register (address from SESS perspective is 0x5008_E000) and Host CPU Wakeup Register (address from SESS perspective is 0x7A01_0308), respectively.

The APSS executes instructions from the BIR, jumps to the MRAM address 0x8000_2000, which contains *bl32.bin*, and executes *bl32.bin*. The *bl32.bin* then copies *devkit-<device>.dtb/appkit-<device>.dtb* from the MRAM address 0x8000_A000 to the SRAM0 region at 0x0239_0000 and gives control to execute *xiplmage* (Linux kernel) from the following memories based on the use cases:

- Option 1: Execute from MRAM to boot Tiny Core Linux (TCL).
- Option 2: Execute from OSPIO to boot user-specific Linux distribution.

The *xiplmage* in turn mounts any of the following *rootfs* based on the use cases:

- *cramfs-xip* from MRAM, if the filesystem is tiny.
- *cramfs-xip* from OSPIO, if the filesystem does not fit in MRAM.
- *ext4* from SDMMC card, if the filesystem needs to be R/W.

On-chip SRAM0 is used to allocate data space for the execution of tiny *xiplmage* and user-space applications. Otherwise, off-chip OSPIO1 HyperRAM is used to allocate data space for the execution of feature-specific/custom *xiplmage* and user-space applications.

The process of downloading and installing the pre-built images necessary to run the Alif Linux distribution on the Ensemble DevKit/AppKit using OSPI flash and OSPI HyperRAM is detailed in the User Guide *Getting Started with Linux Pre-Built Images* available on the Alif Semiconductor [User Guides & App Notes](#) page.

This document is also part of the *Linux Pre-Built Images Archive* for Ensemble DevKit/AppKit, available for download on the same Alif Semiconductor page.

Additionally, a *Hyperram_Linux* document is included with the download, providing further details on the external RAM connected to the Alif SoC via the OSPI0 interface on the DevKit/AppKit platform when the Linux OS is booted from the Cortex-A32 cores.

For building Linux images for the APSS, refer to the GitHub repository [alifsemi/alif_linux-apss-build-setup](#) and follow the instructions in the README file.

9 Alif Semiconductor RTSS SDK

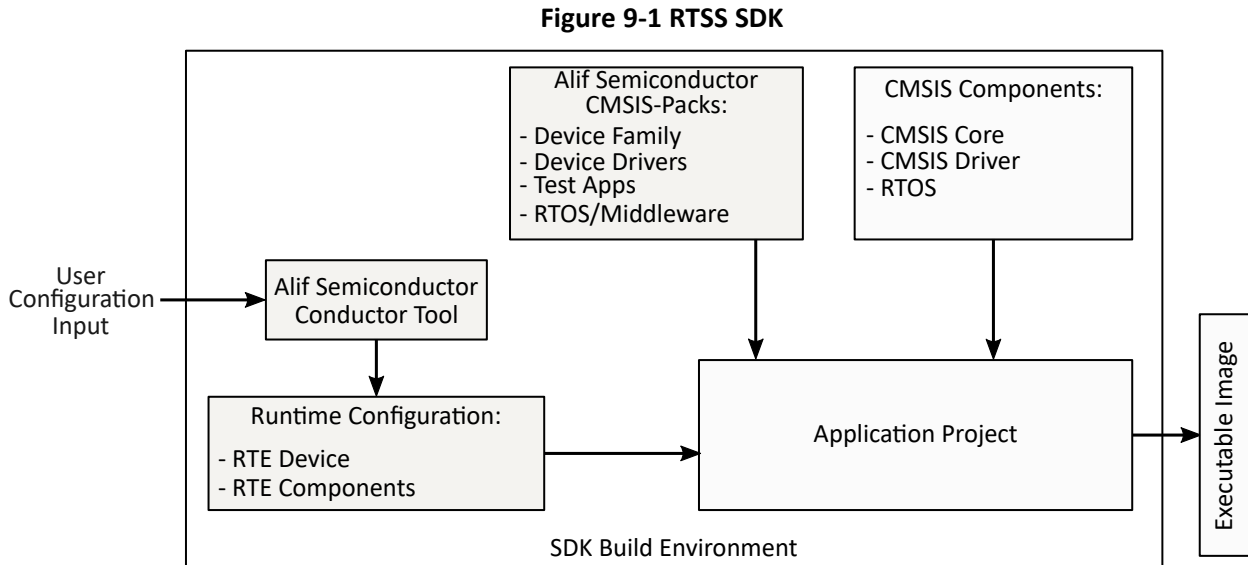
9.1 RTSS SDK Components

The RTSS SDK mainly contains generic CMSIS components and Alif Semiconductor provided CMSIS-Packs.

CMSIS-Packs provided by Alif Semiconductor are for:

- Supporting different device families and associated peripheral drivers
- Other software components / RTOS ports / Middleware

Figure 9-1 shows the different SDK components.



9.2 Generic CMSIS Components

Alif Semiconductor's RTSS SDK is built around CMSIS. It uses various CMSIS components, such as core, drivers, and RTOS.

9.2.1 RTE Device and RTE Components

RTE_Device.h and *RTE_Components.h* control the instantiation of peripheral instances and the drivers that need to be included in the application project. While the *RTE_Components* header file is auto-generated based on the selections made in the CMSIS project *rteconfig* file during project creation, the *RTE device* header file can be modified according to application needs in IDE configuration wizard.

9.2.2 RTOS Project Configuration Using Conductor Tool

Alif Semiconductor provides [Alif Conductor](#) tool to configure cores and different peripherals. The software shall input various application project configuration options to the Conductor tool interface and generate the configuration output. The output of this tool shall be directly utilized in the application project through the *RTE Device* or *RTE Components* header files.

9.2.3 RTOS Application Software Code/Libraries

These are additional code and libraries needed for the project. They are optional and should be supplied by the software.

9.2.4 RTOS Software Project

Combination of all of the above components results in software application project ready to be built.

9.2.5 RTOS Executable Image

After updating all the required application source code, libraries, RTE components, and user-defined settings, the project is built, and the *Executable Image* is created.

9.3 Using SDK to Create/Compile an RTOS Project in an IDE

For steps how to create and compile a project, refer to the respective IDE User Guide *Getting Started with Bare Metal Design* available on the Alif Semiconductor [User Guides & App Notes](#) page.

9.4 RTOS Memory Regions and Predefined Linker Script

The default linker script supplied in SDK (included in Alif Semiconductor CMSIS device Pack) configures the system to use ITCM (0x0000_0000) and DTCM (0x2000_0000) memories. These correspond to the macros `__ROM_BASE(ER)` and `__RAM_BASE(RW region)`.

All available SRAM regions are listed as empty regions.

`NS_REGION_0` region is defined to be used for non-secure TCM usage purposes.

9.5 RTOS System Startup

Secure bootloader copies core bootable images into configured boot memory (TCM is by default).

Then, it releases the RTSS cores from wait state. The core starts booting the image from the configured boot memory. By default, RTSS cores boot up in secure mode.

9.6 RTSS SDK Drivers

9.6.1 RTSS SDK Architecture

The Alif Semiconductor RTSS SDK follows CMSIS-Driver architecture to support its devices across multiple ecosystems.

For more detailed description of CMSIS-Driver framework, refer to [CMSIS-Driver specification](#).

As CMSIS-Driver is OS agnostic, it can be used along with different Bare Metal / RTOS environments.

9.6.2 CMSIS-Driver Overview

The CMSIS-Driver APIs can be divided into two parts: Common Driver functions and Data Transfer functions.

Common Driver functions include:

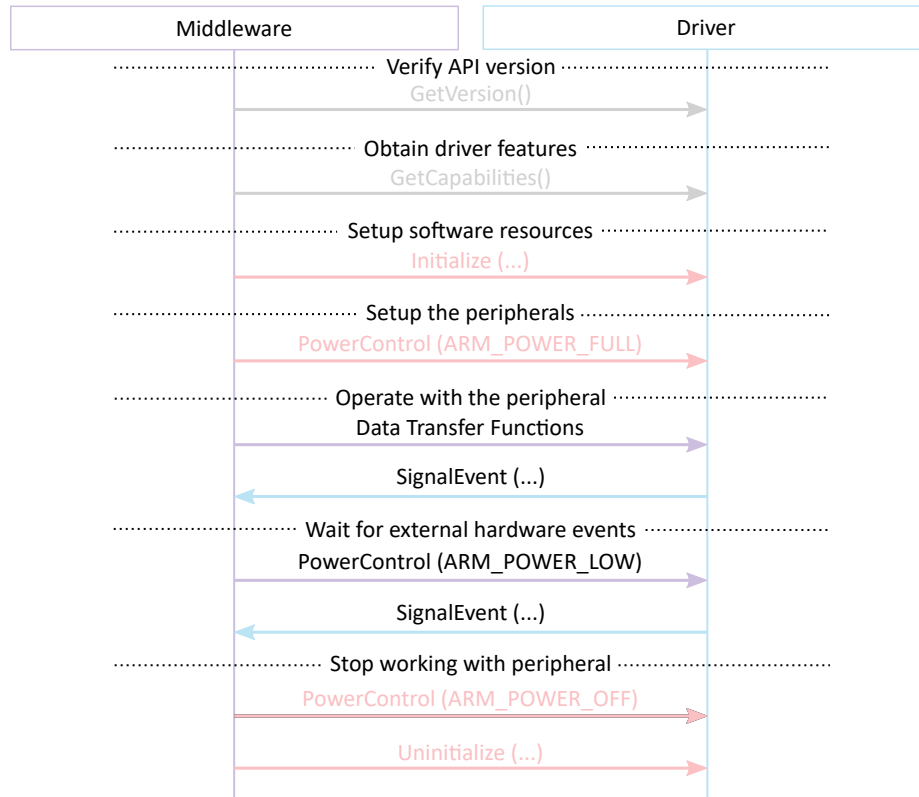
- *GetVersion*—Provides the version number
- *GetCapabilities*—Capabilities supported by the driver
- *Initialize*—Allocates the resources and initialize the device
- *SignalEvent*—*CallBack* function for notifying the operation status
- *PowerControl*—Controls the device power
- *Uninitialize*—De-allocates all the resources and de-initialize the device
- *Control*—Configures the device functionalities using control codes (for example, setting parameters, enabling or disabling a function)

Data Transfer functions include (depending on the device this list can be extended):

- *Send*—Writes data to device
- *Receive*—Reads data from the device
- *Transfer*—Reads and writes data

All Data Transfer functions are non-blocking calls. Once the operation is completed, the status should be updated to the application using the *SignalEvent* (see [Figure 9-2](#)).

Figure 9-2 CMSIS-Driver Call Sequence Flow



CMSIS-Driver APIs are built on top of the *Access Structs*. The *Access Struct* provides the API that should be invoked by the application layer to communicate to the devices. Each instance of devices should have separate *Access Struct*, for example Alif Semiconductor device has four physical I2C controllers and thus four different *Access Structs* to talk to I2C. Each driver should have its own *Access Struct* member APIs.

For more information about the theory of operation for CMSIS and function *Call Sequence* that has to be followed by application, refer to [CMSIS-Driver](#).

9.6.3 RTSS SDK Custom CMSIS Drivers

Alif Semiconductor RTSS SDK has developed CMSIS APIs for devices for which the framework is currently not available in the CMSIS-Drivers framework.

This can be found under `<Alif_CMSIS_Pack_Root>/<version_number>/CMSIS/Include`.

9.6.4 RTSS SDK Configuration

The CMSIS-Driver has many available configuration options defined in `<my project>/RTE/Device/<processor>/RTE_Device.h`.

Configurations such as IRQ priority, FIFO trigger level, and others are defined here for the driver to use. The developer is free to modify these based on the system/application requirements.

Additionally, the drivers should be enabled in `<myproject>.rteconfig`, and this change should be reflected in the `<myproject>/RTE/RTE_Components.h` file.

Alif Semiconductor's RTSS SDK also provides example projects and test applications for reference to streamline the application development process.

NOTE

The application developer should take care of the synchronization when using the driver from multiple thread contexts. CMSIS does not provide any synchronization mechanisms at the driver level.

10 Azure RTOS Software Development Kit

10.1 Azure RTOS SDK Components

For a description of generic CMSIS components, drivers and Alif Semiconductor provided CMSIS-Packs that support all RTSS RTOS designs, refer to [Section 9 Alif Semiconductor RTSS SDK](#).

For the Ensemble Azure RTOS support pack, refer to [alif_ensemble-Azure-RTOS](#).

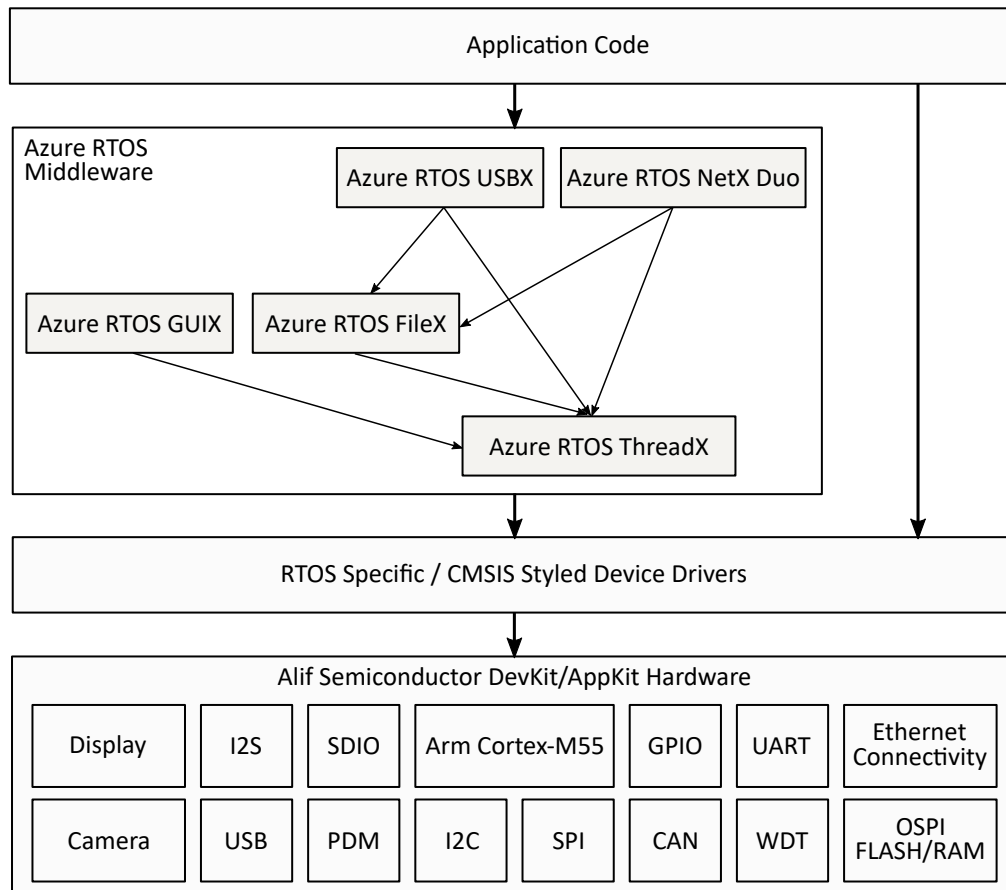
For Alif Semiconductor's Azure peripheral examples, refer to [Ensemble Azure Samples](#).

10.2 Azure RTOS Middleware Support

Azure RTOS is an embedded development suite consisting of ThreadX® as its core, along with extended functionalities provided by components such as GUIX, FileX, USBX, and NetX Duo.

[Figure 10-1](#) depicts Azure RTOS middleware components and their interaction with application and hardware.

Figure 10-1 Azure RTOS Components



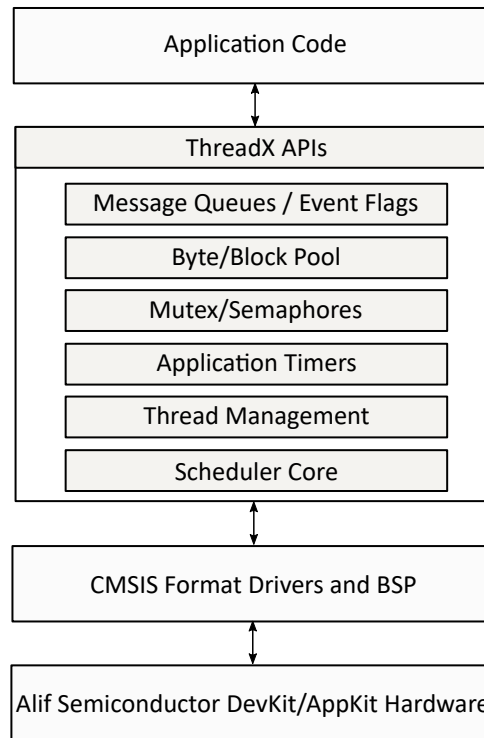
10.2.1 Azure RTOS ThreadX

Azure RTOS ThreadX is a high-performance RTOS kernel designed specifically for resource-constrained embedded systems and applications. Azure RTOS ThreadX has picokernel architecture.

Azure RTOS ThreadX provides services such as advanced preemption-threshold scheduling, message passing, communication, synchronization, timer, memory management, and interrupt management, event-chaining, execution profiling, performance metrics, and system event tracing, etc.

Figure 10-2 shows different services and APIs offered by Azure RTOS ThreadX and their interactions with different components in the system.

Figure 10-2 ThreadX Services



For more API details and usage, refer to the [Azure RTOS ThreadX User Guide](#).

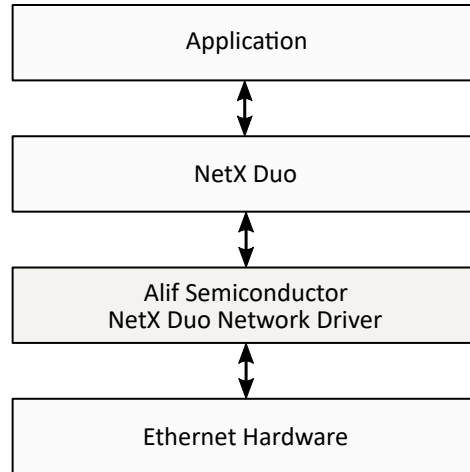
10.2.2 NetX Duo

Azure RTOS NetX Duo is an embedded TCP/IP stack specifically tailored for resource-constrained embedded real-time and IoT environments. Along with the core network protocols such as TCP, UDP, ICMP, and IP (version 4 and version 6), NetX Duo supports a large number of additional higher layer protocols (such as HTTP(S), MQTT, FTP, etc.) as add-ons. Additionally, NetX Duo provides a set of add-on protocols to enable secure communications. These implement Transport Layer Security protocols (SSL/TLS/DTLS) and IPsec.

10.2.2.1 NetX Duo Support in Alif Semiconductor RTSS SDK

Figure 10-3 is a high-level representation of different components in the network stack of the Alif Semiconductor RTSS SDK. Note that NetX Duo depends on the functionalities provided by Azure RTOS ThreadX and assumes its existence. For clarity, ThreadX is not shown in the figure.

Figure 10-3 NetX Duo



As shown in [Figure 10-3](#), NetX Duo depends on a network driver to perform various functions, including Ethernet hardware and physical link initialization, packet transmission, packet reception, and status and control of the Ethernet interface. The Alif Semiconductor's NetX Duo network driver interacts with the Ethernet MAC and PHY hardware components to service different requests generated by NetX Duo. Apart from handling various requests generated by NetX Duo, the network driver is also responsible for detection and handling asynchronous events that happen in the physical media. These include packet reception, packet transmission complete events and link status/mode changes, etc. The Alif Semiconductor network driver supports the following performance-centric features:

- *Zero-copy transmission and reception:* The driver avoids copying packet data between the buffers used by NetX Duo and the buffers used by the Ethernet DMA hardware. This saves CPU cycles and improves performance considerably.
- *Scatter/Gather packet buffers:* The driver is capable of programming the Ethernet DMA hardware to assemble packet payload from multiple discontinuous memory regions in both the TX and RX paths. This enables the application developers to choose and configure the packet sizes in the NetX Duo packet pools as per their needs, without worrying about the MTU of the physical media and the size of the received packets. Note that this feature depends on the NetX Duo packet chaining support.

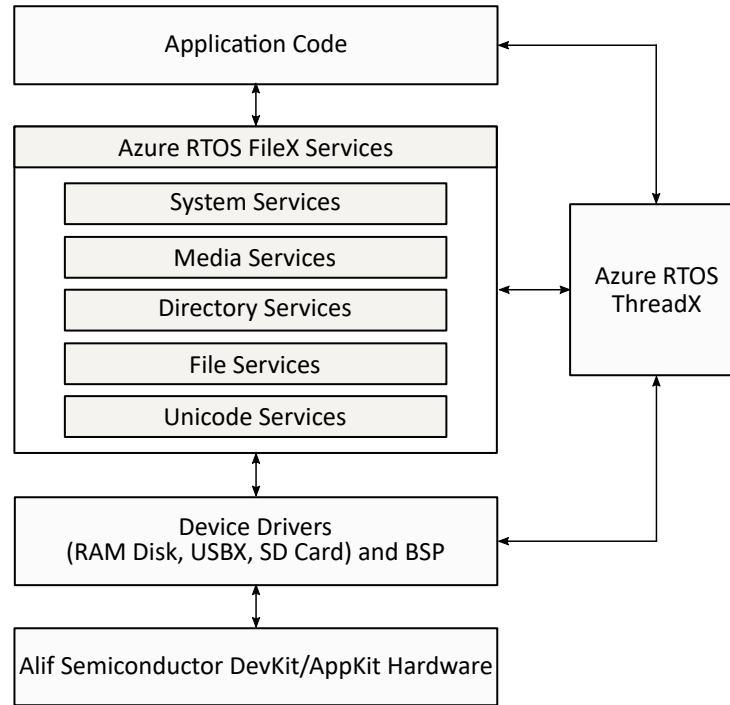
For more details on NetX Duo APIs and usage, refer to the [NetX Duo documentation](#).

10.2.3 FileX

Azure RTOS FileX is file management system for embedded application. Azure RTOS FileX supports most physical media, including RAM disk, USB, SD card. Azure RTOS FileX supports all of Microsoft's file formats, including FAT12, FAT16, FAT32, and exFAT.

Various services provided by FileX and interaction with other components are shown in [Figure 10-4](#).

Figure 10-4 FileX Services



For more information about FileX and its services, refer to the [FileX User Guide](#).

10.2.4 GUIX

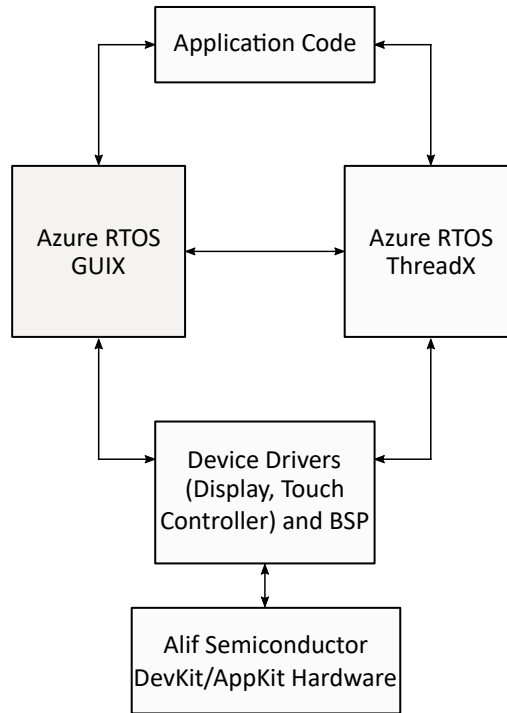
Azure RTOS GUIX is used for GUI implementation on embedded boards. GUIX is GUI software that can be used for real-time applications. GUIX is an event driven framework and works on events pushed into the GUIX event queue. These events are processed in GUIX thread (same as ThreadX thread).

Input drivers, such as touch screens, require additional GUIX threads. The touch input driver sends touch information into the GUIX thread by sending events into the GUIX event queue.

The framework does require hardware-specific input drivers and hardware-specific graphics drivers.

[Figure 10-5](#) shows the Azure RTOS GUIX interaction with different system components.

Figure 10-5 GUIX



10.2.4.1 Azure RTOS GUIX Studio

Azure RTOS GUIX Studio is UI development environment for the GUIX runtime library. It can be run on a Windows Host machine to design display screens for a project. With GUIX Studio, WYSIWYG screen can be designed for GUIX runtime environment. GUIX Studio generates C code that contains all the necessary UI information and code.

10.2.4.1.1 GUIX Studio Requirements

Minimum requirements for GUIX Studio:

- Windows XP (or above)
- 200MB of free RAM
- 2GB of available hard-disk space
- Display of 1024p × 768p with 256 colors

10.2.4.1.2 GUIX Studio Constraints

The GUIX Studio UI design tool has several constraints, as follows:

- A maximum of 4 displays per project
- A maximum of 100,000 widgets per GUIX Studio project
- A maximum of 100,000 distinct resources, for example colors, fonts, pixelpmaps, strings, etc.

For more details about GUIX studio, refer to the [GUIX User Guide](#).

10.2.5 USBX

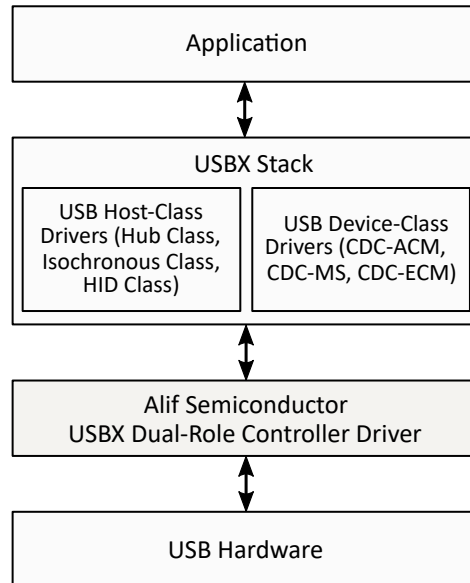
USBX is a high-performance USB stack designed for resource-constraint real-time embedded environment. USBX provides support for both Host and Device modes, including device-class support for mass storage, CDC-ACM, CDC-ECM, registration of different callbacks related to class defined in the application, and host

requests related to setup and configuration. USBX supports Open Host Controller Interface (OHCI) and Enhanced Host Controller Interface (EHCI).

10.2.5.1 USBX Stack Layout

USBX Stack comprises of three major components: Application, USBX Stack, and Dual-Role controller driver as shown in [Figure 10-6](#).

Figure 10-6 USBX



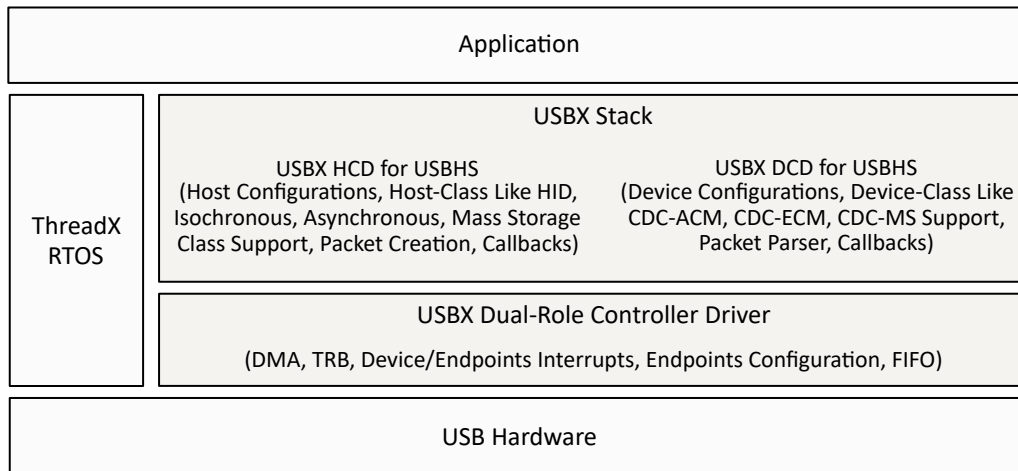
- **Application:** Initialization routines for memory, resource, and USB device-class are part of the application. Callbacks related to device class registers in application. Application utilizes device framework and pass it on USBX Stack. Device framework is divided in the following three categories:
 - **Descriptors:** Device, Configuration, Interface and Endpoint descriptors. These descriptors hold device-specific information, such as class, configurations, interfaces, and endpoints supported by the device.
 - **String:** String descriptor. This descriptor holds the manufacturer, product and serial number of the USB device.
 - **Language:** Language descriptor holds language supported by the device.
- **USBX Stack:** USBX Stack is responsible for USB Host and Device-class implementation, memory resourcing, stack initialization, callbacks, data transfer routines, descriptor parsing, buffer management and returning data transfer requests to controller driver. This layer interacts with both the application and the controller driver. The requests made by the USB Host, such as *GET_DESCRIPTOR*, *GET_CONFIGURATION*, *SET_CONFIGURATION*, and *SET_ADDRESS* for device information and configuration are parsed, the data is stored in a buffer, and finally passed by USBX Stack to driver layer for further processing, such as DMA transactions.
- **Dual-Role controller driver:** Driver handles the access to hardware registers, endpoint configurations, interrupt handling, DMA transfers, and FIFO maintenance. Setup/Data requests from the USB host to the slave device are handled by the driver. Depending upon type of interrupt (type of request), the driver takes decision and interacts with framework. In response, the framework provides data to driver, and the driver initiates DMA transactions. Following are few events and interrupts handled by the controller driver:

- *XferNotReady*
- *XferComplete*
- *XferDataStage*
- *XferStatusStage*.

10.2.5.2 USBX Support for Ensemble DevKit/AppKit

Figure 10-7 shows a simplified diagram of the USBX.

Figure 10-7 USBX Description



Alif Semiconductor DevKit/AppKit USBX provides support for:

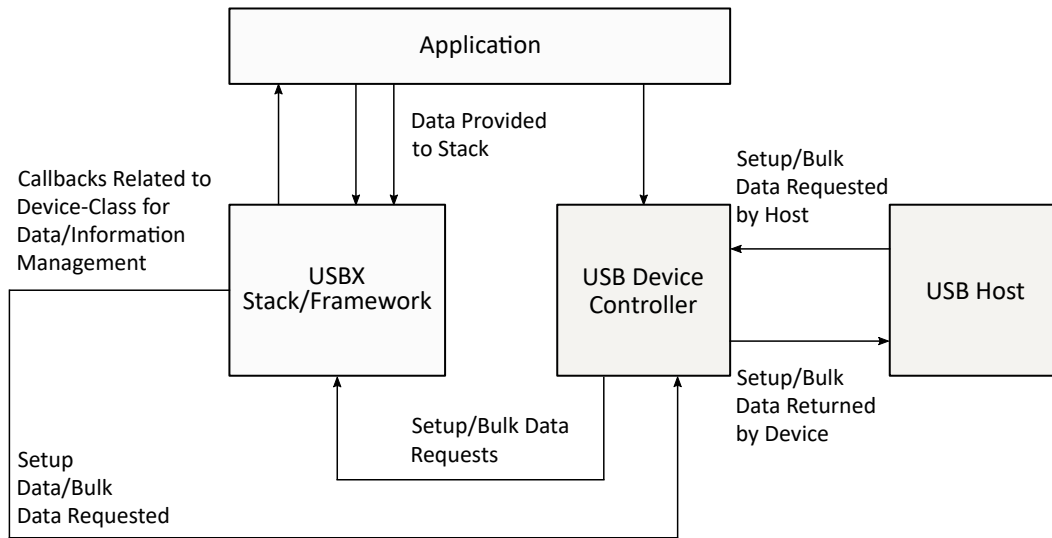
- CDC-ACM device class, which handles communication class requests on interface 0 using a Control/Interrupt endpoint and CDC-ACM data requests on interface 1 using bulk endpoint.
- CDC-ACM class initialization, memory resourcing, device framework parsing, and providing a data buffer the driver.
- TRB creation, DMA transaction initiation, Device and endpoint interrupt handling.

10.2.5.3 USBX Working in Device/Host Mode

In Device mode, USB class registration and stack initialization routines, followed by the device controller initialization routine, are called. The device controller is ready to accept interrupts (SETUP from Host) after configurations related to endpoints and device specific registers. Controller driver gets requests from Host and passes them to the USBX Stack. USBX Stack parses the information and returns back data to driver. Driver initiates DMA transaction on specific endpoint.

In Host mode, Setup packets are created by Host and sent on Control Endpoint 0 OUT/IN once the device controller is ready to accept Setup. Host continues with additional requests and enumerates the device if all the information is passed correctly (see [Figure 10-8](#)).

Figure 10-8 USBX Flow Diagram



10.3 Getting Started App Notes for Azure RTOS

For a complete step-by-step example of installing the needed development environment and using it to create an Alif Semiconductor Ensemble application and debug it, refer to the respective IDE User Guide *Getting Started with Bare Metal Design* available on the Alif Semiconductor [User Guides & App Notes](#) page.

11 FreeRTOS Software Development Kit

11.1 FreeRTOS SDK Components

For a description of generic CMSIS components, drivers, and Alif Semiconductor provided CMSIS-Packs that support all RTSS RTOS designs, refer to [Section 9 Alif Semiconductor RTSS SDK](#).

For the Ensemble FreeRTOS support pack, refer to [alif_ensemble-FreeRTOS-Components](#).

For simple Blinky example with FreeRTOS, refer to [GitHub alif_vscode-freertos-blinky](#).

11.2 FreeRTOS Middleware Support

The FreeRTOS development suite consists of FreeRTOS as the core kernel, with extended functionalities provided by various components such as FreeRTOS+FAT, FreeRTOS+TCP, and third-party middleware stacks for USB and graphics.

11.2.1 FreeRTOS

FreeRTOS is a real-time kernel (or real-time scheduler) on top of which embedded applications can be built to meet their hard real-time requirements. It allows applications to be organized as a collection of independent threads of execution.

FreeRTOS has the following standard features:

- Pre-emptive or co-operative operation
- Very flexible task priority assignment
- Flexible, fast, and lightweight task notification mechanism
- Queues
- Binary semaphores
- Counting semaphores
- Mutexes
- Recursive mutexes
- Software timers
- Event groups
- Tick hook functions
- Idle hook functions
- Stack overflow checking
- Trace recording
- Task run-time statistics gathering
- Optional commercial licensing and support
- Full interrupt nesting model (for some architectures)
- A tick-less capability for extremely low power applications
- Software managed interrupt stack when appropriate (this can help save RAM)

FreeRTOS is supplied as a set of C source files. Some of the source files are common to all ports, while others are specific to a port.

11.2.1.1 FreeRTOSConfig.h

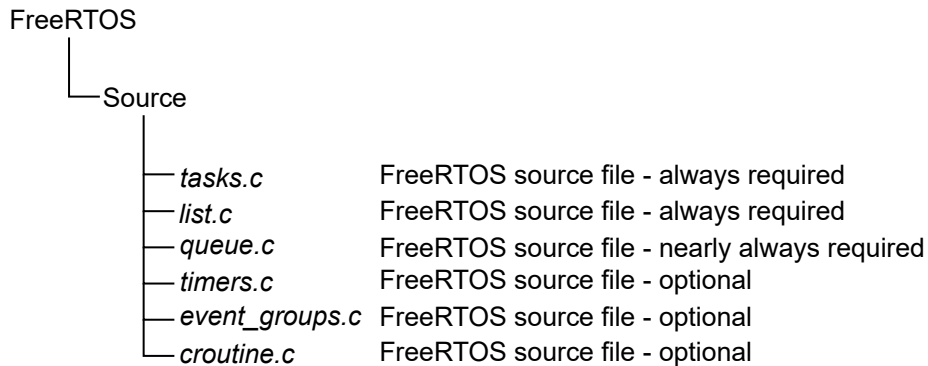
FreeRTOS is configured by a header file called *FreeRTOSConfig.h*.

FreeRTOSConfig.h is used to tailor FreeRTOS for use in a specific application.

11.2.1.2 FreeRTOS Common Source Files

Figure 11-1 shows the list of FreeRTOS port source files.

Figure 11-1 FreeRTOS Common Source Files



The following is a brief description of the FreeRTOS port source files:

- *task.c*: Provides task-related services
- *list.c*: Provides list implementation used by the scheduler
- *queue.c*: Provides both queue and semaphore services
- *timers.c*: Provides software timer functionality
- *event_groups.c*: Provides event group functionality
- *croutine.c*: Implements the FreeRTOS co-routine functionality (rarely used now)

11.2.1.3 FreeRTOS Port Specific Source Files

Source files specific to a FreeRTOS port are contained within the portable directory. The portable directory is arranged as a hierarchy, first by compiler, then by processor architecture.

FreeRTOS also considers heap memory allocation to be part of the portable layer. FreeRTOS provides five example heap allocation schemes. The five schemes are named *heap_1* to *heap_5* and are implemented by the source files *heap_1.c* to *heap_5.c* respectively.

11.2.2 FreeRTOS+TCP

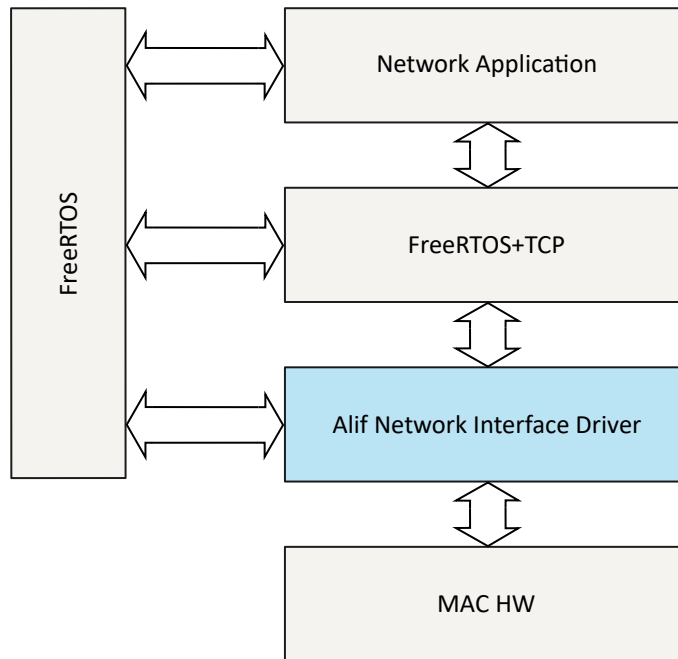
FreeRTOS+ TCP is an embedded TCP/IP networking stack that has been designed to work seamlessly on top of the FreeRTOS kernel. For application programs, FreeRTOS+TCP provides an API that is very similar to the popular Berkeley sockets API. This makes it easy to port applications that use the regular sockets API to the FreeRTOS+TCP environment.

Further information on FreeRTOS+TCP is available on [FreeRTOS+TCP](#).

11.2.2.1 Architecture

Figure 11-2 shows different components of a FreeRTOS+TCP network application in the Alif Semiconductor RTSS SDK.

Figure 11-2 FreeRTOS+TCP Components



Note that the Alif Semiconductor Network Interface driver depends on the generic Ethernet PHY handling code provided by FreeRTOS+TCP to manage the Ethernet PHY hardware.

11.2.2.2 Network Interface Layer

The Network Interface driver layer provides an interface between the FreeRTOS+TCP networking stack and the networking hardware. This layer implements the functions invoked by the stack to initialize and operate the Ethernet MAC hardware. This layer is responsible for programming the MAC hardware to send out the network packets generated by the networking stack and for routing the incoming Ethernet frames to the stack. The FreeRTOS+TCP stack provides different buffer allocation strategies (see [Section 11.2.2.3 Network Buffer Allocation/Management](#)). If the *BufferAllocation_1* strategy is used, the Network Interface driver layer is also responsible for statically allocating the memory needed to store network buffers.

The Alif Semiconductor Network Interface driver supports zero-copy transmission and reception, meaning that the network buffers are not copied to local buffers within the driver before being processed by the MAC DMA engine. This avoidance of data copy saves CPU cycles and thus increases performance.

11.2.2.3 Network Buffer Allocation/Management

The network buffers hold the packets that are received and are being sent out by the networking stack. The total number of buffers (and their descriptors) is configured by the application using a configuration file (see [Section 11.2.2.4 Setting Up and Configuring a FreeRTOS+TCP Application](#)) that is used by the stack. Additionally, the FreeRTOS+TCP stack provides two buffer allocation mechanisms to choose from. This gives the application developer more control over how the buffers are allocated.

If *BufferAllocation_1* strategy is chosen, the memory area for the network buffers is statically allocated by the Ethernet Network Interface driver. On the other hand, if *BufferAllocation_2* strategy is selected, the network buffers are allocated dynamically from the FreeRTOS heap area. Each scheme has its advantages and disadvantages. More information on buffer allocation strategies can be found on [TCP/IP Stack Network Buffers Allocation Schemes](#).

11.2.2.4 Setting Up and Configuring a FreeRTOS+TCP Application

FreeRTOS+TCP network applications are configured using a configuration header file named *FreeRTOSIPConfig.h*. This is similar to how a usual FreeRTOS application is configured using *FreeRTOSConfig.h* file. Various parameters defined in this file can be used to control the execution behavior of both the FreeRTOS+TCP stack and the Network Interface driver. A complete list and description of various parameters can be found on [FreeRTOS+TCP Configuration](#).

More information on FreeRTOS+TCP can be found on [FreeRTOS+TCP](#).

11.2.3 FreeRTOS+FAT

FreeRTOS+FAT is an open-source, fully thread-aware, and scalable FAT12/FAT16/FAT32 DOS/Windows compatible embedded FAT file system.

It provides optional long filename support, optional directory name hashing for speed, task-specific working directories and errno support, and comprehensive error reporting.

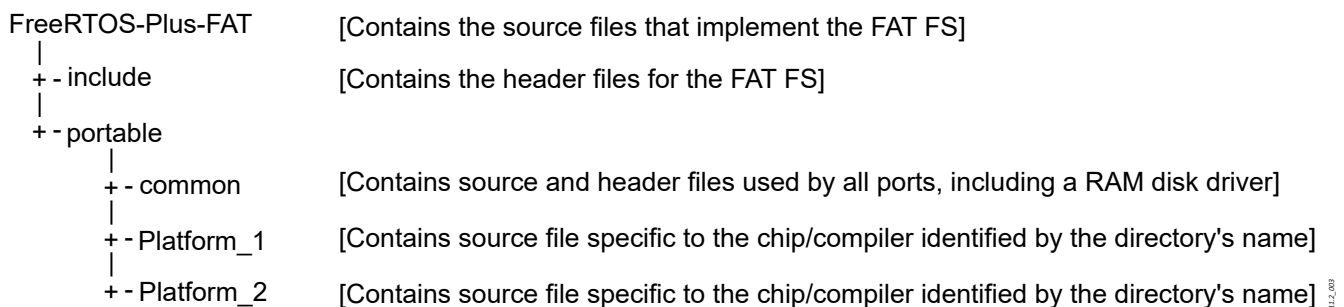
[Table 11-1](#) shows the list of filesystem APIs provided by FreeRTOS+FAT.

Table 11-1 FreeRTOS+FAT API Filesystems

Standard File System API		
ff_fopen()	ff_fread()	ff_fwrite()
ff_fgetc()	ff_fputc()	ff_truncate()
ff_fclose()	ff_rename()	ff_remove()
ff_fseek()	ff_rewind()	ff_tell()
ff_stat()	ff_feof()	ff_seteof()
ff_mkdir()	ff_chdir()	ff_getcwd()
ff_rmdir()	ff_fprintf()	ff_findfirst()
ff_findnext()	ff_filelength()	

Source code organization for FreeRTOS+FAT is shown in [Figure 11-3](#).

Figure 11-3 FreeRTOS+FAT Code Organization



11.2.3.1 FreeRTOS+FAT Configuration

Applications using FreeRTOS+FAT must provide a *FreeRTOSFATConfig.h* header file that describes the parameters to configure the FreeRTOS+FAT.

11.2.4 Graphics Middleware Support

Various third-party graphics libraries can be utilized along with FreeRTOS to provide graphics support.

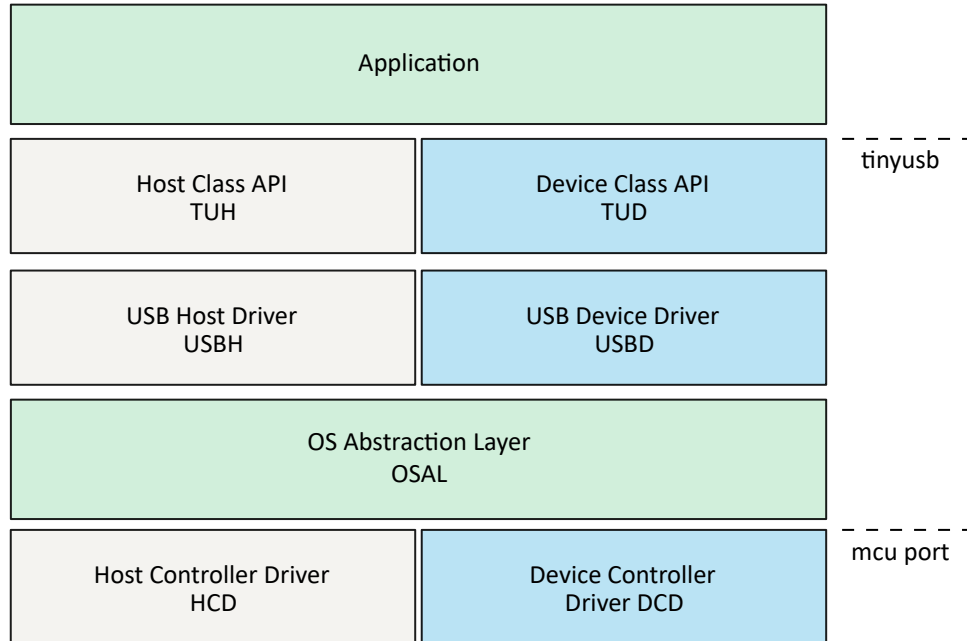
11.2.5 USB Middleware Support

Various third-party USB libraries can be utilized along with FreeRTOS to provide USB stack support. Alif intends to use TinyUSB as a solution for the FreeRTOS USB stack.

11.2.5.1 TinyUSB

TinyUSB is an open-source cross-platform USB Host and Device stack for embedded systems.

Figure 11-4 TinyUSB Block Diagram



11.2.5.2 Application

TinyUSB initialization routines are part of the TinyUSB stack. TinyUSB specific callback routines are called through the applications responsible for stack and controller Initializations. The application utilizes a device framework and passes it on to the TinyUSB stack. The device framework is divided into three categories:

- **Descriptors:** Device, Configuration, Interface, and Endpoint descriptors. These descriptors hold device-specific information, such as class, configurations, interface, and endpoints supported by the device.
- **String:** String descriptor. This descriptor holds the manufacturer, product, and serial number of the USB device.
- **Language:** Language descriptor holds the language supported by the device.

11.2.5.3 TinyUSB Stack

TinyUSB stack is responsible for USB Host and Device class implementation, memory resourcing, callbacks, stack initialization, initialization of synchronization resources, data transfer routines, buffer management, and queue management. The layer interacts with the Application and with the hardware (MCU) through the OS Abstraction Layer (OSAL).

The requests made by USB Host, such as GET_DESCRIPTOR, GET_CONFIGURATION, SET_CONFIGURATION, SET_ADDRESS for Device are queued in the TinyUSB stack and made available to the Controller once there is an interrupt for device SETUP. The data to USB Host is made available by TinyUSB stack through data transfer callbacks and other routines.

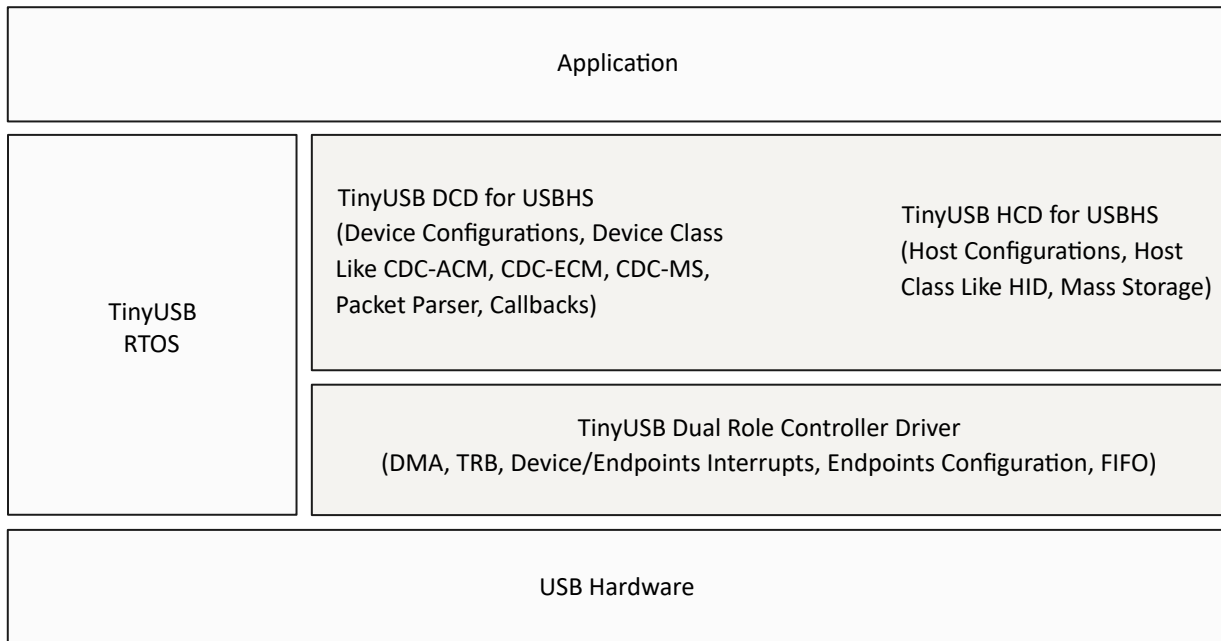
OS Abstraction Layer: TinyUSB is completely thread-safe by pushing all Interrupt Service Request (ISR) events into a central queue, then processing them later in the non-ISR context task function. It also uses semaphore/mutex to access shared resources, such as CDC FIFO. Therefore the stack needs to use some of the OS's basic APIs. The device SETUP-related requests are queued by OSAL and on-demand, data is passed by controller through the respective FIFOs. The OSAL is also responsible for providing basic data structures for TinyUSB, which may support concurrency when used with an RTOS. Without an RTOS, it simply handles concurrency issues between the main codes and interrupts.

Dual Role Controller Driver: Driver handles access to hardware registers, endpoint configurations, interrupt handling, DMA transfer, and FIFO maintenance. SETUP/Data requests by USB Host to slave device are handled by the controller. Depending on the type of interrupt (that is, the type of requests made by USB Host) driver makes decisions and interacts with the framework. The return framework calls routines added to the OSAL queue and returns data to the Host through DMA transactions. A few of the events handled by the controller are *XferComplete*, *XferDataStage*, *XferInProgress*, and *XferStatsStage*.

11.2.5.4 TinyUSB Support for Alif Semiconductor Ensemble DevKit/AppKit

Figure 11-5 shows a simplified diagram of the USBX.

Figure 11-5 Simplified Diagram of TinyUSB



11.2_005

Alif Semiconductor DevKit/AppKit TinyUSB supports:

- CDC-ACM device class support that handles class requests on interface 0 using a Control/Interrupt endpoint and CDC-ACM data requests on interface 1 using a bulk endpoint.
- CDC-ACM class initialization, memory resourcing, device framework parsing, and provisioning of data buffers to the driver.
- TRB creation, DMA transaction initiation, Device, and Endpoint interrupt handling.

11.2.5.5 TinyUSB Working in Device/Host Mode

In Device mode, stack initialization routines are called from the application, followed by device controller initialization routines. The device controller is ready to accept interrupts (Setup from the Host) after

configuring the endpoints and device-specific registers. The controller driver gets requests from the Host, calls the queued callbacks in OSAL, and returns data to the Host on specific endpoints through DMA. In Host mode, Setup packets are created by the Host and sent on Endpoint 0 OUT/IN once the device controller is configured and ready to accept Setup packets. The Host continues with additional requests and enumerates the device if all information is passed correctly.

11.2.5.6 TinyUSB Device Stack

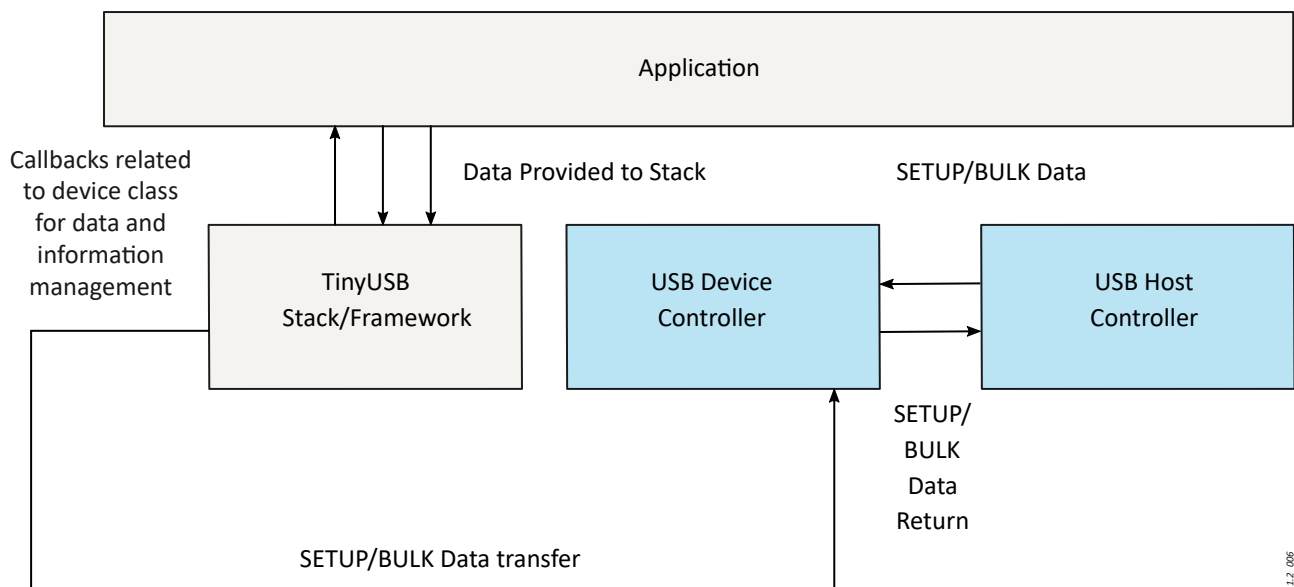
TinyUSB Device stack supports multiple device configurations by dynamically changing USB descriptors and different low power functions, such as suspend, resume, and remote wakeup. Following device classes are supported:

- USB Audio Class 2.0 (UAC2)
- Bluetooth Host Controller Interface (BTH HCI)
- Communication Device Class (CDC)
- Device Firmware Update (DFU): only Runtime
- Human Interface Device (HID): Generic (In and Out), Keyboard, Mouse, Gamepad
- Mass Storage Class (MSC): with multiple LUNs
- Musical Instrument Digital Interface (MIDI)
- Network with RNDIS, CDC-ECM (work in progress)
- USB Test and Measurement Class (USBTMC)
- Vendor-specific class support with generic In and Out endpoints. Can be used with MS OS 2.0 compatible descriptor to load winUSB driver without INF file.
- WebUSB with vendor-specific class

TinyUSB Host Stack Supports:

- Human Interface Device (HID): Keyboard, Mouse, Generic
- Mass Storage Class (MSC)
- Hub currently only supports 1 level of hub

Figure 11-6 TinyUSB Device Stack



11.2_006

12 Zephyr Alif SDK (ZAS)

12.1 Zephyr Introduction

Zephyr is a small RTOS for connected, resource-constrained and embedded devices. See more on the [Zephyr's website](#).

Features:

- A small monolithic kernel
- Highly configurable / Modular for flexibility
- Cross architecture support
- Devicetree support
- Optimized device driver model
- A set of protocol stacks such as IPv4 and IPv6, 802.15.4 (LR-WPAN), Bluetooth Low Energy (BLE), CAN-FD, MQTT.
- Virtual File System Interface with LittleFS and FATFS Support

Zephyr uses a meta-tool called West to execute Kconfig, CMake, and build system commands (Make or Ninja). CMake builds applications with the Zephyr kernel in two stages:

1. Configuration stage: CMakeLists.txt build scripts are executed to generate host-native build scripts.
2. Build stage: The generated build scripts are executed to compile the application.

12.2 Overview of Zephyr Alif SDK

The Zephyr Alif SDK (ZAS) is a comprehensive suite of tools that empowers software developers. Designed to elevate professionals' development skills, this software development kit (SDK) enhances proficiency and facilitates the creation of high-quality software. With an intuitive design and a wide array of tools, ZAS becomes a powerful asset for any software development project. By utilizing this SDK, developers can create efficient and top-quality software.

12.3 ZAS Repositories

12.3.1 Zephyr

The official Zephyr repository serves as the core development hub for the Zephyr real-time operating system (RTOS). These components are publicly available and open source, allowing developers from the community to contribute and collaborate. The [zephyr_alif](#) repository is the Alif Semiconductor port of the public Zephyr project repository.

12.3.2 HAL

The [hal_alif](#) repository is Alif Semiconductor's HAL internal integration layer for Zephyr.

Here, local integration of Alif drivers and feature changes occurs before they are officially incorporated into the open-source Zephyr.

12.3.3 sdk-alif

The [sdk-alif](#) repository includes Alif-specific materials that are not intended to be shared with the upstream Zephyr project. It is also the starting point for ZAS. Here's what you'll find in this section:

- *Sample Applications and Tests*

Alif provides the latest and updated sample applications and tests specific to Alif's hardware and features. Additional custom tests can also be found here.

- **Improved APIs and Drivers**

If there are drivers with improved APIs that are not yet included in the appropriate Zephyr OS, you will find them in this section. These APIs enhance the functionality of Alif's hardware components.

- **Middleware Protocols and Libraries**

Alif's middleware protocols, configurations, and libraries are available here. These resources facilitate communication and interaction with Alif's hardware.

- **New Protocols and Example Applications** Explore new protocols, such as CEVA BLE, Matter, Zigbee, or any other protocols not currently part of Zephyr in this section. You can find example applications that demonstrate how to use these protocols effectively.

- **SDK Documentation, Binaries, and Libraries** Access comprehensive documentation for the ZAS SDK here. You can find the necessary binaries and libraries required for developing with Alif's SDK:

- [AlifSemiFiles.zip](#) that contains the following directories:

- **Config files:**

1. Host setup script to install required packages to compile Zephyr on Linux host machine.
2. JSON files to create an Application Table Of Contents (ATOC) file containing Zephyr application, which gets programmed into MRAM using SETOOLS.

- **Prebuilt images:**

1. Zephyr binaries (zephyr_e7_rtssh*.bin) of sample applications such as “helloworld”, which display boot up messages on the serial console of the UART4B for the RTSS-HE and UART2A for the RTSS-HP.
2. A RTSS-HE binary (PC_Tool_HE.bin), that reads binary data from the host PC tool through UART2A and programs into OSPI1 NOR flash.

- **Utilities:**

1. Alif Security toolkit that supports all Ensemble and Balletto devices.
 2. Host PC tool to select and send Zephyr binaries from host PC for programming into OSPI1 NOR flash.
 3. bayer2rgb binary file – Command line utility to convert Bayer grid data to RGB data.
 4. video_camera_on_display_using_gpl_code.patch – Software patch to display camera capture on to LCD panel.
- [release_notes.pdf](#) – Release Notes listing contents, bug fixes, known issues, etc.
 - [user_guide.pdf](#) – Getting started with Zephyr for Alif Semiconductor devices and kits.
 - [bare_metal_src_for_ospi1_boot_v2.tar.xz](#) – Bare-metal code, which configures OSPI1 NOR flash into XIP mode.

12.4 mcuboot_alif

[mcuboot_alif](#) repository contains an Alif port of the MCUboot bootloader. See more at <https://docs.mcuboot.com/>

12.5 cmsis_alif

[cmsis_alif](#) is a fork from <https://github.com/zephyrproject-rtos/cmsis> for Alif Semiconductor specific changes. It contains parts of the ARM::CMSIS software pack.

13 Legal and Support Information

13.1 Disclaimers

Portions Copyright © 2021 Synopsys, Inc. Used with permission. All rights reserved. Synopsys & DesignWare are registered trademarks of Synopsys, Inc.

Legal Notice – Please Read

Alif Semiconductor™ reserves the right, without notice, to alter, edit, update, make corrections, and improvements to Alif documentation and products at any time. It is the responsibility of customers to maintain the most current versions of documentation before making any purchases from Alif. The information found in this documentation is provided to purchasers solely for the purpose of enabling hardware and software implementation of Alif products.

Alif neither takes any responsibility for, nor guarantees the appropriateness of, its products for a specific purpose. Customers accept the responsibility for the selection and incorporation of Alif products into their systems and Alif has no liability with respect thereto. ALIF ALSO DISCLAIMS ANY AND ALL LIABILITY WITH RESPECT THERETO, INCLUDING WITHOUT LIMITATION, DIRECT, CONSEQUENTIAL, INDIRECT, SPECIAL AND INCIDENTAL DAMAGES. ADDITIONALLY, ALIF DISCLAIMS AND EXCLUDES ALL WARRANTIES, WHETHER STATUTORY, EXPRESS OR IMPLIED, INCLUDING ANY IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT AND THOSE ARISING FROM COURSE OF DEALING AND USAGE OF TRADE.

Alif Semiconductor and the Alif logo are trademarks of Alif. For more information about our trademarks please visit our website at <https://alifsemi.com/legal/>. The omission of any Alif trademark, product name or any other name from this list does not constitute a waiver of Alif's intellectual property rights.

The recipient of this document does not have permission to copy, reprint, reproduce, duplicate, share, in any form, in whole or in part, unless prior written consent from Alif is obtained.

Please contact an Alif representative at contact@alifsemi.com if you have any questions regarding the information in this document.

Alif sells products according to standards terms and conditions of sales, which can be found at: <https://alifsemi.com/legal/>

13.2 Related Documents and Tools

- Alif Semiconductor device series-specific Hardware Reference Manual (HWRM)
- Alif Semiconductor device series-specific Datasheet

For additional Alif Semiconductor technical documentation and software resources please visit:

- [User Guides & App Notes](#)
- [Software & Tools](#)

For a list of all Alif Semiconductor's repositories, refer to [Alif Semiconductor GitHub](#).

For a full list of Ensemble embedded software packages, refer to [Ensemble SDK](#).

13.3 Contact Information

For more information visit our website [Alif Semiconductor](#) or contact us:

contact@alifsemi.com

US HQ – Silicon Valley, CA

7901 Stoneridge Drive, Suite 300
Pleasanton, CA 94588

13.4 Trademarks

The Alif logo, Ensemble, and *aiPM* are trademarks of Alif Semiconductor. For additional information about Alif Semiconductor trademarks, refer to <https://alifsemi.com/legal/>.

Arm, Cortex, CoreSight, TrustZone, AMBA, Thumb, and Ethos are registered trademarks or trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

Neon, and Helium are trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

Secure Digital and SD is a registered trademark of SD Card Association.

Synopsys & DesignWare are registered trademarks of Synopsys, Inc.

eMMC is trademarks of MultiMediaCard Association.

All other product or service names are the property of their respective owners.

13.5 Abbreviations

3

3GPP

3rd Generation Partnership Project

A

ACP

Accelerator Coherency Port

ADC

Analog to Digital Converter

AES

Advanced Encryption Standard

AHI

Application Host Interface

AI

Artificial Intelligence

aiPM

autonomous intelligent Power Management

AON

Always On

API

Application Programming Interface

APSS

Application Processor Subsystem

ATOC

Application Table of Content

B

BER

Bit-Error Ratio

BLE

Bluetooth Low Energy

BLIT

Block Image Transfers

BOD

Brown-Out Detect

BOR

Brown-Out Reset

BPU

Breakpoint Unit

C

CAN

Controller Area Network

CANFD

Controller Area Network with Flexible Data rate

CCC

Common Command Code

CDC

Configurable DPI Controller

CDM

Charged Device Model

CLUT

Color Look-Up Table

CMP

Comparator

CMSIS

Common Microcontroller Software Interface Standard

CNN

Convolutional Neural Network

CPI

Camera Parallel Interface

CPU

Central Processing Unit

CRC

Cyclic Redundancy Check

CSI

Camera Serial Interface

D

DAC

Digital to Analog Converter

DAP

Debug Access Port

DDR

Double Data Rate

DL

Display List / Download

DLR

Display List Reader

DM

Device Manufacturer

DMA

Direct Memory Access

DMAC

Direct Memory Access Controller

DMIC

Digital Microphone

DPA

Differential Power Analysis

DPI

Display Parallel Interface / Display Pixel Interface

DPU

Data Processing Unit

DRD

Dual-Role Device

DSC

Display Stream Compression

DSI

Display Serial Interface

DTCM

Data Tightly-Coupled Memory

DWT

Data Watchpoint and Trace

E

ECB

Electronic Codebook

ECC

Error-Correcting Code / Elliptic-Curve Cryptography

eDRX

Extended Discontinuous Reception

EMC

Electromagnetic Compatibility

EMFI

Electromagnetic Fault Injection

EMI

Electromagnetic Interference

EMS

Electromagnetic Susceptibility

EOL

End of Life

EoTp

End of Transmission Packet

EPU

Extension Processing Unit

ESD

Electrostatic Discharge

eSIM

Embedded Subscriber Identity Module

ETH

Ethernet

EVTRTR

Event Router

EWIC

External Wakeup Interrupt Controller

F

FC

Firewall Component

FIR

Finite Impulse Response

FSR

Full-Scale Range

G

GIC

Generic Interrupt Controller

GNSS

Global Navigation Satellite Subsystem

GOPS

Giga Operations Per Second

GPIO

General-Purpose Input/Output

GPS

Global Positioning System

GPU

Graphics Processing Unit

H

HBM

Human Body Model

HCI

Host Communication Interface

HDR

High Data Rate

HE

High Efficiency

HFRC

High-Frequency Resistor-Capacitor

HFXO

High-Frequency Crystal Oscillator

HMI

Human Machine Interface

HP

High Performance

HPP

High Performance Point

HUK

Hardware Unique Key

HWRM

Hardware Reference Manual

HWSEM

Hardware Semaphore

I

I2C

Inter-Integrated Circuit

I2S

Inter-IC Sound

I3C

Improved Inter-Integrated Circuit

IBI

In-Band Interrupt

ICMP

Internet Control Message Protocol

ICV

Integration Circuit Vendor

IDE

Integrated Design Environment

IFG

Interframe Gap

IFU

Instruction Fetch Unit

IIR

Infinite Impulse Response

IMD

Intermodulation Distortion

IoT*Internet of Things***IPC***Inter-Process Communication***IPI***Image Pixel Interface***IRQRTR***Interrupt Router***iSIM***Integrated Subscriber Identity Module***ISP***In-System Programming***ITCM***Instruction Tightly-Coupled Memory***iUICC***integrated Universal Integrated Circuit Card***IWIC***Internal Wakeup Interrupt Controller*

L

LCD*Liquid Crystal Display***LCS***Life Cycle State***LDE***Lockdown Extension***LDO***Low Drop-Out***LE***Low Energy***LFRC***Low-Frequency Resistor-Capacitor*

LFXO

Low-Frequency Crystal Oscillator

LOM

Listen Only Mode

LP

Low-Power

LPCMP

Low-Power Comparator

LPGPIO

Low-Power General-Purpose Input/Output

LPI2C

Low-Power Inter-Integrated Circuit

LPI2S

Low-Power Inter-IC Sound

LPM

Link Power Management

LPP

Low Performance Point

LPPDM

Low-Power Pulse Density Modulation

LPRTC

Low-Power Real-Time Counter

LPSPi

Low-Power Serial Peripheral Interface

LPTIMER

Low-Power Timer

LSB

Least Significant Bit

LTE

Long-Term Evolution

LUT

Look-Up Table

M

MAC

Media Access Controller

MAU

Memory Authentication Unit

MBI

Master Bus Interface

MCU

Microcontroller Unit

ME

Monitor Extension

MHU

Message Handling Unit

ML

Machine Learning

MMU

Memory Management Unit

MPE

Master Permission Entry

MPU

Memory Protection Unit

MRAM

Magnetoresistive Random-Access Memory

MSL

Moisture Sensitivity Level

MTL

MAC Transaction Layer

MVE

M-profile Vector Extension

MWS

Mobile Wireless Standard

N

NMI

Non-Maskable Interrupt

NPP

Nominal Performance Point

NPU

Neural Processing Unit

NPU-HE

Neural Processing Unit-High Efficiency

NPU-HP

Neural Processing Unit-High Performance

NRZ

Non-Return-to-Zero

NS

Non-Secure

NVIC

Nested Vectored Interrupt Controller

NVM

Non-Volatile Memory

O

OCS

OEM-signed Configuration Settings

OEM

Original Equipment Manufacturer

OPP

Operating Performance Point

OSPI

Octal Serial Peripheral Interface

OToC*OEM-signed Table of Contents***OTP***One Time Programmable*

P

PCB*Printed Circuit Board***PCM***Pulse Code Modulation***PD***Power Domain***PDM***Pulse Density Modulation***PE***Protection Extension***PLL***Phase-Locked Loop***PMU***Performance Monitoring Unit***POR***Power-On-Reset***PPI***PHY Protocol Interface / Private Peripheral Interrupt***PPS***Precise Positioning Service***PPU***Power Policy Unit***PSC***Power Sequence Controller***PSM***Power Saving Mode*

PSRAM

Pseudo-Static Random-Access Memory

PSRR

Power Supply Rejection Ratio

PWM

Pulse Width Modulation

Q

QEC

Quadrature Encoder Counter

R

RAI

Release Assistance Indication

RDC

Receiver Delay Compensation

RF

Radio Frequency

RFI

Radio Frequency Interference

RLE

Run-Length Encoding

RMA

Return Merchandise Authorization

RNN

Recurrent Neural Network

RO

Read Only

ROM

Read Only Memory

RoT

Root-of-Trust

RSA*Rivest–Shamir–Adleman***RSE***Region Size Extension***RSSI***Received Signal Strength Indicator***RSTC***Reset Controller***RTC***Real-Time Counter***RTOS***Real-Time Operating System***RTSS***Real-Time Subsystem***RW***Read/Write*

S

SAR*Successive Approximation Register***SAU***Security Attribution Unit***SBI***Slave Bus Interface***SCU***Snoop Control Unit***SDA***Serial Data***SDIO***Secure Digital Input/Output***SDMMC***Secure Digital / Embedded Multimedia Card*

SDR

Single Data Rate

SE

Secure Enclave / Secure Enable

SESS

Secure Enclave Subsystem

SFD

Start of Frame Data / Start Frame Delimiter

Sgi

Software Generated Interrupt

SHA

Secure Hash Algorithm

SIM

Subscriber Identity Module

SIMD

Single Instruction Multiple Data

SJW

Synchronization Jump Width

SMP

Symmetric Multi-Processing

SNR

Signal-to-Noise Ratio

SPA

Simple Power Analysis

SPI

Serial Peripheral Interface / Shared Peripheral Interrupt

SRAM

Static Random-Access Memory

SSP

Synchronous Serial Protocol

SST

Single Shot Transmission

STB

Store Buffer

STOC

System Table of Content

SWD

Serial Wire Debug

SWRM

Software Reference Manual

T

TCM

Tightly-Coupled Memory

TCP

Transmission Control Protocol

TDC

Transmitter Delay Compensation

TE

Translation Extension

TEE

Trusted Execution Environment

TFT

Thin-Film-Translator

TGU

TCM Gate Unit

TOC

Table of Content

TRNG

True Random Number Generator

TSENS

Temperature Sensor

U

UART

Universal Asynchronous Receiver/Transmitter

UDE

Unprivileged Debug Extension

UDP

User Datagram Protocol

UI

Unit Interval / User Interface

UL

Upload

UPP

Ultra-Low Performance Point

USB

Universal Serial Bus

UTIMER

Universal Timer

V

VTOR

Vector Table Offset Register

W

WDT

Watchdog Timer

WFE

Wait For Event

WFI

Wait For Interrupt

X

XIP

eXecute-in-Place

XO

Execute Only

Z

ZI

Zero Initialized

14 Revision History

Table 14-1 Revision History

Date	Revision	Description
December 2024	1.8	Added section Section 12 Zephyr Alif SDK (ZAS)
October 2024	1.7	Changes from previous revision include: ■ Updated Figure 2-16 Cold Boot Sequence ■ Updated Section 3.2.1.1 Initialization ■ Updated Section 3.2.2.3 Certificate Chain Location and Identification ■ Removed Section 3.6.4 Additional Information and merged the updated content into Section 3.1 SESS Introduction ■ Updated Figure 4-3 CMSIS-Pack Window ■ Clarified throughout the document that the DevKit information also relates to the AppKit ■ Added links to relevant documentation and software from the Alif Semiconductor website throughout the document ■ Applied various editorial alignments
September 2023	1.6	Changes from previous revision include: ■ Updated CMSIS packs version in Section 6.2 Create or Import Project Files into the IDE ■ Updated Section 7.1.6.3 OSPI with AES ■ Applied various editorial alignments
June 2023	1.5	Changes from previous revision include: ■ Updated Table 2-2 Memory Map for E7 Device ■ Added Section 2.8 Power Management ■ Added clarification for Figure 4-3 CMSIS-Pack Window ■ Updated Section 5 Integrated Design Environments and Toolchains ■ Updated Section 7.1.1.3 IPC Topology Using MHU ■ Applied various editorial alignments ■ Updated all URLs to Alif Semiconductor documentation and products
April 2023	1.4	Changes from previous revision include: ■ Changed DeviceConfig tool name to Conductor and updated the corresponding URLs throughout the document ■ Updated Section 2.8.1 Boot Process ■ Updated Section 3.2.1.1 Initialization ■ Updated Section 3.2.2.1 Locate STOC ■ Updated Section 3.2.4 Error and Panic Values ■ Updated Section 3.5.1 SEROM UART Banner Message ■ Updated Section 6.1.2 Production Device Configuration Using the Conductor Tool ■ Updated Section 7.1.6.1 OSPI Overview and Section 7.1.6.2 OSPI Design ■ Updated Section 8.2 Memory Architecture Used by the APSS ■ Updated Section 8.3 APSS Boot Up and Figure 8-4 APSS Boot Up Events ■ Updated various Alif Semiconductor's URLs throughout the document ■ Replaced Arm DS references with generic IDE references throughout the document ■ Applied various editorial alignments

Date	Revision	Description
December 2022	1.3	Initial release